universität innsbruck

Dissertation

Proof-of-Work Consensus

Protocol Design, Security, and Incentives

Patrik Keller

advised by
Univ.-Prof. Dr.-Ing. Rainer Böhme

August 2025

Abstract

Cryptocurrencies are distributed systems that strive to provide the functions of money without relying on central authorities or third parties. Many cryptocurrencies, most notably Bitcoin, use proof-of-work to authorize participation: anyone can participate in the system, but in order to do so, they have to solve computational puzzles. The security of such systems is layered: the network layer ensures that the participants can communicate, the consensus layer coordinates state updates, the application layer enforces the semantics of the cryptocurrency, and the incentive layer motivates participation by assigning cryptocurrency-denominated rewards. Crucially, the incentive layer affects the security of the consensus layer: the consensus protocol assumes that the majority of participants are honest, so the incentive layer should motivate *honest* behavior.

With my dissertation, I aim to advance the state-of-the-art for proof-of-work consensus protocols, addressing both their immediate security properties on the consensus layer and the secondary effects introduced by the incentive layer. This includes both the improvement of analytic methodology and the design of new protocols. These two aspects—design and analysis—are inherently intertwined: novel protocol designs usually demand new analytic tools, while the constraints of existing analytic methods shape the scope of viable protocol designs.

My dissertation consists of three published and peer-reviewed papers, each making their own contributions towards these goals. The first paper explores how the security on the consensus layer can benefit from mandating multiple puzzle solutions per block, whereas Bitcoin has only one. This results in the proposal and analysis of the Parallel Proof-of-Work consensus protocol. The second paper explores how the enhanced security of Parallel Proof-of-Work can be balanced with the requirements of the application and incentive layers. This results in the proposal and analysis of the Tailstorm cryptocurrency, which uses a novel incentive mechanism that discounts rewards to punish malicious behavior. The third paper explores how the incentive analysis of Tailstorm can be generalized to a broad range of proof-of-work consensus protocols. This results in an analytical framework that finds optimal incentive attacks, automatically, given a concise protocol specification.

Contents

1 Introduction **1**

2 Background **4**
2.1 Distributed Systems 4
2.2 Proof-of-Work 5
2.3 Bitcoin 7
2.4 Alternative Protocols 13

3 My Research **15**
3.1 Motivation and Scope 15
3.2 Methodology 16
3.3 List of Publications 19
3.4 Synthesis 22
3.5 Future Work 23
3.6 Limitations 25
3.7 Conclusion 26

References **26**

My Publications **32**
Paper A: Parallel Proof-of-Work with Concrete Bounds 32
Paper B: Tailstorm: A Secure and Fair Blockchain for Cash Transactions 63
Paper C: Automated Selfish Mining Analysis for DAG-Based PoW Consensus Protocols . 90

1 Introduction

I submit this dissertation in partial fulfillment of the requirements for the degree of Doctor of Philosophy in the discipline of Computer Science at the Leopold-Franzens-Universität Innsbruck, under the supervision of Prof. Rainer Böhme, head of the Security and Privacy Lab at the same institution. The core of this work consists of three peer-reviewed articles, each accepted for publication in the proceedings of acknowledged international conferences. To meet the academic requirements, I demonstrate here that these articles are thematically and methodologically interconnected, and embed them within a broader context: I introduce the field of research and its current state of knowledge, present the overarching research question, detail my contributions, and critically evaluate the results. Additionally, I outline open challenges, discuss the limitations of my methodology, and explore potential directions for future work.

Focus and Context My research is focused on the design, security, and incentives of proof-of-work consensus protocols, as they are used in cryptocurrencies like Bitcoin. Cryptocurrencies are distributed systems that are designed to provide the functions of money—medium of exchange, store of value, and unit of account—without coordination of a central party or nation-state authority. From a user's perspective, the ideal cryptocurrency needs to solve three key problems: clearly establishing who owns what at all times, fast and reliable processing of transactions, and controlling the creation of new currency to prevent arbitrary inflation. Achieving these goals in a decentralized setting requires coordinating all participants, some of whom may have conflicting interests. This coordination is the fundamental challenge addressed by the consensus protocol.

A Layered View on Cryptocurrencies To understand the architecture of cryptocurrencies, it is useful to decompose them into a stack of four layers, as shown in Figure 1.

The users of the cryptocurrency interact with the *application layer*: they hold currency and initiate transfers by creating and submitting transactions, with the expectation that the system will reliably confirm them.

Beneath this is the *consensus layer*. Here, a distributed group of nodes listens for new transactions, as they are submitted by the users on the application layer, batches these transactions into blocks, and appends them to a shared, distributed data structure: the blockchain. The hand-off to the application layer is an ordered list of blocks that sequences all confirmed transactions. The desired property here is that the list of blocks is constantly growing, while the history remains immutable and consistent across all nodes. With these properties, the consensus layer abstracts the coordination between the distributed nodes: the application layer can treat the blockchain as if it were a local data structure.

Naturally, both the application layer and the consensus layer require a medium of communication: cryptocurrency users want to submit their transactions to the nodes and the nodes must, at the minimum, exchange their blocks in order to agree on a consistent blockchain. This communication is provided by the *network layer*.

In my dissertation, I focus on such consensus protocols that use proof-of-work to authorize participation: a node is only permitted to propose a new block after it has successfully solved a computational puzzle. A key aspect of this design is that it is *permissionless*: anyone can connect

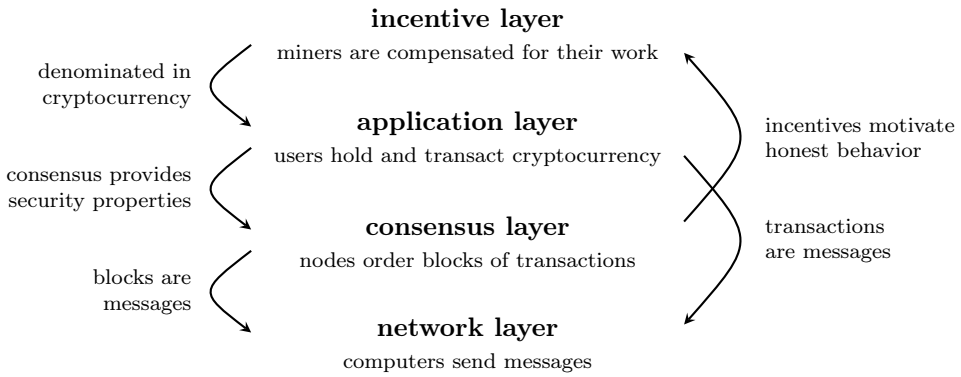


Figure 1: Cryptocurrency protocol stack: layers with primary functions; arrows mark dependencies.

a computer to the network, run the node software, and begin solving puzzles to participate in the consensus protocol, without authenticating with the other nodes or a third party.

This raises a critical question: why *should* anyone dedicate computational resources to this task? This is where the *incentive layer* becomes essential. To motivate participation, the system rewards the nodes based on the puzzles they solve and the blocks they contribute to the chain. These rewards are given in the form of new currency, created at the application layer and granted to the nodes participating at the consensus layer. From this perspective, nodes work to obtain economic rewards. Drawing an analogy to the costly extraction of precious metals, the nodes are called *miners*, the puzzle solving is called *mining*, and the creation of currency is called *minting*.

The layered view on cryptocurrencies in Figure 1 also guides the understanding of security. Network layer security is about ensuring that nodes and users can reliably communicate their blocks and transactions. Consensus layer security guarantees useful properties for the blockchain given suitable assumptions about the network layer. Application layer security guarantees the semantics of the cryptocurrency given the security properties of the blockchain.

Security of the Consensus Layer My focus lies on the security of the consensus layer. A secure blockchain consensus protocol is expected to provide three key properties, even if some nodes ignore the protocol's rules and act maliciously:

1. **Consistency:** All honest nodes must agree on the ordering of blocks. This ensures a single, immutable history of transactions and agreement on who owns what on the application layer.
2. **Progress:** The blockchain must not stop growing, meaning new blocks are continually appended. This ensures that the application does not halt.
3. **Chain quality:** A certain fraction of new blocks must be contributed by honest participants. This prevents that malicious miners perpetually censor some of the transactions.

To guarantee these properties, two assumptions are required:

1. **Bounded Propagation Time:** New blocks, after being shared by one honest node, must propagate to all other honest nodes within bounded time. This ensures that all honest nodes have a reasonably synchronized view of the blockchain.
2. **Honest Majority:** The majority of nodes must follow the protocol. Since participation is authorized via proof-of-work, the majority is measured in terms of computational resources.

A reasonable (and required) majority threshold is that one half of the puzzle solving power is controlled by honest miners, although some protocols and properties might require stricter thresholds.

Challenges at the Intersection of Security and Incentives The honest majority assumption stands in stark contrast to the permissionless nature of the system, where anyone can connect their machines and begin mining, and the earlier observation that some of the miners might be primarily motivated by the economic rewards of the incentive layer. This creates two critical second-order problems, where the properties of the incentive layer might affect the security of the consensus layer.

1. **Fairness:** Rewards for mining should be proportional to the computational resources a miner contributes. Any unfair bias violating that rule creates an economic incentive to consolidate mining power. Smaller miners would be driven to join larger pools to benefit from this bias, gradually centralizing the control over the system. If left unchecked, this might lead to a situation where a single entity exceeds the security threshold, effectively becoming a single point of failure and undermining the premise of decentralization.
2. **Incentive Compatibility:** No behavior should be more profitable than following the protocol's rules. Otherwise, if dishonest behavior is more lucrative than honest participation, the honest majority assumption becomes difficult to justify: rational, profit-seeking miners would gravitate towards dishonest participation, making the assumption that a majority will act honestly not just a theoretical concern, but also an irrational expectation.

Research Goals With my dissertation, I aim to advance the state of the art for proof-of-work consensus protocols, addressing both their immediate security properties and the secondary effects introduced by the incentive layer. Specifically, I focus on two interrelated goals: improving the methodology for analyzing security and incentives, enabling the evaluation of a broader range of protocols and properties, and designing new consensus protocols and incentive mechanisms that better align with the desired goals. These two aspects—design and analysis—are inherently intertwined: novel protocol designs often demand new analytic tools, while the constraints of existing analytic methods shape the scope of viable protocol designs.

Research Questions

RQ1: What novel consensus protocols and incentive mechanisms can be designed to enhance the security, fairness, and incentive compatibility of proof-of-work cryptocurrencies?

RQ2: How can existing methods for analyzing the security, fairness, and incentive compatibility be improved to match a broader range of proof-of-work consensus protocols?

Structure The remainder of my dissertation is structured as follows. The next section provides essential background and sets the stage for my research: Section 2.1 relates to the fields of distributed systems and Byzantine fault tolerance; Section 2.2 explains the distinctive aspects of proof-of-work; Section 2.3 presents the canonical cryptocurrency system Bitcoin, including its security guarantees and weaknesses; and Section 2.4 summarizes alternative designs aimed at addressing these weaknesses. The third section focuses on my own research contributions: Section 3.1 motivates the research and

defines its scope: Section 3.2 outlines my methodology for the stochastic modeling of proof-of-work; Section 3.3 lists my three papers, including summaries and contribution statements for co-authored works; Section 3.4 synthesizes and critically assesses the contributions of the papers as a whole; Section 3.5 discusses potential applications and extensions of my methodology; Section 3.6 identifies fundamental limitations of my approach; and Section 3.7 concludes. Lastly, I fully reproduce my three published, peer-reviewed papers, which form the foundation of my dissertation.

2 Background

2.1 Distributed Systems

A distributed system is a network of computers, called *nodes* or *participants*, that work together to achieve a common goal. While each node has its own local state and operates independently, they all follow the same algorithm, called *protocol*. Distributed systems research is about designing protocols that enable the nodes to reach their common goal—even when facing issues like message delays, network partitions, node failures, and attacks. This involves specifying the goals, quantifying the stress factors, and analyzing worst-case implications.

Reliability and Security Stress can be quantified along two main axes: reliability and security. For reliability, one typically assumes a fail-stop model, where nodes may stop communicating but otherwise behave correctly. Reliable protocols tolerate a certain number of node failures—due to maintenance, hardware issues, or network outages—without compromising overall system functionality. Stress is usually measured by comparing the numbers of *faulty* and *correct* nodes.

Security, on the other hand, considers arbitrary faults. Instead of simply becoming unresponsive, faulty nodes may deviate from the protocol, send incorrect, conflicting messages, and withhold information. Going back to a work of Lamport, Shostak, and Pease [49], these failures are often called Byzantine faults, and the field of study is known as Byzantine fault tolerance (BFT).

This dissertation takes the security perspective: I assume that correct nodes always follow the protocol and do not suffer from benign faults. Faulty nodes may behave arbitrarily. To emphasize the attack scenario, I call correct nodes *honest* and failing nodes *adversarial* or *dishonest*.

Reliable Broadcast and Total Order Broadcast Two classic problems in distributed systems research are reliable broadcast, getting messages to all nodes, and total order broadcast, getting messages to all nodes and agree on their order. More formally, reliable broadcast (RB) is about satisfying the following three properties:

1. RB-Reliability: If an honest node RB-sends a message, then all honest nodes eventually RB-deliver this message.
2. RB-Agreement: If an honest node RB-delivers a message, then all honest nodes eventually RB-deliver this message.
3. RB-Integrity: Honest nodes RB-deliver any message at most once; and they can correctly identify which node RB-sent each message.

Note that I distinguish between receiving and delivering messages. Receiving is passive: the message arrives from the network. But fulfilling the properties requires taking action about which messages

are delivered and when. For example, a single message might be received multiple times, even in a benign setting, after forwarding across different nodes. To achieve RB-Integrity, the recipients can (and should) decide to deliver the message at most once.

The distinction between RB-Reliability and RB-Agreement is subtle but important. Reliability ensures that messages sent by *honest* nodes reach all other honest nodes. Agreement, in contrast, includes the messages sent by *dishonest* nodes and ensures that they are forwarded correctly. The properties do not make any assumptions about message delivery by dishonest nodes—they can do whatever they want.

Total order broadcast (TOB) extends reliable broadcast with a fourth property:

1. TOB-Reliability: If an honest node TOB-sends a message, then all honest nodes eventually TOB-deliver this message. = RB-Reliability
2. TOB-Agreement: If an honest node TOB-delivers a message, then all honest nodes eventually TOB-deliver this message. = RB-Agreement
3. TOB-Integrity: Honest nodes TOB-deliver any message at most once; and they can correctly identify which node TOB-sent each message. = RB-Integrity
4. TOB-Order: If an honest node TOB-delivers message m' after message m, then any honest node TOB-delivers m' after is has TOB-delivered m.

Total order broadcast is sometimes called atomic broadcast, the terms are largely synonymous [20]. The ordering of messages is vital for a wide range of distributed systems, including the distributed execution state machines [48]. Blockchains and cryptocurrencies rely on the ordering of messages as well, but the ordering is established with a new kind of algorithm.

2.2 Proof-of-Work

The traditional fail-stop and BFT models assume a fixed number of participants and authenticated pairwise communication links. This allows to count participants and conduct majority votes—at the expense of knowing and limiting the participants in advance. Proof-of-work consensus protocols avoid this limitation. The trick is to restrict participation based on computational resources instead of counting nodes. This section gives a short overview of the history of this approach.

Moderately Hard Puzzles Computational puzzles are tasks that are hard to solve while the solution is easy to verify. Moderately hard means the difficulty can be adjusted to match the available solving power. This idea goes back to the 1990s, when researchers proposed to use moderately hard puzzles to control spam and prevent resource abuse in systems with unidentified senders [7, 8, 22, 34]. The puzzle slows down the sending side to reduce load on the receiving side. Difficulty moderation allows the receiver to adjust to demand and advancements to the puzzle solving itself.

One widely adopted puzzle type—and the one I focus on in this dissertation—is based on the partial inversion of cryptographic hash functions [7, 8, 34]. The receiver specifies parts of the hash function input and sets a condition for the output, e. g., a certain number of leading zeros. The sender's task is to adjust the remaining parts of the input—sometimes called *nonce*, short for number used once—until the output condition is met. Fresh puzzles can be generated by altering the preconfigured part of the input. Difficulty adjusts based on the output condition, e.g., fewer or more

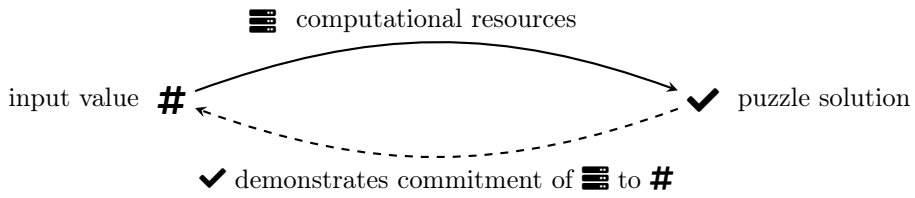


Figure 2: Proof-of-work is a verifiable commitment of computational resources to a given value.

leading zeros. To find a solution, the sender has to evaluate the hash function many times. The receiver, on the other hand, can quickly verify the solution by applying the hash function once.

Proof-of-Work as Commitment Nowadays, the moderately hard puzzles that are based on the partial inversion of hash functions are usually called *proof-of-work*. A useful interpretation of proof-of-work is that it serves as a commitment of computational resources to a given input value: In order to find a solution, one first has to set the input value and then apply the hash function many times to find a valid nonce. After changing the input value, the solution becomes invalid and one has to start searching from scratch. This binds the resources spent on hash function evaluation to the input value. I illustrate this interpretation in Figure 2.

Cryptocurrency An early application of proof-of-work is Wei Dai's b-money [17, 18] where users can issue their own currency by solving moderately hard puzzles. The puzzle inputs are chosen such that the puzzle solution binds the issued coins to a public key, chosen by the issuer. To send money, users submit signed transaction statements that assign the involved funds to other public keys by broadcasting them to the other users. Signing the transaction statement requires access to the private key of the current owner. As a result, only the current owner can unlock funds for transfer.

An inherent problem here is double spending: While honest users never attempt to spend the same funds twice, malign users can easily do this. To be of any use, the system has to reject such conflicting transactions. To find agreement on which transactions to accept and which to reject, b-money relies on a special broadcast primitive that I call cryptocurrency broadcast (CCB):

1. CCB-Liveness: if an honest user CCB-submits a transaction, then all honest users eventually CCB-accept this transaction.
2. CCB-Consistency: if an honest user CCB-accepts a transaction, then all honest users eventually CCB-accept this transaction.

Consistency is to avoid double spending: all honest users accept the same transactions. Liveness ensures that all benign transactions—the ones that are not part of a double spend attack—get processed.

CCB-on-TOB Dai [17, 18] observed that CCB can be implemented using TOB: To CCB-submit a transaction, users TOB-send it. CCB-transactions become TOB-messages. On the receiving side, users attempt to CCB-accept all transactions that are TOB-delivered. But before CCB-accepting a transaction, users check whether a conflicting transaction has been CCB-accepted before. If so, they discard the delivered—but not yet accepted—transaction.

With this simple mechanism, CCB-Liveness follows from TOB-Reliability: As honest users never issue conflicting transactions, their CCB-submitted transactions pass the filter on TOB-deliver and will eventually be CCB-accepted. CCB-Consistency follows from TOB-Order: All nodes TOB-deliver the transactions in the same order, hence all users CCB-accept the same set of transactions.

Permissionless Systems Dai proposed b-money in 1998 [17, 18]. TOB algorithms have been around at that time but they did not suit Dai's needs. Dai aimed to create a currency system where users operate under pseudonyms rather than real names, and where any internet user could participate without needing authorization from a central authority or other users [17]. The BFT model is at odds with these goals: The number of participants is configured up front and the nodes can identify each other—either by relying on trusted pairwise communication channels or a cryptographic key setup [2]. In this sense, BFT systems are inherently *permissioned* [2]. Dai envisioned b-money to be *permissionless*.

Due to the lack of permissionless TOB algorithms, Dai proposed to order the transactions by posting them to a public discussion board like Usenet. This of course is a trade-off: It shifts trust to third parties, who could then engage in de-anonymizing the pseudonymous users, censoring transactions, and reordering messages to facilitate double spending.

2.3 Bitcoin

Satoshi Nakamoto addressed this issue in 2008 [51]: For his cryptocurrency Bitcoin, he proposes an algorithm that orders b-money style transactions in a permissionless manner. Instead of counting nodes and conducting majority votes, like it is prevalent in BFT systems, Bitcoin rate-limits participation using moderately hard puzzles. Proof-of-work becomes dual-use: The puzzle solution does not only issue new coins, but it also gives the issuer a say in the underlying consensus algorithm that orders the cryptocurrency transactions.

Protocol Cryptocurrency users CCB-submit their transactions, by broadcasting them to the nodes of the consensus layer. The nodes batch the transactions into *blocks* and continually append new blocks to the *blockchain*. The blockchain is a distributed hash-linked list where each block contains a pointer (hash reference) to a single *parent* block. By resolving these pointers recursively, each block identifies a unique (up to hash collisions) sequence of blocks. This sequence of blocks also defines which transactions have been included up to this point and in which order.

Appending new blocks requires a proof-of-work: the hash of any block (interpreted as integer) must be lower than a given *difficulty threshold*. This throttles write-access to the distributed data structure and gives the network time to synchronize the latest blocks before the blockchain continues to grow. To reach agreement on the sequence of blocks, honest nodes follow the *longest chain rule*: they append new blocks to the longest sequence of blocks they know.

To satisfy CCB-Consistency and CCB-Liveness, Bitcoin adds a few safeguards: when receiving blocks from the network, honest nodes reject blocks containing transactions that conflict with earlier transactions. Similarly, when proposing their own block, honest nodes never include conflicting transactions. Besides that, honest nodes do not discriminate among the transactions to include.[1]

[1] In practice, blocks have a size limit and the system has to deal with congestion: the nodes prioritize transactions based on user-defined transaction fees. I intentionally ignore this detail of the application layer in my dissertation.

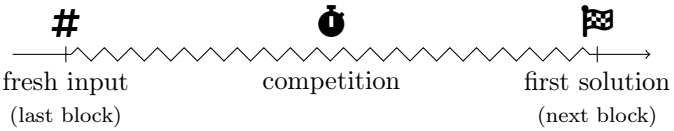


Figure 3: Bitcoin uses proof-of-work for authorization. Solving the puzzle for the next block is a competition among the miners. Only the winners of this competition may propose new blocks and advance the protocol execution.

In jargon, working to append new blocks is called *mining*. The nodes who engage in mining are called *miners* to contrast cryptocurrency *users* who observe the blockchain and submit new transactions. To offset the cost of mining, each block includes a *coinbase* transaction that documents the issuance of new currency to the successful miners. A block *confirms* its predecessors in the hash-linked list. The number of predecessors of a block, and thus the length of the associated chain, is called *height*. If two blocks have the same parent, this is called a *fork*. A fork is *resolved* when all miners agree on which *branch* of the fork they want to confirm—all other branches become *stale*. Unresolved forks are called *active* and forks that cause a permanent split, for example due to protocol update, are called *hard fork*. Blocks that are not confirmed by the longest chain are called *orphans*.

Proof-of-Work as Authorization Mechanism In Bitcoin, proof-of-work is not merely a commitment of resources but also a form of authorization. In order to proceed with protocol execution and extend the longest chain, miners have to present a puzzle solution. With every new block the puzzle is reset and amendments to the blockchain are blocked until the next puzzle is solved.

This authorization mechanism also creates a form of competition among the miners. Apart from propagation delays, the miners receive the new puzzle at roughly the same time. The miner who first finds a solution wins the competition and, by sharing the new block, also starts the next round of the competition. Figure 3 illustrates the concept of proof-of-work as authorization mechanism and the competition among miners. Note how Figures 2 and 3 depict fundamentally different interpretations of the same proof-of-work puzzle.

This form of authorization relies on the stochastic properties of the proof-of-work puzzle. Consider the contrary: a puzzle with a deterministic solving algorithm. In this case, the participant with the most computational power would consistently win, contribute all blocks, and claim all minted coins. This not only makes the system uninviting for weaker participants, it also creates a single point of failure regarding the censorship of transactions. Hash-based puzzles, however, exhibit inherent randomness. The time to find a solution is exponentially distributed. Each participant has a fair chance of success: the probability of winning is directly proportional to a participant's relative hash rate—their share of the total computational power measured in hashes per unit time.

Proof-of-Work as Leader Election In the BFT consensus protocol literature, leader election is described as mechanism to optimize the protocol's performance in the optimistic case: the designated leader coordinates protocol execution, the other participants check up on the leader, and—in case of misbehavior—a new leader is elected [15, 61].

Taking inspiration from BFT research, proof-of-work is often described as a leader election mechanism as well: the miner who first solves the puzzle for the next block is elected as leader. This leader has the right to order the most recent transactions in his block proposal. The other miners

check up on the leader—whether he included only valid transaction in his block—and if so, reset the puzzle and start the next leader election for the upcoming block.

While superficially the analogy seems to hold, I also want to point out a few problems. First, in Bitcoin the leader is elected temporally, for one block proposal, and then replaced, even if the block proposal was okay. This stands in contrast to BFT style leader election, where leadership is sticky as long as the leader behaves correctly and leader replacement involves elaborate sub-protocols that should largely be avoided during ordinary execution. Second, in BFT-style protocols, the leader election mechanism usually results in agreement on the next leader. In Bitcoin, the agreement is not immediate: While there are active forks, there are also multiple competing leaders and even the history of leaders might be unclear. This ambiguity resolves only in the long run, as a part of the consensus protocol itself.

Due to these gaps in the analogy, I prefer to describe proof-of-work not as a leader election mechanism, but as a way to authorize participation. Those miners who solve the puzzle may proceed with execution—but only for a limited number of steps.

Note that proof-of-work *can* be used as leader election. One protocol doing this is Bitcoin-NG [23], where the last successful miner is allowed to append additional blocks *without* proof-of-work. My Parallel Proof-of-Work protocol, as presented in Paper A, uses a leader election mechanism as well.

A Single Puzzle is not Reliable The hash-based puzzles ensure that all miners get a fair chance for the next block. The exponential distribution, however, is also quite chaotic. Already in the original context, preventing denial of service attacks with moderately hard puzzles, it was observed that solutions can be found much faster than expected, and that replacing a single difficult puzzle with multiple easier ones, makes the outcome more predictable [34].

In Bitcoin, the expected time for solving a single puzzle, and hence the expected time between blocks, is 10 minutes. Nevertheless, there is a 1 % chance that the puzzle is solved within 6 seconds. The probability of solving two 10-minute puzzles within twice the time (12 seconds), however, is only about 0.02 %. Solving ten puzzles within 60 seconds (10x) happens with probability about 3×10^{-17}—practically negligible.

The same principle applies to the distribution of who solves the puzzle first, and hence contributes the next block to the chain. A miner who operates 30 % of the hash rate is expected to mine 3 out of 10 consecutive blocks. However, there is a 5 % chance of mining 6 out of 10 blocks. Increasing the observation window mitigates that risk. The chance of mining 12 out of 20 blocks (2x) is about 0.5 %. The chance of mining 60 out of 100 blocks (10x) is only about 5×10^{-10}.

Double Spending Attacks To launch double spending attacks in Bitcoin, the adversary creates two transactions that both spend the same coins. The *public* transaction assigns the funds to the legitimate recipient and is shared with the rest of the network. The *private* transaction assigns the funds to the adversary themselves and is not shared with the other miners. While the honest miners work to include the public transaction in their version of the blockchain, the attacker creates a fork that confirms the private transaction. If the attacker manages to grow their private chain longer than the public chain, they can replace the public chain by releasing the private chain, relying on the longest chain rule, and the double spend attack succeeds: the funds are not assigned to the legitimate recipient but remain with the adversary.

Whether such an attack succeeds depends on the attackers puzzle solving power, the number of confirmations before the legitimate recipient accepts the payment as final, and the realizations of the random puzzle solving delays, in particular, how many blocks are mined by the attacker and how many by the honest miners. Nakamoto [51] showed in his initial report on Bitcoin that for an attacker with less than 50 % of the hash power, the success probability decays exponentially with each additional confirmation required by the recipient. Subsequent work [55] provides some guidance on how to choose the number of confirmations for a given transaction value, but Bitcoin leaves the final choice to the user.

From Attack Analysis to Security Proofs Nakamoto and subsequent works focus on a single attack strategy for double spending [35, 51, 55]. The first rigorous security proof followed in 2015 [26, 27]: taking inspiration from BFT-style security analysis, Garay et al. define security properties, formalize their assumptions, and then prove that Bitcoin meets those properties under those assumptions. In other words, their analysis covers worst-case scenarios and a wide range of attack strategies. For my thesis, the work of Garay et al. is of particular interest because they formalize the security properties of blockchain consensus.

Blockchain Consensus Garay et al. [26, 27] formalize Bitcoin's core consensus protocol in the so-called *Bitcoin Backbone Protocol*. This protocol aims to maintain a single, growing sequence of blocks—the blockchain—in a decentralized network. I refer to this task as *blockchain consensus* (BC), and summarize the security goals as follows:

1. BC-Progress: the blockchain grows continually.
2. BC-Quality: a certain fraction of new blocks is contributed by honest miners.
3. BC-Consistency: once a block is appended, it cannot be reordered or removed.

BC is the foundation of the Bitcoin cryptocurrency: its ordering serves as the basis for CCB.

CCB-on-BC Bitcoin implements CCB on top of BC: To CCB-submit transactions, cryptocurrency users broadcast them to the miners, who then include them in their new blocks. The miners verify all transactions and ensure that no coin is spent twice. Offending transactions and blocks that confirm such transactions are ignored. Besides that, honest miners do not discriminate among the transactions or against particular cryptocurrency users.[2] The cryptocurrency users CCB-accept all transactions that are included in the blockchain, after they have accumulated enough confirmations.

Bitcoin's application layer is designed to guarantee the CCB-properties given that the BC properties hold. The proof can be sketched as follows: Assuming BC-Progress and BC-Quality, every valid transaction will eventually appear in some honest block, yielding CCB-Liveness. Assuming BC-Consistency, all honest nodes agree on the sequence of blocks and hence the set of transactions, giving CCB-Consistency.

Assumptions in the Backbone Protocol To proof the BC security properties for the Bitcoin Backbone Protocol [26, 27], Garay et al. make two assumptions. First, they assume an honest majority among the miners. Unlike in the BFT setting, where the majority is defined by setting a

[2] As mentioned before, I ignore the block size limit and the prioritization based on transaction fees.

threshold on the number of dishonest nodes, they assess the majority in terms of relative mining power (or hash rate) [2]. Second, they assume a synchronous network, where block broadcasts—from one honest miner to all other honest miners—complete within a fixed message propagation delay Δ.

Eventual Security In alignment with the earlier observation that individual proof-of-work puzzles are unreliable, the security properties of Bitcoin emerge only after observing enough puzzles. Garay et al. prove in 2015 that the error probabilities for BC-Progress and BC-Quality shrink exponentially as the observation window grows [26]. They also show that the error probability for BC-Consistency decays exponentially when users wait for additional confirmations before considering a block's transactions as final [26]. Subsequent works extend these results to more relaxed communication assumptions [27, 52], simplify the proofs [27], and derive tight bounds on the maximum adversarial hash rate and message delays the system can tolerate before failure [21, 28]. Nonetheless, all these results are asymptotic, guaranteeing security only in the long run, as the observation window grows.

Concrete Bounds In 2021, Li, Guo, and Ren [50] present closed-form expressions for the remaining consistency-failure probability as a function of the number of confirmations k. These formulas can be inverted: given a desired failure probability, users can solve for the minimum k that meets the goal. This development transforms Bitcoin's eventual security into actionable guidance for setting confirmation policies depending on the users' risk tolerance. In the following years, the proof was simplified [32] and the bounds tightened [29].

Asynchrony Subverts Security In Bitcoin, every new block must reference the latest block. But network propagation delays introduce a brief window of uncertainty after each mining event. During this period, miners may unknowingly extend a stale chain, leading to the creation of orphaned blocks (comp. Figure 4). These orphans represent more than just wasted effort; they reveal how network latency weakens the honest majority's ability to maintain their dominance. As propagation delays increase—or as the time between blocks decreases—the effective hash rate of honest miners drops and the protocol's resistance to attacks erodes [26–29, 32, 50, 52]. This dynamic imposes practical constraints: larger blocks take longer to propagate, increasing the window of asynchrony, and with it, the risk of orphans and inconsistency.

Interestingly, recent work suggests that a moderate orphan rate is not necessarily harmful, and can even be tolerated to improve time-to-finality [29, 32, 50]. By accepting a higher orphan rate under shorter block intervals, one can reach stronger consistency guarantees in the same expected time—challenging the traditional view that all lost work is purely detrimental. But ultimately, there remains a fundamental trade-off: properties that improve security—such as small blocks and long intervals—come at the cost of throughput, while efforts to scale the system, e.g., by increasing block size or frequency, may inadvertently weaken security.

Difficulty Adjustment In permissionless systems like Bitcoin, the computational power fluctuates as miners come and go or improve their hardware [60]. To maintain a steady block rate, Bitcoin adjusts the mining difficulty every 2016 blocks based on how long the previous set took to mine. This estimate is derived from untrusted block timestamps and smooths out noise and manipulation by averaging over a large window. The recalculated difficulty aims to preserve the 10-minute block



(a) **Small** propagation delay Δ. (b) **Big** propagation delay Δ.

Figure 4: The effect of network delays in Bitcoin. Arrows indicate hash-references to parent blocks; the horizontal axis represents time. If message propagation delays are short (small Δ, left), miners can reference the last block most of the times. If the propagation delays are long (big Δ, right), the latest block might not be fully propagated when the next block is mined. In that case, only one of the blocks will be confirmed in the longest chain, the other will be orphaned.

interval. Without this feedback loop, rising hash power would accelerate block production, potentially harming consistency, while a drop would stall progress.

Incentives To offset the costs of mining, Bitcoin assigns a reward the miner of each block. Given the stochastic properties of proof-of-work as competition (see above), this implements a lottery, where the expected prize is proportional to the total work expended.

The mechanism for delivering the prize to the miner is as follows: Each block begins with a special transaction, the *coinbase*, which may create a limited amount of new currency and assign it to an arbitrary address. Each miner constructs their own block proposal and sets the coinbase transaction to pay the reward to themselves. Note however, that the assignment only takes effect if the block will be confirmed as part of the longest chain.

"Monetary Policy" The Bitcoin protocol gradually reduces the block reward over time, halving it every 210 000 blocks. Starting from an initial reward of 50 BTC, this geometric reduction continues until new coin issuance effectively stops, capping the total supply at 21 million coins. This built-in monetary policy ensures a fixed supply and predictable issuance schedule.

Transaction Fees In addition to minting new coins, the coinbase may also collect funds voluntarily offered by users. To incentivize timely inclusion in a block, transactions can specify a fee. This fee can be reassigned by the coinbase of the block that includes the transaction. Since the miner controls the coinbase, they receive the fee. In the long run, as the block reward declines to zero, transaction fees will remain the sole direct incentive for miners in Bitcoin. Nevertheless, I chose to ignore transaction fees in all three papers.

Unfairness in Benign Networks An incentive scheme is fair, if it rewards miners in proportion to the puzzle solving power they contribute to the system. Bitcoin exhibits a structural unfairness. This stems from block propagation delays: the winning miner knows the block immediately and can start solving the next puzzle without delay. In contrast, all other miners must first receive and validate the new block before they can start mining. This gives the previous winner a head start in the next round. Over time, this effect compounds: strong miners, who already win more often, also benefit more frequently from this early access. This bias systematically disadvantages weaker miners

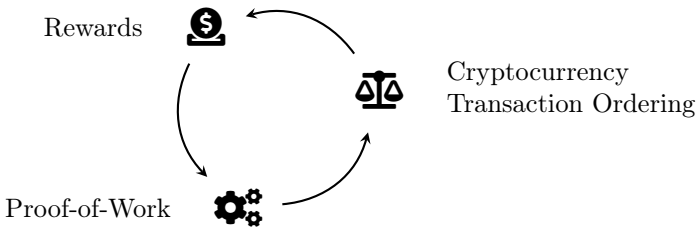


Figure 5: Cyclic dependency in proof-of-work cryptocurrencies: the system issues rewards to compensate mining; the rewards are denominated in cryptocurrency; the cryptocurrency relies on the transaction ordering provided by the proof-of-work consensus algorithm.

and undermines proportional fairness [4]. The issue is inherent to the protocol's design and persists even under benign network conditions.

Incentive Compatibility BFT-style security assumes that a limited fraction of participants behaves maliciously [2]. The same assumption underlies the security proofs for Bitcoin—not in terms of participants, but in terms of hash rate. Yet, as Bitcoin attracts participation through a distributed lottery, it is natural to adopt an economic perspective: are miners here to maximize rewards, rather than to follow the rules?

This framing reveals a cyclic dependency, illustrated in Figure 5: the protocol uses rewards to incentivize honest behavior, but those rewards retain value only if the system remains secure. Security, in turn, assumes that most participants act honestly and follow the protocol as intended. But in a permissionless setting, miners may behave rationally—maximizing profit even if it means deviating from the rules—and this puts at risk the security assumptions.

Ideally, rational behavior aligns with honest behavior. This is the notion of *incentive compatibility* [16, 33]: a protocol is incentive compatible if following the rules maximizes the expected reward. Bitcoin aspires to this ideal—but it does not achieve it.

Selfish Mining In 2014, Eyal and Sirer [24] showed that rational miners should deviate from the protocol if they aim to maximize long-term rewards. By selectively withholding newly mined blocks, a *selfish miner* can maintain a private fork of the blockchain. As long as the selfish miner manages to stay ahead of the public chain, they can strategically release their private blocks to override recent work by honest miners. This causes discarding of honest blocks and wasted mining effort—ultimately shifting a disproportionate share of rewards to the selfish miner.

The selfish mining strategy is profitable for miners controlling a third of the total hash rate or more [24]. If the miner can exploit network propagation delays—a standard assumption in BFT-style security—the threshold becomes even lower [24, 30, 56].

2.4 Alternative Protocols

Cryptocurrency users expect a responsive payment system: new transactions should appear on-chain quickly (low first confirmation latency), and they should be accepted shortly thereafter (fast time to finality). Both aspects of usability face technical limitations in Bitcoin.

Regarding time to finality, we have discussed in Section 2.3 that Bitcoin requires multiple confirmations to reduce the risk of inconsistency. To recap, this stems from the stochastic nature of the hash-based puzzle, which is required to build a fair lottery for mining rewards and give weaker miners a chance to contribute their own blocks.

First confirmation latency is fundamentally limited by the block interval: users must wait for the next block before their transaction can be included. One might think naively, that first confirmation latency can be improved by reducing the block interval. This however, as discussed above, also increases network asynchrony because it leaves less time for the existing blocks to propagate before the next block is mined. As asynchrony increases the risk of inconsistency, unfairness, and incentive incompatibility, optimizing first confirmation latency by simply reducing the block intervals will negatively affect security.

A related challenge arises with growing adoption: users expect their transaction to appear in the next block, but if demand exceeds capacity, some must wait longer. This throughput limitation can be addressed by increasing the block size. However, larger blocks take longer to propagate, which again increases asynchrony and reinforces the same threats: inconsistency, unfairness, and incentive incompatibility.

Considering these inherent trade-offs between usability and security in Bitcoin, it has become a fashion to propose alternative blockchain consensus protocols that enable more responsive and scalable cryptocurrency implementations, ideally without sacrificing security. Some of the most significant ideas are the following:

Non-Work Blocks One idea is to allow appending additional blocks of transactions, which themselves do not require a proof-of-work, between standard proof-of-work blocks. In the initial proposal [23], the miner of the last proof-of-work block could append these intermediate blocks unilaterally, without additional checks and balances. Later protocols introduced BFT-style consensus among participants to collectively decide which intermediate blocks are accepted [3, 19, 46, 54]. A key challenge in this approach is to synchronize the two consensus layers—BFT and permissionless proof-of-work—and their differing assumptions.

Parallel Chains Another approach involves running multiple Bitcoin-like blockchains in parallel, with auxiliary protocols to synchronize their state and resolve conflicts [9, 62]. This represents a form of scaling out—adding parallelism—rather than scaling up by increasing block size or frequency.

Multiple Puzzles per Block Some protocols split the proof-of-work of each block into multiple smaller parts [12, 53]. This design is motivated by the observation that the security, consistency, and fairness properties of proof-of-work emerge only when multiple puzzles are considered together. By including several independent puzzles within a single block, these protocols aim to achieve the same statistical guarantees that Bitcoin can only offer in the long run, after many blocks.

DAG-Structured Blockchains Bitcoin's blockchain is linear: each new block must reference the most recent one. This introduces ambiguity during propagation, leads to orphaned blocks, and gives bigger miners an unfair advantage in the competition for the next block.

Some protocols embrace this situation rather than trying to avoid it. Instead of discarding orphaned blocks, they integrate them into the blockchain structure. In practice, this means that a

block may reference multiple parents—the blockchain becomes a Directed Acyclic Graph (DAG).

There are multiple levels of sophistication in using the DAG structure productively. Simpler approaches use the formerly orphaned blocks to make the mining lottery more fair, while still discarding the transactions they contain [53, 58]. More advanced designs integrate all blocks into the total order, making the transactions they carry available to the application layer [1, 57].

3 My Research

3.1 Motivation and Scope

Bitcoin has been thoroughly analyzed, both in terms of security and incentive structure. Many alternative protocols promise improvements in some aspects, but it remains unclear whether they do so without sacrificing other desirable properties. Most protocol proposals come with only partial analysis, targeting either security or incentives, but rarely both. Compounding the issue, assumptions and security definitions vary across papers, making it difficult to conduct fair comparisons among different proposals [63].

For example, for most of the alternative protocols discussed above—DAG-structured chains [1, 57, 58], parallel chains [9, 62], and multiple puzzles per block [12, 53]—there are still no concrete failure bounds. As a result, users lack reliable guidance on how long they should wait before accepting a transaction as final. Meanwhile, the protocols that aim to provide finality by layering BFT-style consensus on top of proof-of-work [3, 19, 46, 54], as well as some DAG-based protocols [57, 58], do not address the incentive layer, ignoring issues such as fairness and selfish mining.

With this dissertation, I aim to push the boundaries of proof-of-work consensus protocols. I want to improve consensus across all relevant dimensions: consistency, fairness, and incentive compatibility. To this end, I contribute both new protocol designs and methods for their analysis. At the same time, I seek to establish comparability across protocols by developing shared assumptions, unified models, and consistent evaluation criteria.

My focus lies on the implementation and analysis of blockchain consensus (BC) and its incentives, deliberately abstracting the details of the network and application layers. From the network layer, I assume that blocks are disseminated using a reliable broadcast (RB) primitive. The application layer implements a cryptocurrency to support the rewards on the incentive layer, following the CCB-on-BC constructions outlined above.

Within the space of alternative designs, I restrict my attention to protocols that employ multiple puzzles per block or use DAG-structured chains. I do not consider protocols that layer an additional consensus mechanism on top of proof-of-work, nor do I pursue architectures that coordinate multiple parallel blockchains.

In my analysis, I adopt several simplifying assumptions to keep the models tractable. I treat difficulty adjustment only in general terms: block production adapts to the available hash rate and the number of orphaned blocks, but I do not model the specific adjustment mechanism or its internal dynamics. For incentive analysis, I measure utility purely in terms of protocol-defined rewards, ignoring external economic factors such as exchange rates, electricity prices, or hardware costs, as well as the possibility of mining in other systems, or engaging in bribery, on-chain (e.g., through transaction fees) and off-chain (side payments). These simplifications allow me to isolate the effects

of consensus and reward mechanisms without the confounding influence of broader market dynamics.

I restrict my attention to protocols that use proof-of-work to authorize participation, because the stochastic properties of the puzzle impose unique requirements on the analysis. The restricted scope allows me to develop a unified methodological framework across all contributions, enabling direct and fair comparisons between protocols that vary only in structure (e.g., multiple puzzles per block or DAGs) rather than in their fundamental assumptions.

3.2 Methodology

My three papers rely on a recurring methodological foundation: the stochastic modeling of proof-of-work. The individual aspects of this modeling are carefully selected from various sources, but their combination into a coherent framework is a novel aspect of my work. In this section, I provide a brief introduction to this approach—deeper discussions of the individual topics can be found in my three papers.

Proof-of-Work on a Single Node Proof-of-work uses a hash-based puzzle to set up a competition among the miners. The system fixes parts of the input to a cryptographic hash function and the miners' task is to modify the remaining input such that the resulting hash value meets a certain threshold. Relying on the cryptographic properties of the hash function—specifically, the *random oracle model* [11, 26, 27]—repeated guessing is the only viable solving strategy: change the nonce, evaluate the hash function, check whether the output meets the difficulty threshold, and repeat.

From the stochastic modeling perspective, this constitutes repeated Bernoulli trials. Most iterations of the repeated guessing algorithm are failures: the hash does not meet the threshold. Successes represent those iterations where the miner actually solves the puzzle. In this terminology, the puzzle difficulty sets the success probability of each trial. In blockchain systems, this success probability is extremely small. E.g., the Bitcoin network currently evaluates about 10^{21} hashes per second [13], while the expected time between successes is about 10 minutes.

Continuous Time Approximation The puzzle solving time is a well-known stochastic stopping time: the number of failed trials before the first trial is successful. This stopping time follows the geometric distribution. In our case, where the success probability is very small, the discrete geometric distribution can be approximated by its continuous counterpart, the exponential distribution, with negligible loss of accuracy.

I use the continuous-time domain in all three papers. I do not consider the success probability of individual trials, nor the exact hash rate of miners. Instead, I focus on the exponentially distributed time between puzzle solutions, measured in conventional units of time—seconds, minutes, or hours.

Proof-of-Work in the Network The continuous-time abstraction allows me to compare and aggregate the puzzle-solving processes of all participants on a homogeneous time scale. While technically each miner operates with their own hash rate and engages in an independent puzzle-solving process, my analysis typically focuses on a global interpretation: the system produces puzzle solutions with exponentially distributed inter-arrival times. The global solving rate is assumed to be constant—or more precisely, controlled. For example, in Bitcoin, the difficulty is regularly adjusted to maintain an average solving rate of six puzzles per hour.

Individual hash rates still matter in the global mining process, but only in a relative sense: given that a puzzle solution occurs, the random variable indicating which miner solved this puzzle is distributed proportionally to the miners' relative hash rates.

Communication Delays In addition to puzzle solving, another source of entropy during protocol execution arises from communication latency. In my stochastic modeling, I treat individual network delays as unknown and put random variables as placeholders. Later, during analysis, these random variables are either instantiated with a specific probability distribution or worst-case realizations.

To simplify the modeling, I assume that all delays are either attributable to the puzzle solving itself, to the miners' deterministic actions, or to network delays. Network delays capture the time between a message being sent by one node and being delivered to another. This includes, for example, the time between receiving a message and its delivery to the protocol logic—such as the time spent validating the message. Another example is the time it takes to load a new puzzle onto the mining hardware. In distributed mining setups—such as public mining pools—this kind of delay is certainly non-negligible. I treat this as a propagation delay as well; it does not interrupt or pause the mining process itself.

This abstraction helps to keep the modeling of proof-of-work clean and focused: puzzle solutions appear with exponentially distributed inter-arrival times, and the puzzle-solving rate is known. All other timing variability is accounted for as communication delay.

Discrete Event Simulation These simplifications allow me to adopt a discrete view of system execution. Rather than observing the entire continuous time domain, I focus on two types of events: successful mining and message delivery.

I think of distributed system execution as a discrete-event simulation. This simulation maintains a queue of future events, indexed by time. The main loop repeatedly removes the earliest event from the queue and executes the associated event handler. The event handler captures the deterministic parts of protocol execution and may also schedule additional future events, each separated by a random, non-negative delay.

For example, the event handler for mining success would select the winning miner—sampled from all miners, weighted by hash rate—and let that miner process the new block according to the protocol rules. In Bitcoin, this would involve updating the longest chain and broadcasting the block. The event handler would then schedule the delivery of the block to the other miners after random delays.

This discrete-event simulation framework is also well-suited for modeling adversarial behavior. Arbitrary protocol deviations—such as block withholding or private chain mining—can be captured by modifying the event handlers, but only for the dishonest miners.

This way of thinking also extends to attack search and security analysis. In attack search, adversarial nodes are given some freedom in choosing their actions as part of an optimization process. In security analysis, the event loop structure helps to model worst-case scenarios—without losing detail about the stochastic nature of proof-of-work itself.

Security Analysis With Markov Chains Security research aims to understand how attackers can disrupt system execution or manipulate it to serve their own interests. The first step in any analysis is to set its scope—typically in the form of assumptions about what the attacker can and

cannot do. The next step is to determine what remains possible within these assumptions, ideally through a formal proof.

In the case of proof-of-work blockchain protocols, the assumptions usually include the following [2, 10, 26, 27]: the proof-of-work mechanism functions as intended; a given fraction of the total hash rate is controlled by honest miners; honest miners follow the protocol; and communication delays are bounded by some maximum delay Δ.

From these assumptions, one can apply logical reasoning to derive useful system properties that hold independently of the attacker's behavior. To sketch an example from Paper A: In Bitcoin, if an honest node mines a block of height n at time t, then all honest nodes will be aware of a chain of at least height n by time $t + \Delta$. Moreover, at that same time $t + \Delta$, for an attacker to succeed in a consistency attack, they must present a private chain of height $n + 1$ or greater. If no such chain exists, the attack cannot succeed at that time.

The final step is to evaluate, quantitatively, how likely the system is to remain in the set of states where the relevant security properties hold, given the stochastic behavior of proof-of-work. A useful tool here is the *Markov chain*, a mathematical model in which the system transitions between a set of states according to probabilistic rules [45]. The key property of a Markov chain is that the probability distribution over next state depends only on the current state, and not on its history. This allows discrete-state Markov chains to be represented as two-dimensional matrices and, crucially, enables the evaluation of all possible system evolutions in parallel via matrix multiplication. The probability of being in any given state after n transitions can be computed by multiplying the state transition matrix n times with the current state vector. For large n, most of these multiplications can be avoided using exponentiation by squaring. This is useful, for example, in the analysis of eventual consistency in Bitcoin, where one can observe that the probability of reaching an inconsistent state (e.g., due to a fork) decays exponentially as the observation window increases.

Attack Search with Markov Decision Processes[3] Finding attacks against blockchain consensus protocols involves sequential decision-making, where an attacker must make a series of choices over time. The outcomes of these decisions are non-deterministic due to the inherently stochastic nature of mining. The right mathematical tool for describing such problems is the Markov Decision Process (MDP) [56].

An MDP is a probabilistic state machine. The attacker, in this context also called the *agent*, observes the current state of the system and chooses an *action*. The MDP defines probabilistic state transitions, specifying the probability distribution for the next state based on the current state and the chosen action.

A sequence of state transitions, starting from the (probabilistic) initial state until the process ends, is called an *episode*. Each state transition may have an effect, such as assigning a reward. The goal of the agent is to maximize an *objective function*, such as the total reward accumulated during an episode. Non-terminating models are also feasible, in which case the objective is to maximize the discounted future rewards.

The strategy an agent uses to decide which action to take in each state is called a *policy*. Solving an MDP is about finding a policy that maximizes the objective function. Applying a fixed policy to a MDP results in a Markov chain.

[3] A version of this paragraph was published before, as Section 2 of my third publication [37, Paper C].

A good, textbook-sized introduction to the topic is provided by Sutton and Barto [59]. The book offers a formal mathematical description of MDPs and covers solving algorithms from first principles. These methods can be broadly categorized as follows:

Linear programming: The Bellman equations yield an optimality condition for the policy, which can be reformulated as a linear programming problem. Algorithms like Simplex can solve this exactly, ensuring an optimal policy is found.

Dynamic programming: Algorithms such as value iteration and policy iteration use the Bellman equations to define an iterative optimization step. Repeatedly applying this step converges on the optimal policy. When problem sizes become too large for linear programming, these methods offer practical solutions without losing much accuracy.

Approximate methods: Modern reinforcement learning (RL) techniques, such as Q-learning or policy-gradient methods, train a simplified surrogate model by observing the probabilistic state machine. The resulting policies may not be optimal in the original MDP, but these methods are highly versatile: they are applicable when exact solutions are infeasible and effective even when the underlying MDP is unknown or changes over time.

3.3 List of Publications

My dissertation consists of three published research articles. In the following I list these papers with title, authors, and a short summary. For the first two papers, those that are co-authored, I append a contribution statement to highlight my own contributions.

Paper A: Parallel Proof-of-Work With Concrete Bounds

First author; co-authored with Rainer Böhme. Presented at the *4th ACM Conference on Advances in Financial Technologies, AFT 2022, in Cambridge, MA*. Published by *ACM* in the proceedings of said conference: `https://doi.org/10.1145/3558535.3559773`



This publication's cite key is [41]. Earlier versions of this work are available as preprints [40, 42]. The paper is reproduced in full on Pages 32–62 of this dissertation.

Research Question

> How does the security of blockchain consensus protocols improve when mandating multiple proofs-of-work per block, which can be mined in parallel without coordination?

Summary Recall from Section 2 that in proof-of-work blockchain consensus protocols, the security properties arise only after solving or observing multiple independent puzzles, and that Bitcoin is inherently sequential: each puzzle depends on the solution of the previous one. Network asynchrony delays the propagation of solutions and threatens the protocol's security.

To address this, we propose a new family of blockchain consensus protocols, called $\mathcal{B}_k$, where each block requires k independent proofs-of-work. This design captures the security benefits of multiple puzzles within a single block while mitigating the impact of asynchrony: the k puzzles per block can be mined in parallel.

To analyze the security of $\mathcal{B}_k$, we define a related family of auxiliary protocols, $\mathcal{A}_k$, each representing a single interval between two consecutive blocks. Using Markov chain analysis, we derive explicit upper bounds on the probability of consistency failures in $\mathcal{A}_k$, expressed as a function of the attacker compute power α and the network synchrony parameter Δ. We then optimize the protocol parameter k to minimize failure probabilities across various network scenarios (combinations of α and Δ), benchmarking our results against the then-recent concrete bounds established for Bitcoin [50]. For example, $\mathcal{A}_k$ configured for $k = 51$ puzzles and 10 minutes expected runtime has a consistency failure probability of at most $2.2 \cdot 10^{-4}$ when facing an attacker who controls 25 % of the mining power in a network where messages propagate within $\Delta = 2$ seconds. The best configuration of Bitcoin faces a failure probability of 9 % in the same conditions.

Accompanying the theoretical analysis of $\mathcal{A}_k$, we evaluate the full blockchain protocol $\mathcal{B}_k$ using discrete event network simulation. The results demonstrate resilience under severe network conditions including excessive network delays, churn, benign failures, and adversarial behavior.

Contribution Statement I contributed the design of the blockchain protocol $\mathcal{B}_k$, the auxiliary protocol $\mathcal{A}_k$, and the security analysis of $\mathcal{A}_k$. Rainer conceptualized the experimental evaluation, including the selection of parameters to optimize and the design of network scenarios for the discrete event simulations, as well as the overall structure of the paper. I implemented and executed the experiments, including both parameter optimization and the simulation framework. We wrote the paper in equal parts: I formulated internal drafts which we then iteratively refined in collaborative, "pair-programming"-style writing sessions.

Paper B: Tailstorm: A Secure and Fair Blockchain Protocol

First author; co-authored with Ben Glickenhaus, George Bissias, and Gregory Griffith. Presented at the *5th Conference on Advances in Financial Technologies, AFT 2023, in Princeton, NJ.* Published by *Schloss Dagstuhl – Leibniz-Zentrum für Informatik* in the proceedings of said conference: `https://doi.org/10.4230/LIPIcs.AFT.2023.6`

This publication's cite key is [44]. An earlier version of this work is available as preprint [43]. The paper is reproduced in full on Pages 63–89 of this dissertation.

Research Question

> How can the security guarantees of Parallel Proof-of-Work at the consensus layer (Paper A) be balanced with the performance, fairness, and incentive compatibility requirements of the application and incentive layers?

Summary[4] Recall from Section 2 that proof-of-work cryptocurrencies rely on a balance of security and fairness in order to maintain a sustainable ecosystem of miners and users. Users demand fast and consistent transaction confirmation, and in exchange drive the adoption and valuation of the cryptocurrency. Miners provide the confirmations, however, they primarily seek rewards. In unfair systems, miners can amplify their rewards by consolidating mining power. Centralization, however, undermines the security guarantees of the system and might discourage users.

[4] This summary is based on the abstract of the publication itself.

In this paper we present a blockchain consensus protocol, Tailstorm, that strikes this balance. Tailstorm merges multiple recent protocol improvements addressing security, confirmation latency, and throughput with a novel incentive mechanism improving fairness. Tailstorm implements parallel proof-of-work, as analyzed in Paper A, with k puzzles per block to obtain state-of-the-art consistency guarantees. Inspired by Bobtail [12] and Storm [6], we structure the individual puzzles in a tree which, by including a list of transactions with each proof-of-work, reduces confirmation latency and improves throughput. The proposed incentive mechanism discounts rewards based on the depth of this tree. Thereby, it effectively punishes information withholding, the core attack strategy used to reap an unfair share of rewards in other protocols.

We back these claims with a comprehensive analysis. We present a generic system model which allows us to specify Bitcoin, $\mathcal{B}_k$ from Paper A, and Tailstorm from a joint set of assumptions. We provide an analytical bound for the fairness of Tailstorm and Bitcoin in honest networks and we confirm the results through simulation. We evaluate the effectiveness of dishonest behavior through reinforcement learning. Our attack search reproduces known optimal strategies against Bitcoin, uncovers new ones against $\mathcal{B}_k$, and confirms that Tailstorm's reward discounting makes it more resilient to incentive layer attacks.

Lastly, we have implemented a prototype of the Tailstorm cryptocurrency as a hard fork of Bitcoin Cash. The client software is ready for testnet deployment and we also publish its source online [31].

Contribution Statement[5] George originally developed the idea of reward discounting as a way to combat withholding attacks in Bobtail. George and Gregory refined that idea into an early version of Tailstorm suitable for a hard fork of Bitcoin Cash. They also contributed the prototype implementation. Patrik contributed the protocol specification, simulation-based analyses, and reinforcement learning interface. George contributed the analytical orphan rate analysis. George and Patrik contributed the hard-coded reference policies. Ben and Patrik conducted the attack search with reinforcement learning. Guided by earlier stages of the analyses, Patrik provided improvements to Tailstorm consensus and its transaction handling. Patrik and George wrote this paper.

Paper C: Automated Selfish Mining Analysis for DAG-Based PoW Consensus Protocols

Single author. Presented at the *23rd International Conference on Applied Cryptography and Network Security, ACNS 2025, in Munich, Germany.* Published by *Springer* in the proceedings of said conference: `https://doi.org/10.1007/978-3-031-95761-1_8`



This publication's cite key is [37]. Earlier and updated versions of this work are available as preprints [36, 38]. The paper is reproduced in full on Pages 90–116 of this dissertation.

Research Question

> How can the attack search methodology applied to Tailstorm in Paper B be generalized to other proof-of-work consensus protocols? Can this be used to assist protocol design?

[5] Copied from the publication [44, Pages 21–22]; corrected to clarify my role in the prototype implementation (none).

Summary[6] Recall that selfish mining is a form of strategic rule-breaking aimed at maximizing rewards in proof-of-work blockchain protocols, and that Markov Decision Processes (MDPs) are the preferred tool for identifying optimal strategies in Bitcoin and similar linear chain protocols. Protocols increasingly adopt DAG-based chain structures, for which MDP analysis is more involved. To date, researchers have tailored specific MDPs for each protocol. As a result, protocol design suffers from long feedback loops: every protocol change implies manual work on the MDP.

To overcome this, I propose a generic attack model that covers a wide range of protocols, including Ethereum Proof-of-Work, GhostDAG [57], and Parallel Proof-of-Work [37, Paper A] and implement tools to apply the incentive analysis automatically. My approach is modular and mechanical: I specify each protocol as a concise program; my tooling then derives the corresponding selfish mining MDP. Solving the resulting MDPs with established methods [64], I obtain optimal attack strategies against the specified protocols.

Notably, this is the first selfish mining analysis of GhostDAG, a protocol that has escaped MDP-based treatment before. But not only that: the ability to compare protocols under a unified framework and iterate rapidly during protocol design is a significant advance in itself.

3.4 Synthesis

My dissertation aims to advance the design and analysis of proof-of-work consensus protocols across three dimensions: consistency, fairness, and incentive compatibility. In this section, I synthesize how my papers as a whole contribute to this goal: Paper A provides a foundational consistency analysis, Paper B addresses practical questions regarding protocol design and incentives, Paper C generalizes my approach to a broader range of protocols.

In Paper A, I introduce Parallel Proof-of-Work and lay the theoretical foundation for my research. My Markov chain analysis of the auxiliary protocol $\mathcal{A}_k$ provides concrete consistency failure bounds for a single round of voting. The full protocol $\mathcal{B}_k$ extends this to the blockchain setting by iteratively invoking $\mathcal{A}_k$. To further validate the design of $\mathcal{B}_k$, I implement a discrete event simulation framework which allows me to evaluate the performance and robustness of the protocol under various network conditions, including excessive propagation delays and malicious behavior. This work demonstrates that concrete security bounds are feasible for non-linear protocols, and thereby sets a new standard that has previously been only available for the sequential proof-of-work blockchain protocol used in Bitcoin. However, Paper A primarily focuses on consistency, leaving fairness and incentive compatibility unaddressed. It also leaves open how the application layer, for example a cryptocurrency, can leverage the individual proof-of-work votes.

Paper B proposes to structure the individual proof-of-work votes of each block as a tree—sequentially when possible and parallel when needed. The design and analysis of the proposed protocol, Tailstorm, carefully leverage the results of Paper A while also addressing its shortcomings: First, Tailstorm stores cryptocurrency transactions in the individual votes—now called subblocks—ensuring that each proof-of-work contributes to the progress of the application layer. Second, Tailstorm subblocks referencing the same parent block—now called summary—are always eligible for the next summary, even if they contain conflicting transactions. This ensures that the parallel voting semantics of $\mathcal{B}_k$ are preserved and that the security guarantees of $\mathcal{A}_k$ translate to the new protocol. Third, Paper B proposes a reward mechanism for Parallel Proof-of-Work that substantially

[6] This summary is based on the abstract of the publication itself.

improves the fairness and the incentive compatibility of the protocol. Methodologically, Paper B reuses the discrete event simulation framework from Paper A, to both analyze fairness of Tailstorm in benign networks and to evaluate concrete attack strategies. The RL-based attack search builds on the discrete event simulation as well: I instrument the malign node such that it interrupts the simulation at key decision points. Each time, the RL agents observes the system's state and controls the node's actions accordingly. This approach works well to uncover strategies we have not anticipated in the manually crafted attacks. But there are shortcomings as well: the instrumentation and observed features are hand-coded for each simulated protocol, which raises questions about whether the attack space covers all realistic attacks. Additionally, the RL-based search is not exhaustive, meaning some relevant attack strategies might have escaped discovery.

Paper C advances the methodology by introducing a generic framework for automated and exhaustive attack search against a wide range of DAG-based proof-of-work consensus protocols. The protocol designer specifies the honest behavior as a concise program. My tooling then automatically derives the corresponding MDP and computes an *optimal* attack strategy using dynamic programming. This eliminates the need for manual instrumentation and removes any doubt originating from the approximate nature of reinforcement learning.

In summary, the research progresses from theoretical foundations in Paper A, to practical innovations in Paper B, and finally to methodological automation in Paper C. Paper A establishes Parallel Proof-of-Work and extends concrete bounds to non-linear protocols. Paper B implements Parallel Proof-of-Work in the Tailstorm cryptocurrency, addressing fairness and incentive compatibility while reusing the simulation framework of the first paper. Based on the insights of Papers A and B, Paper C introduces a generic MDP framework, enabling rigorous and automated analysis of selfish mining strategies across a wide range of protocols. This progression highlights the interplay between protocol evolution and methodological advancements, culminating in a framework that facilitates the rapid design and evaluation of proof-of-work consensus protocols in the future.

3.5 Future Work

Building on the synthesis, this section identifies several opportunities for future work to address open questions in my research. These tasks can be pursued without altering my assumptions and methodology, and offer direct extensions to further advance the design and analysis of proof-of-work consensus protocols.

Merging the benefits of MDP-based analysis and RL A key limitation of the MDP-based approach in Paper C is the need to restrict the state space to ensure tractability. However, this restriction has significant advantages: it enables the derivation of optimal policies through dynamic programming and provides clear, interpretable results. In particular, the monotonicity of the restricted state space—where any behavior feasible in a smaller state space remains feasible in a larger one—yields a reliable lower bound on attack performance. This stands in contrast to RL-based approaches, which support infinite state spaces but suffer from two sources of error: insufficient state exploration and imprecise policy estimation.

I believe the best results can be achieved by combining both approaches. A logical next step would be to extend the generic MDP framework from Paper C with an RL-based attack search, as demonstrated in Paper B. This would allow to validate RL's effectiveness on problem sizes where the

optimal policy is already known. The insights gained could then guide the tuning of RL algorithms for larger problem instances that surpass the capabilities of dynamic programming.

Finding the best reward scheme A missed opportunity in Paper C for establishing coherence with Paper B is the lack of analysis of Tailstorm's tree-style voting and reward discounting within the MDP-based framework. This would have been straightforward: specify the protocol, re-apply the mechanical analysis pipeline, and report the results.

This also highlights a limitation of Paper B: the proposed tree-based reward discounting scheme is primarily guided by intuition—specifically, my co-author George's intuition. To others, the design might come across somewhat arbitrary. Could there be even more effective ways to allocate and discount rewards in Parallel Proof-of-Work?

I have begun exploring an alternative approach: DAG-based reward discounting [39], leveraging the RL-based analysis framework from Paper B. However, while the results are promising, the design still feels arbitrary. Ultimately, the reward scheme itself should be optimized. For example, an evolutionary algorithm could be applied to iteratively refine the protocol's reward scheme. The generic and modular framework of Paper C can support this process.

Closing a gap in the consistency analysis In Paper A, I argue for the consistency of Parallel Proof-of-Work, by providing mathematical evidence for the auxiliary protocol $\mathcal{A}_k$. However, this auxiliary protocol only covers a single-round of voting. As a result, my argument leaves a clear gap regarding the synchronization across episodes.

A comprehensive analysis of the entire blockchain protocol would be desirable, akin to the approaches in [29, 32, 50], which derive concrete consistency failure bounds for Bitcoin. However, I cannot provide actionable advice for translating one of these analyses to Parallel Proof-of-Work.

A quick win, and a missed opportunity in Paper A, would be to compare the single-round $\mathcal{A}_k$ to a linear version where puzzles are interdependent like in Bitcoin. This would involve adjusting the Markov chain analysis of $\mathcal{A}_k$ such that concurrently mined puzzles are discarded. The approach would allow us to isolate the effects of parallel voting without analysing the full blockchain protocol $\mathcal{B}_k$.

Applying attack search to non-economic problems Another, more rigorous approach to improving the security analysis of Parallel Proof-of-Work could be to extend the MDP-based attack search framework of Paper C—currently focused on selfish mining—to include non-economic optimization goals, such as violating consistency or slowing progress. Such an adjustment would primarily involve modifying the objective function of the MDP, while most of the existing methodology and implementation could be reused.[7]

For example, to identify optimal strategies for violating consistency, one could define an objective function that scales rewards based on the number of blocks rewritten in the defender's view of the chain. Evaluating the optimal strategy would then provide concrete failure bounds for consistency.

A notable side effect of this approach is that it is not limited to Parallel Proof-of-Work. It generalizes to the entire class of DAG-based blockchain protocols covered in Paper C.

Connect to model checking During the end phase of writing this dissertation, I realized that the automated attack search methodology of Paper C has some similarities to model checking: after

[7] I considered this before implementing the MDP-based framework—the architecture accommodates such changes.

specifying the program, the analysis can be applied automatically. It is probably worth exploring whether the formal verification literature provides tools that can be applied to my kind of problem.

A superficial search reveals a very promising candidate: the PRISM model checker [47]. Apparently, the tool's feature set fully covers my methodology (Section 3.2): continuous and discrete time stochastic processes, bounded communication delays, discrete event simulation, Markov chains, Markov decision processes, costs and rewards, and generation of optimal strategies. This begs the question, is it feasible to compile the consensus protocol specifications of Paper C into PRISM models instead of MDPs and do the remaining parts of the analysis pipeline follow?

3.6 Limitations

This section outlines the key limitations of my research, focusing on challenges that go beyond the scope of my current methodology and assumptions. These limitations represent fundamental obstacles that cannot be resolved through straightforward extensions or refinements of my work. Addressing them will likely require new techniques and insights from other fields of research—if they can be solved at all.

Economic security In this work, I use incentive compatibility [16, 33] as a way to extend the classic security notions with respect to economic considerations: While my security arguments rely on the assumption that a certain fraction of the network behaves honestly independent of economic arguments, incentive compatibility helps *reinforce* this security by ensuring that rational miners behave honestly as well.

A significant simplification I make, is that rewards arise solely from the structure of the blockchain. In real-world cryptocurrency systems, block space is limited, and users bid for inclusion of their transaction by attaching a fee. In my dissertation, I ignore this dynamic fee market entirely.

A known problem with transaction fees is that they enable a form of bribing: For example, a selfish miner can incentivize other rational miners to resolve block races in their favor: When the attacker actively excludes high-fee transactions from their private blocks, other rational miners are incentivized to adopt the private chain, to then append the high-fee transactions themselves [14]. For Bitcoin, the implications are severe. As already pointed out in the original work on selfish mining [24], if all block races are resolved in the attacker's favor, selfish mining becomes profitable for all miners—even the very weak ones—and incentive compatibility is lost.

More generally speaking, my modeling builds on a very narrow definition of rationality: I assume a flat utility function—accumulated block reward per time spent mining—which effectively precludes shorting the cryptocurrency, engaging in side payments, and other forms of external incentives. In reality, sophisticated financial instruments and bribery mechanisms exist, and they can certainly motivate rational miners to deviate from the protocol. If all miners act rationally, however, even the classic notion of security is at risk [25].

Permissionless systems All protocols discussed in my work are designed for the permissionless model, where anyone can join and leave the system without asking for confirmation. This lack of central identity has significant implications for fairness: A resourceful participant can pretend to be many weaker participants, making it impossible to enforce corrective or redistributive measures. At best, future rewards and control are proportional to the resources the participants already have.

In proof-of-work systems, this proportionality is eroded by the economies of scale of mining. Larger miners typically enjoy lower operational costs, benefiting from volume discounts on electricity or the ability to design and deploy custom mining hardware [60]. As a result, mining power naturally concentrates in the hands of a few large players [5].

Speaking of electricity, it is important to acknowledge that proof-of-work systems, by design, consume significant amounts of energy to rate-limit participation. While I do not view this energy use as inherently problematic, the environmental costs are often externalized to future generations, leading to intergenerational inequality.

These problems—economies of scale and excessive energy use—are less pronounced in proof-of-stake systems. In such systems, rewards and control are typically scaled in proportion to the participant's holdings of the system's cryptocurrency itself. This aligns closely with the proportionality that a permissionless system can ideally achieve, but it also raises questions about whether the system is still fully permissionless. After all, new participants must first obtain cryptocurrency from existing participants before they can start to create their own.

On a more general note, permissionless systems are often motivated by a desire to escape central control. This vision can be admirable in certain contexts, particularly when states betray their populations and authorities become oppressive. In functioning societies, however, the state is typically seen as a representation of the collective will. Its role is to protect individual freedoms while ensuring that the influence of the wealthy and powerful does not overshadow the rights of others. Through redistributive measures, it seeks to address inequalities and promote fairness, even if imperfectly. Permissionless systems alone cannot achieve these outcomes, because resourceful participants can pretend not to be.

3.7 Conclusion

I set out to enhance proof-of-work consensus mechanisms across three key dimensions: security, fairness, and incentive compatibility. In my three papers, I have contributed new protocol designs that achieve these goals, as well as new methodology that supports the evaluation of future designs.

My first paper establishes the theoretical foundation of Parallel Proof-of-Work, demonstrating improved security over sequential protocols like Bitcoin. The second paper extends this protocol with a novel incentive scheme, the discounting of rewards, to discourage selfish mining. The last paper generalizes my analytical framework, enabling the automated evaluation of other proof-of-work consensus protocols in the future.

References

[1] Ittai Abraham, Danny Dolev, Ittay Eyal, and Joseph Y. Halpern. "Colordag: An Incentive-Compatible Blockchain". In: *37th International Symposium on Distributed Computing, DISC 2023, October 10-12, 2023, L'Aquila, Italy*. Ed. by Rotem Oshman. Vol. 281. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 1:1–1:22. https://doi.org/10.4230/LIPIcs.DISC.2023.1.

[2] Ittai Abraham and Dahlia Malkhi. "The Blockchain Consensus Layer and BFT". in: *Bull. EATCS* 123 (2017). http://eatcs.org/beatcs/index.php/beatcs/article/view/506.

[3] Ittai Abraham, Dahlia Malkhi, Kartik Nayak, Ling Ren, and Alexander Spiegelman. "Solida: A Blockchain Protocol Based on Reconfigurable Byzantine Consensus". In: *21st International Conference on Principles of Distributed Systems, OPODIS 2017, Lisbon, Portugal, December 18-20, 2017*. Ed. by

James Aspnes, Alysson Bessani, Pascal Felber, and João Leitão. Vol. 95. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017, 25:1–25:19. https://doi.org/10.4230/LIPIcs.OPODIS.2017.25.

[4] Mohamed Alzayat, Johnnatan Messias, Balakrishnan Chandrasekaran, Krishna P. Gummadi, and Patrick Loiseau. *Modeling Coordinated vs. P2P Mining: An Analysis of Inefficiency and Inequality in Proof-of-Work Blockchains*. Preprint. 2021. https://arxiv.org/abs/2106.02970.

[5] Nick Arnosti and S. Matthew Weinberg. "Bitcoin: A Natural Oligopoly". In: *Manag. Sci.* 68.7 (2022), pp. 4755–4771. https://doi.org/10.1287/mnsc.2021.4095.

[6] awemany. *Storm: Using Weak Proof-of-Work for instant confirmations on Bitcoin Cash*. 2019. https://github.com/awemany/storm-sim/blob/master/whitepaper/storm-wp-2019-08-30.pdf.

[7] Adam Back. *Hashcash - A Denial of Service Counter-Measure*. 2002. http://www.hashcash.org/papers/hashcash.pdf.

[8] Adam Back. *Hashcash Spam Prevention & Firewalls – Cypherpunks Mailing List Archive*. 1997. https://mailing-list-archive.cryptoanarchy.wiki/archive/1997/12/1e5b8113649b27a084cbd156bbbcb9bc930be21ede63d9cff4bc41a1dabf1cd3/.

[9] Vivek Kumar Bagaria, Sreeram Kannan, David Tse, Giulia Fanti, and Pramod Viswanath. "Prism: Deconstructing the Blockchain to Approach Physical Limits". In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*. Ed. by Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz. ACM, 2019, pp. 585–602. https://doi.org/10.1145/3319535.3363213.

[10] Shehar Bano, Alberto Sonnino, Mustafa Al-Bassam, Sarah Azouvi, Patrick McCorry, Sarah Meiklejohn, and George Danezis. "SoK: Consensus in the Age of Blockchains". In: *Proceedings of the 1st ACM Conference on Advances in Financial Technologies, AFT 2019, Zurich, Switzerland, October 21-23, 2019*. ACM, 2019, pp. 183–198. https://doi.org/10.1145/3318041.3355458.

[11] Mihir Bellare and Phillip Rogaway. "Random Oracles are Practical: A Paradigm for Designing Efficient Protocols". In: *CCS '93, Proceedings of the 1st ACM Conference on Computer and Communications Security, Fairfax, Virginia, USA, November 3-5, 1993*. Ed. by Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby. ACM, 1993, pp. 62–73. https://doi.org/10.1145/168588.168596.

[12] George Bissias and Brian Neil Levine. "Bobtail: Improved Blockchain Security with Low-Variance Mining". In: *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society, 2020. https://www.ndss-symposium.org/ndss-paper/bobtail-improved-blockchain-security-with-low-variance-mining/.

[13] *Blockchain.com: Total Hash Rate in the Bitcoin Network*. Accessed 13th Aug 2025. https://www.blockchain.com/explorer/charts/hash-rate.

[14] Miles Carlsten, Harry A. Kalodner, S. Matthew Weinberg, and Arvind Narayanan. "On the Instability of Bitcoin Without the Block Reward". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*. Ed. by Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi. ACM, 2016, pp. 154–167. https://doi.org/10.1145/2976749.2978408.

[15] Miguel Castro and Barbara Liskov. "Practical Byzantine Fault Tolerance". In: *Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI), New Orleans, Louisiana, USA, February 22-25, 1999*. Ed. by Margo I. Seltzer and Paul J. Leach. USENIX Association, 1999, pp. 173–186. https://doi.org/10.5555/296806.296824.

[16] Jonathan Chiu and Thorsten Koeppl. "Incentive Compatibility on the Blockchain". In: *Social Design: Essays in Memory of Leonid Hurwicz*. Ed. by Walter Trockel. Springer, 2019, pp. 323–335. https://doi.org/10.1007/978-3-319-93809-7_20.

[17] Wei Dai. *PipeNet 1.1 and B-Money – Cypherpunks Mailing List Archive*. 1998. https://mailing-list-archive.cryptoanarchy.wiki/archive/1998/11/0e3ccb59b898d6b3dc41250477bb70f593e29f5519439e1ad189bd7fa9234d45/.

[18] Wei Dai and Adam Back. *Wei Dei's "B-Money" Protocol – Cypherpunks Mailing List Archive*. 1998. https://mailing-list-archive.cryptoanarchy.wiki/archive/1998/12/56d10cf5f88e5d6d68797c6a3b94a25e355e561a2b8636b6b1f53bd577802fb4/.

[19] Christian Decker, Jochen Seidel, and Roger Wattenhofer. "Bitcoin meets strong consistency". In: *Proceedings of the 17th International Conference on Distributed Computing and Networking, Singapore, January 4-7, 2016*. ACM, 2016, 13:1–13:10. https://doi.org/10.1145/2833312.2833321.

[20] Xavier Défago, André Schiper, and Péter Urbán. "Total order broadcast and multicast algorithms: Taxonomy and survey". In: *ACM Comput. Surv.* 36.4 (2004), pp. 372–421. https://doi.org/10.1145/1041680.1041682.

[21] Amir Dembo, Sreeram Kannan, Ertem Nusret Tas, David Tse, Pramod Viswanath, Xuechao Wang, and Ofer Zeitouni. "Everything is a Race and Nakamoto Always Wins". In: *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*. Ed. by Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna. ACM, 2020, pp. 859–878. https://doi.org/10.1145/3372297.3417290.

[22] Cynthia Dwork and Moni Naor. "Pricing via Processing or Combatting Junk Mail". In: *Advances in Cryptology - CRYPTO '92, 12th Annual International Cryptology Conference, Santa Barbara, California, USA, August 16-20, 1992, Proceedings*. Ed. by Ernest F. Brickell. Vol. 740. Lecture Notes in Computer Science. Springer, 1992, pp. 139–147. https://doi.org/10.1007/3-540-48071-4_10.

[23] Ittay Eyal, Adem Efe Gencer, Emin Gün Sirer, and Robbert van Renesse. "Bitcoin-NG: A Scalable Blockchain Protocol". In: *13th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2016, Santa Clara, CA, USA, March 16-18, 2016*. Ed. by Katerina J. Argyraki and Rebecca Isaacs. USENIX Association, 2016, pp. 45–59. https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eyal.

[24] Ittay Eyal and Emin Gün Sirer. "Majority Is Not Enough: Bitcoin Mining Is Vulnerable". In: *Financial Cryptography and Data Security - 18th International Conference, FC 2014, Christ Church, Barbados, March 3-7, 2014, Revised Selected Papers*. Ed. by Nicolas Christin and Reihaneh Safavi-Naini. Vol. 8437. Lecture Notes in Computer Science. Springer, 2014, pp. 436–454. https://doi.org/10.1007/978-3-662-45472-5_28.

[25] Bryan Ford and Rainer Böhme. *Rationality is Self-Defeating in Permissionless Systems*. Preprint. 2019. https://arxiv.org/abs/1910.08820.

[26] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. "The Bitcoin Backbone Protocol: Analysis and Applications". In: *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part II*. ed. by Elisabeth Oswald and Marc Fischlin. Vol. 9057. Lecture Notes in Computer Science. Springer, 2015, pp. 281–310. https://doi.org/10.1007/978-3-662-46803-6_10.

[27] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. "The Bitcoin Backbone Protocol: Analysis and Applications". In: *J. ACM* 71.4 (2024), 25:1–25:49. https://doi.org/10.1145/3653445.

[28] Peter Gazi, Aggelos Kiayias, and Alexander Russell. "Tight Consistency Bounds for Bitcoin". In: *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*. Ed. by Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna. ACM, 2020, pp. 819–838. https://doi.org/10.1145/3372297.3423365.

[29] Peter Gazi, Ling Ren, and Alexander Russell. "Practical Settlement Bounds for Longest-Chain Consensus". In: *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part I*. ed. by Helena Handschuh and Anna Lysyanskaya. Vol. 14081. Lecture Notes in Computer Science. Springer, 2023, pp. 107–138. https://doi.org/10.1007/978-3-031-38557-5_4.

[30] Arthur Gervais, Ghassan O. Karame, Karl Wüst, Vasileios Glykantzis, Hubert Ritzdorf, and Srdjan Capkun. "On the Security and Performance of Proof of Work Blockchains". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*. Ed. by Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi. ACM, 2016, pp. 3–16. https://doi.org/10.1145/2976749.2978341.

[31] Gregory Griffith and George Bissias. *Tailstorm Node Implementation*. 2023. https://gitlab.com/georgebissias/BCHUnlimited/-/tree/tailstorm_prototype.

[32] Dongning Guo and Ling Ren. "Bitcoin's Latency-Security Analysis Made Simple". In: *Proceedings of the 4th ACM Conference on Advances in Financial Technologies, AFT 2022, Cambridge, MA, USA, September 19-21, 2022*. Ed. by Maurice Herlihy and Neha Narula. ACM, 2022, pp. 244–253. https://doi.org/10.1145/3558535.3559791.

[33] Aljosha Judmayer, Nicholas Stifter, Alexei Zamyatin, Itay Tsabary, Ittay Eyal, Peter Gazi, Sarah Meiklejohn, and Edgar R. Weippl. "SoK: Algorithmic Incentive Manipulation Attacks on Permissionless PoW Cryptocurrencies". In: *Financial Cryptography and Data Security. FC 2021 International Workshops - CoDecFin, DeFi, VOTING, and WTSC, Virtual Event, March 5, 2021, Revised Selected Papers*. Ed. by Matthew Bernhard, Andrea Bracciali, Lewis Gudgeon, Thomas Haines, Ariah Klages-Mundt, Shin'ichiro Matsuo, Daniel Perez, Massimiliano Sala, and Sam Werner. Vol. 12676. Lecture Notes in Computer Science. Springer, 2021, pp. 507–532. https://doi.org/10.1007/978-3-662-63958-0_38.

[34] Ari Juels and John G. Brainard. "Client Puzzles: A Cryptographic Countermeasure Against Connection Depletion Attacks". In: *Proceedings of the Network and Distributed System Security Symposium, NDSS 1999, San Diego, California, USA*. The Internet Society, 1999. https://www.ndss-symposium.org/ndss1999/cryptographic-defense-against-connection-depletion-attacks/.

[35] Ghassan Karame, Elli Androulaki, and Srdjan Capkun. "Double-spending fast payments in bitcoin". In: *the ACM Conference on Computer and Communications Security, CCS'12, Raleigh, NC, USA, October 16-18, 2012*. Ed. by Ting Yu, George Danezis, and Virgil D. Gligor. ACM, 2012, pp. 906–917. https://doi.org/10.1145/2382196.2382292.

[36] Patrik Keller. *Automated Selfish Mining Analysis for DAG-Based PoW Consensus Protocols*. Preprint. 2025. https://arxiv.org/abs/2501.10888.

[37] Patrik Keller. "Automated Selfish Mining Analysis for DAG-Based PoW Consensus Protocols". In: *Applied Cryptography and Network Security - 23rd International Conference, ACNS 2025, Munich, Germany, June 23-26, 2025, Proceedings, Part I*. ed. by Marc Fischlin and Veelasha Moonsamy. Vol. 15825. Lecture Notes in Computer Science. Springer, 2025, pp. 215–243. https://doi.org/10.1007/978-3-031-95761-1_8.

[38] Patrik Keller. *Generic Selfish Mining MDP for DAG Protocols*. Preprint. 2023. https://arxiv.org/abs/2309.11924.

[39] Patrik Keller. *Parallel Proof-of-Work with DAG-Style Voting and Targeted Reward Discounting*. Preprint. 2023. https://arxiv.org/abs/2312.03111.

[40] Patrik Keller and Rainer Böhme. *HotPoW: Finality from Proof-of-Work Quorums*. Preprint. 2019. https://arxiv.org/abs/1907.13531.

[41] Patrik Keller and Rainer Böhme. "Parallel Proof-of-Work with Concrete Bounds". In: *Proceedings of the 4th ACM Conference on Advances in Financial Technologies, AFT 2022, Cambridge, MA, USA, September 19-21, 2022*. Ed. by Maurice Herlihy and Neha Narula. ACM, 2022, pp. 1–15. https://doi.org/10.1145/3558535.3559773.

[42] Patrik Keller and Rainer Böhme. *Parallel Proof-of-Work with Concrete Bounds*. Preprint. 2022. https://arxiv.org/abs/2204.00034.

[43] Patrik Keller, Ben Glickenhaus, George Bissias, and Gregory Griffith. *Tailstorm: A Secure and Fair Blockchain for Cash Transactions*. Preprint. 2023. https://arxiv.org/abs/2306.12206.

[44] Patrik Keller, Ben Glickenhaus, George Bissias, and Gregory Griffith. "Tailstorm: A Secure and Fair Blockchain for Cash Transactions". In: *5th Conference on Advances in Financial Technologies, AFT 2023, October 23-25, 2023, Princeton, NJ, USA*. ed. by Joseph Bonneau and S. Matthew Weinberg. Vol. 282. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 6:1–6:26. https://doi.org/10.4230/LIPIcs.AFT.2023.6.

[45] Lucianna Kiffer, Rajmohan Rajaraman, and Abhi Shelat. "A Better Method to Analyze Blockchain Consistency". In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*. Ed. by David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang. ACM, 2018, pp. 729–744. https://doi.org/10.1145/3243734.3243814.

[46] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Nicolas Gailly, Ismail Khoffi, Linus Gasser, and Bryan Ford. "Enhancing Bitcoin Security and Performance with Strong Consistency via Collective Signing". In: *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*. Ed. by Thorsten Holz and Stefan Savage. USENIX Association, 2016, pp. 279–296. https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/kogias.

[47] Marta Z. Kwiatkowska, Gethin Norman, and David Parker. "PRISM 4.0: Verification of Probabilistic Real-Time Systems". In: *Computer Aided Verification - 23rd International Conference, CAV 2011,*

Snowbird, UT, USA, July 14-20, 2011. Proceedings. Ed. by Ganesh Gopalakrishnan and Shaz Qadeer. Vol. 6806. Lecture Notes in Computer Science. Springer, 2011, pp. 585–591. https://doi.org/10.1007/978-3-642-22110-1_47.

[48] Leslie Lamport. "Time, Clocks, and the Ordering of Events in a Distributed System". In: *Commun. ACM* 21.7 (1978), pp. 558–565. https://doi.org/10.1145/359545.359563.

[49] Leslie Lamport, Robert E. Shostak, and Marshall C. Pease. "The Byzantine Generals Problem". In: *ACM Trans. Program. Lang. Syst.* 4.3 (1982), pp. 382–401. https://doi.org/10.1145/357172.357176.

[50] Jing Li, Dongning Guo, and Ling Ren. "Close latency-security trade-off for the Nakamoto consensus". In: *AFT '21: 3rd ACM Conference on Advances in Financial Technologies, Arlington, Virginia, USA, September 26 - 28, 2021*. Ed. by Foteini Baldimtsi and Tim Roughgarden. ACM, 2021, pp. 100–113. https://doi.org/10.1145/3479722.3480992.

[51] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. https://bitcoin.org/bitcoin.pdf.

[52] Rafael Pass, Lior Seeman, and Abhi Shelat. "Analysis of the Blockchain Protocol in Asynchronous Networks". In: *Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part II*. ed. by Jean-Sébastien Coron and Jesper Buus Nielsen. Vol. 10211. Lecture Notes in Computer Science. 2017, pp. 643–673. https://doi.org/10.1007/978-3-319-56614-6_22.

[53] Rafael Pass and Elaine Shi. "FruitChains: A Fair Blockchain". In: *Proceedings of the ACM Symposium on Principles of Distributed Computing, PODC 2017, Washington, DC, USA, July 25-27, 2017*. Ed. by Elad Michael Schiller and Alexander A. Schwarzmann. ACM, 2017, pp. 315–324. https://doi.org/10.1145/3087801.3087809.

[54] Rafael Pass and Elaine Shi. "Hybrid Consensus: Efficient Consensus in the Permissionless Model". In: *31st International Symposium on Distributed Computing, DISC 2017, October 16-20, 2017, Vienna, Austria*. Ed. by Andréa W. Richa. Vol. 91. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017, 39:1–39:16. https://doi.org/10.4230/LIPIcs.DISC.2017.39.

[55] Meni Rosenfeld. *Analysis of Hashrate-Based Double Spending*. Preprint. 2014. https://arxiv.org/abs/1402.2009.

[56] Ayelet Sapirshtein, Yonatan Sompolinsky, and Aviv Zohar. "Optimal Selfish Mining Strategies in Bitcoin". In: *Financial Cryptography and Data Security - 20th International Conference, FC 2016, Christ Church, Barbados, February 22-26, 2016, Revised Selected Papers*. Ed. by Jens Grossklags and Bart Preneel. Vol. 9603. Lecture Notes in Computer Science. Springer, 2016, pp. 515–532. https://doi.org/10.1007/978-3-662-54970-4_30.

[57] Yonatan Sompolinsky, Shai Wyborski, and Aviv Zohar. "PHANTOM GHOSTDAG: a scalable generalization of Nakamoto consensus: September 2, 2021". In: *AFT '21: 3rd ACM Conference on Advances in Financial Technologies, Arlington, Virginia, USA, September 26 - 28, 2021*. Ed. by Foteini Baldimtsi and Tim Roughgarden. ACM, 2021, pp. 57–70. https://doi.org/10.1145/3479722.3480990.

[58] Yonatan Sompolinsky and Aviv Zohar. "Secure High-Rate Transaction Processing in Bitcoin". In: *Financial Cryptography and Data Security - 19th International Conference, FC 2015, San Juan, Puerto Rico, January 26-30, 2015, Revised Selected Papers*. Ed. by Rainer Böhme and Tatsuaki Okamoto. Vol. 8975. Lecture Notes in Computer Science. Springer, 2015, pp. 507–527. https://doi.org/10.1007/978-3-662-47854-7_32.

[59] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd edition. Bradford Books, 2018. http://www.incompleteideas.net/book/the-book.html.

[60] Aviv Yaish and Aviv Zohar. "Correct Cryptocurrency ASIC Pricing: Are Miners Overpaying?" In: *5th Conference on Advances in Financial Technologies, AFT 2023, October 23-25, 2023, Princeton, NJ, USA*. ed. by Joseph Bonneau and S. Matthew Weinberg. Vol. 282. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 2:1–2:25. https://doi.org/10.4230/LIPIcs.AFT.2023.2.

[61] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan-Gueta, and Ittai Abraham. "HotStuff: BFT Consensus with Linearity and Responsiveness". In: *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*. Ed. by Peter Robinson and Faith Ellen. ACM, 2019, pp. 347–356. https://doi.org/10.1145/3293611.3331591.

[62] Haifeng Yu, Ivica Nikolic, Ruomu Hou, and Prateek Saxena. "OHIE: Blockchain Scaling Made Simple". In: *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*. IEEE, 2020, pp. 90–105. https://doi.org/10.1109/SP40000.2020.00008.

[63] Ren Zhang and Bart Preneel. "Lay Down the Common Metrics: Evaluating Proof-of-Work Consensus Protocols' Security". In: *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 175–192. https://doi.org/10.1109/SP.2019.00086.

[64] Roi Bar Zur, Ittay Eyal, and Aviv Tamar. "Efficient MDP Analysis for Selfish-Mining in Blockchains". In: *AFT '20: 2nd ACM Conference on Advances in Financial Technologies, New York, NY, USA, October 21-23, 2020*. ACM, 2020, pp. 113–131. https://doi.org/10.1145/3419614.3423264.

Parallel Proof-of-Work with Concrete Bounds

Patrik Keller Rainer Böhme

Abstract

Authorization is challenging in distributed systems that cannot rely on the identification of nodes. Proof-of-work offers an alternative gate-keeping mechanism, but its probabilistic nature is incompatible with conventional security definitions. Recent related work establishes concrete bounds for the failure probability of Bitcoin's *sequential* proof-of-work mechanism. We propose a new family of state replication protocols that use *parallel* proof-of-work. Our bottom-up design from an agreement sub-protocol allows us to give concrete bounds for the failure probability in adversarial synchronous networks. State updates can be sufficiently secure to support commits after one block, removing the risk of double-spending in many applications. We offer guidance on the optimal choice of parameters for a wide range of network and attacker assumptions. Simulations show that the proposed construction is robust even against partial violations of our design assumptions.

Version Notes

This paper was published in the proceedings of the *4th ACM Conference on Advances in Financial Technologies, AFT 2022, in Cambridge, MA*. I here reproduce the published version, with minimal formatting adjustments applied to fit the body of this work. If possible, I recommend reading the original, open-access document instead: `https://doi.org/10.1145/3558535.3559773`

1 Introduction

Bitcoin's use of proof-of-work puzzles to secure state replication *without* relying on the identification of nodes was praised as a technical novelty [2]. While initially supported with heuristic arguments [52], the security of the so-called Nakamoto consensus has been analyzed rigorously over the past decade [20, 30, 32, 45, 56]. All of these works prove *asymptotic* security in various models. Only recently, in AFT '21, Li et al. [50] gave *concrete* security bounds for the failure probability in adversarial synchronous networks. While asymptotic bounds establish that a protocol is secure if one waits "long enough," concrete bounds tell users *how* long they have to wait before accepting a state update as final. All major threats against Bitcoin's security, including double-spending and selfish mining, exploit this uncertainty in some way or another [15, 25, 34, 42].

Nakamoto consensus uses *sequential* proof-of-work: each puzzle refers back to exactly one previous puzzle solution (Fig. 1, left half). A number of *non-sequential* proof-of-work protocols deviate from this scheme to improve throughput or mitigate known security threats [10, 46, 64, 66]. The approaches seem promising, but their design is heuristic. For example, Bobtail [10] argues that multiple puzzles per block imply more regular block intervals which makes it harder for attackers to double-spend. All proposals lack security proofs, let alone concrete bounds. Therefore, one fundamental question remains open: can non-sequential proof-of-work improve the security of state replication?

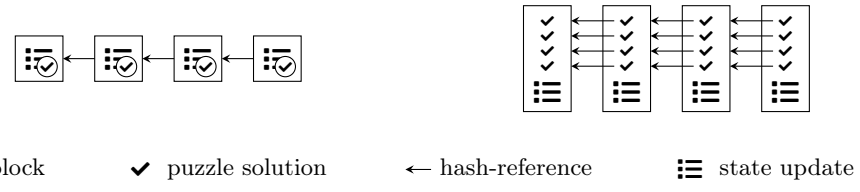


Figure 1: Schematic comparison of sequential (Bitcoin, left) and parallel (proposed, right) proof-of-work blockchains.

This work proposes a principled construction of state replication. We start from the assumptions established by the literature on sequential proof-of-work [20, 30, 32, 45, 50, 56]. We then show how agreement on the latest state can be reached with bounded worst-case failure probabilities. By repeating the agreement procedure, we obtain a family of replication protocols that inherit the concrete error bound. The proposed scheme uses k independent puzzles per block; we thus call it *parallel* proof-of-work (Fig. 1, right half).

To showcase the advantage of parallel proof-of-work, we evaluate a protocol instance that uses $k = 51$ puzzles per block while maintaining Bitcoin's expected block interval of 10 minutes. It guarantees consistency *after one block* up to a defined failure probability of $2.2 \cdot 10^{-4}$ for an attacker with 25 % compute power and two seconds worst-case message propagation delay (cf. Table 3 below in Sect. 2.6). Attacking consistency, e. g., by rewriting an already confirmed block, requires spending work on thousands of blocks without success. For comparison, the optimal configuration of sequential proof-of-work, a "fast Bitcoin" with 7 blocks per minute, has a failure probability of 9 % in the same conditions [50].[1] An attacker would succeed once in roughly every 2 hours.

This paper makes several contributions. We define a family of proof-of-work agreement protocols $\mathcal{A}_k$, provide upper bounds for the worst-case failure probability, and show how to find optimal parameters for different attacker and synchrony parameters. Then we construct a family of replication protocols $\mathcal{B}_k$, which invoke $\mathcal{A}_k$ iteratively to secure a blockchain. We implement $\mathcal{B}_k$ and evaluate it for robustness and security within and beyond the design assumptions using network simulation. We parametrize our simulations to allow for a direct comparison to sequential proof-of-work as used in Bitcoin. We offer guidance on how to parametrize $\mathcal{B}_k$ for other settings. For replicability and future research, we make the protocol and simulation code available online [44].

The paper is organized along these contributions. Section 2 presents and analyzes the agreement protocol. We specify the replication protocol in Section 3 and evaluate it in Section 4. We discuss the relation to the relevant literature, limitations, and future work in Section 5. Section 6 concludes.

2 Proof-of-Work Agreement

Agreement protocols allow their participants to unambiguously agree on a single value [59]. We consider a family of agreement protocols $\mathcal{A}_k$ where the participants cast votes for the value with the most existing votes until k votes exist for the same value. We rate-limit using proof-of-work: each vote requires a puzzle solution. The votes are independent, hence puzzles can be solved in parallel.

In the remainder of this section, we state our model for distributed systems and proof-of-work

[1] Bitcoin as deployed is clearly worse. Li et al. [50] do not even provide a failure probability for the common rule of thumb to wait six blocks (1 hour).

(Sec. 2.1), we specify $\mathcal{A}_k$ (Sec. 2.2), and evaluate its security considering worst-case message scheduling (Sec. 2.3) as well as adversarial behavior (Sec. 2.4). We also guide the choice of parameters (Sec. 2.5) and compare to the best known security bounds for sequential proof-of-work (Sec. 2.6).

2.1 Model

We describe an environment that simulates the execution of $\mathcal{A}_k$ over continuous time. We encode assumptions on computation, communication, and proof-of-work by explicitly defining some aspects of the environment. In other places, we give room for worst-case behavior. For example, we do not restrict the number of participating nodes to reflect a "permissionless" system.

2.1.1 Event-Based Computation

We specify the protocol as a set of event-handlers. The environment maintains the local state for each participating node and it invokes event-handlers for initialization, proof-of-work, and message delivery. Each invocation takes place at a single point in time. We write $\langle$ event $\rangle$ for events without associated data and $\langle$ event | data $\rangle$ otherwise.

The environment invokes the $\langle \text{init} \mid x \rangle$ handler for each node at time 0. The initialization values x can be different for each node. When a node A invokes the procedure TERMINATE(x), the value x becomes A's local result of the agreement protocol. The environment will stop invoking further event-handlers for A. We say A terminated with return value x.

2.1.2 Communication

We adopt the Δ-synchronous communication model from prior analyses of Nakamoto consensus [20, 32, 45, 50, 56]. Nodes share information by calling the BROADCAST procedure. If a node A calls BROADCAST(m) at time t, then the environment invokes the $\langle \text{deliver} \mid m \rangle$ handler on each receiving node $B \neq A$ at or before time $t + \Delta$. This reflects a setting where a network-level attacker can delay any message for up to Δ time.

2.1.3 Proof-of-Work

We adopt the continuous time model for proof-of-work from prior analyses of Nakamoto consensus [20, 50, 60, 66]: the environment has a proof-of-work mechanism P_λ that activates nodes at random times. In stochastic terms, P_λ is a homogeneous Poisson process with rate λ. For this presentation, it is instructive to describe P_λ as a stochastic clock, where the delays between consecutive ticks are independent and exponentially distributed with rate parameter λ. We define $\bar{d} = 1/\lambda$ for the expected delay between consecutive ticks.

Let t_i denote the time of the i-th tick. The environment activates exactly one node per tick at the corresponding time t_i by invoking the $\langle \text{activate} \rangle$ handler. We call this invocation the i-th activation.

In practice, proof-of-work is implemented using hash-based cryptographic puzzles that have to be solved by trial-and-error. Attackers cannot choose *which* node finds the next solution. In our model, the environment may choose which node is activated at each tick. Thereby, we eliminate one source of randomness and replace it with a worst-case realization.

Algorithm 1 Agreement protocol $\mathcal{A}_k$

```
1: upon event ⟨ init | x ⟩ do
2:     p ← x                                          ▷ preferred value
3:     for y ∈ ℕ do votes(y) ← 0
4: upon event ⟨ activation ⟩ do                       ▷ proof-of-work, see Sec. 2.1.3
5:     BROADCAST(vote p)
6:     votes(p) ← votes(p) + 1
7: upon event ⟨ deliver | vote x ⟩ do
8:     votes(x) ← votes(x) + 1
9:     if votes(x) > votes(p) then p ← x
10: upon ∃x | votes(x) ≥ k do
11:    TERMINATE(x)
```

2.2 Protocol $\mathcal{A}_k$

We specify $\mathcal{A}_k$ in Algorithm 1. During initialization, each node sets the preferred value (ln. 2) and initializes the vote counters to zero for all values (ln. 3). Whenever a node is activated by the environment (through proof-of-work), it broadcasts a vote for its preferred value and updates the vote counter accordingly (ln. 5-6). All nodes count the received votes and update their preference to the value with the highest counter (ln. 8-9). After receiving the k-th vote for a value x, the nodes terminate returning x (ln. 11).

Similar protocols exist in the literature on Byzantine Fault Tolerance (BFT). Typically, they rely on round-based execution where each node is activated once per round. Such rate-limiting requires strong identification of all participating nodes. We eliminate this requirement by sourcing the activations from proof-of-work [2].

2.3 Security Against Network-Level Attacks

Good agreement protocols ensure that all nodes terminate (liveness) and that they terminate with the same value (safety) [29]. We analyze how the choice of k affects the liveness and safety of $\mathcal{A}_k$. Our analyses depend on the parameters of the environment, i. e., the maximum propagation delay Δ and the proof-of-work rate λ. For now, we assume that all nodes follow the protocol. We analyze adversarial behavior in Section 2.4.

Liveness is straightforward. For n nodes, there are at most n different initialization values. After $n \cdot k$ activations, at time $t_{n \cdot k} + \Delta$ more specifically, there must be one value for which each of the nodes has at least k votes. This implies termination of all nodes.

Safety is straightforward for the special case $\Delta = 0$. Message broadcast and the corresponding message delivery happen at the same time. At the first activation t_1, the activated node broadcasts a vote for its preferred value. The other nodes receive it immediately and update their preferred value accordingly. From then on, all nodes stay synchronized as they keep voting for the same value. At time t_k all nodes have k votes for the same value and terminate.

For $\Delta > 0$, safety becomes more involved. The broadcast of one vote might overlap with the next activation, two votes might cancel out, and the synchronization can be delayed. In order to analyze these failure modes, we first define our notion of safety.

Definition 1 (Failure)**.** We say inconsistent termination or inconsistency failure, if an execution

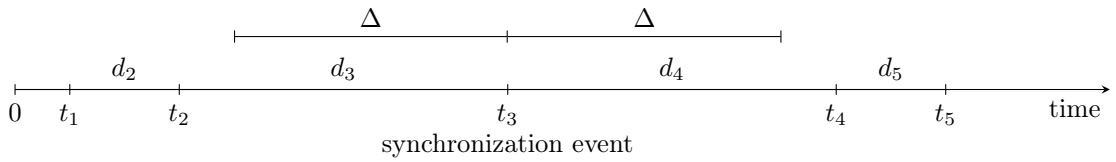


Figure 2: Activation times t_i and activation delays d_i for one particular realization of P_λ. The third activation is a synchronization event.

results in two or more nodes terminating with different values.

Definition 2 (Safety)**.** $\mathcal{A}_k$ is ε-safe for a given parametrization (Δ, λ) of the environment, if the probability that $\mathcal{A}_k$ executed in the environment results in inconsistent termination is at most ε. Probabilities are taken over the realization of the stochastic clock P_λ. Initialization values, message propagation delays, and choice of activated nodes are set to the worst case for the given realization.

To show ε-safety, we first argue that certain realizations $\{t_i\}$ of the random activation times imply synchronization on a single preferred value. We then measure the probability of such realizations.

Definition 3 (Activation delay)**.** Let $\{t_i\}$ be a realization of the random activation times and let $t_0 = -\infty$. We define $d_i = t_i - t_{i-1}$. We call d_i the i-th activation delay.

Definition 4 (Synchronization event)**.** We say that t_i is a synchronization event, if both $d_i > \Delta$ and $d_{i+1} > \Delta$.

Figure 2 illustrates these definitions for one realization. Similar concepts were previously called *uniquely successful round* [30], *convergence opportunity* [56], and *loner* [20, 50].

Proposition 1. *If t_i is a synchronization event, then all running nodes prefer the same value at time $t_i + \Delta$.*

Proof. Let $d_i > \Delta$ and $d_{i+1} > \Delta$. This restriction imposes the following order of events,

$$t_{i-1} < t_{i-1} + \Delta < t_i < t_i + \Delta < t_{i+1} \ . \tag{1}$$

Observe that the first $i - 1$ votes are fully propagated at the time of the i-th activation. Just before the i-th activation, all nodes see the same votes. The order of votes does not matter. If nodes prefer different values, then there is a tie between these preferred values.

One node is activated at time t_i and votes for its preferred value x. The other nodes receive the vote until $t_i + \Delta$. Receiving nodes that prefer x leave their preference unchanged. Receiving nodes that prefer a different value adopt x because the new vote is breaking the tie. Activation $i + 1$ happens later, thus there is no other vote that can interfere. □

Proposition 2. *Let $\{t_i\}$ be a realization where the first synchronization event happens before t_{2k}. Then all nodes running $\mathcal{A}_k$ return the same value.*

Proof. Two nodes terminating with different values requires at least $2k$ votes (Alg. 1, l. 10). Let t_i denote the first synchronization event in $\{t_i\}$ and let $i < 2k$. At time t_i, less than $2k$ votes exist and all nodes are aware of all existing votes. If one node has terminated returning x, then all nodes have terminated returning x. Otherwise, all nodes are still running. By Proposition 1 all nodes prefer the

Table 1: Discrete state transitions modelling Proposition 2.

	state after i transitions	state after $i \to i+1$	
#	interpretation	$d_{i+2} \leq \Delta$	$d_{i+2} > \Delta$
s_1	$d_{i+1} \leq \Delta \wedge$ no synchronization event	s_1	s_2
s_2	$d_{i+1} > \Delta \wedge$ no synchronization event	s_1	s_3
s_3	$\exists j \leq i \mid t_j$ is synchronization event	s_3	s_3

same value y at time $t_i + \Delta$. Nodes activated at or after t_{i+1} will vote for y until all nodes terminate returning y. □

Proposition 2 provides a sufficient condition for consistency which depends on the realization of the stochastic clock. To measure the space of realizations satisfying this condition, we construct a discrete Markov chain with three states. The random state transitions happen at the ticks of the stochastic clock P_λ. Before the first synchronization event, we use two states to track whether the last delay was greater than Δ (state s_2) or not (state s_1). The model enters the terminal state s_3 if two consecutive delays are greater than Δ. Since $d_1 = \infty$ by Definition 3, we set the start state to s_2. By construction, the Markov chain is in state s_3 after i transitions if and only if there was a synchronization event at or before time t_i. Table 1 describes the states and transitions more formally.

For $i > 1$, the activation delays d_i are independent and exponentially distributed with rate λ by the definition of the stochastic clock (Sec. 2.1.3). The probability that $d_i \leq \Delta$ is $1 - e^{-\lambda\Delta}$. This gives us the Markov chain depicted in Figure 3.

Proposition 3 (Safety)**.** *$\mathcal{A}_k$ is $b_0(\lambda, \Delta, k)$-safe for*

$$\boldsymbol{M}(\lambda, \Delta) = \begin{pmatrix} 1 - e^{-\lambda\Delta} & e^{-\lambda\Delta} & 0 \\ 1 - e^{-\lambda\Delta} & 0 & e^{-\lambda\Delta} \\ 0 & 0 & 1 \end{pmatrix}, \tag{2}$$

$$\boldsymbol{v} = \begin{pmatrix} 0 & 1 & 0 \end{pmatrix}, \text{ and} \tag{3}$$

$$b_0(\lambda, \Delta, k) = 1 - \left(\boldsymbol{v} \times \boldsymbol{M}(\lambda, \Delta)^{2k-1}\right)[3], \tag{4}$$

where [3] *denotes selection of the third element.*

Proof. $\mathbf{M}(\lambda, \Delta)$ describes the Markov chain depicted in Figure 3 in matrix form. We assign vector $\mathbf{v}$ to initialize the Markov model in state s_2. The third element of the result of $\mathbf{v} \times \mathbf{M}(\lambda, \Delta)^{2k-1}$ describes the probability that the Markov model is in state s_3 after $2k - 1$ random transitions. Our claim follows by the construction of the Markov chain, Definition 2, and Proposition 2. □

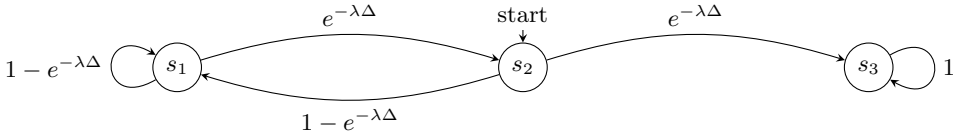


Figure 3: Graphical representation of the Markov chain.

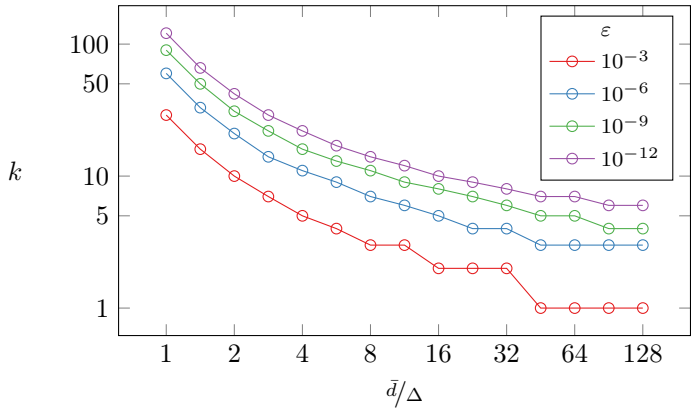


Figure 4: Minimum k, such that $\mathcal{A}_k$ is ε-safe by Proposition 3.

Observe that b_0 depends only on the vote threshold k and the product of λ and Δ. It is instructive to interpret λ as the inverse of the expected activation delay $\bar{d}$ and use the synchrony parameter Δ as a unit of time. Figure 4 visualizes the bound $b_0(\lambda, \Delta, k)$ for different combinations of $\bar{d}$ (as multiple Δ) on the x-axis and k on the y-axis. Both parameters have a positive effect on the safety of $\mathcal{A}_k$.

Proposition 3 guarantees ε-safety for $\mathcal{A}_k$ against strong network-level attacks. We now extend the argument to adversarial voting.

2.4 Security Against Malicious Voting

We now consider attackers who can cast votes. While equivocation is a major concern in the setting with identified nodes, the use of proof-of-work in $\mathcal{A}_k$ completely removes this issue [2]: in practice, every vote is authenticated with a costly solution to a computational puzzle [22]. Therefore, the only remaining attack strategy is to withhold votes to release them later. We address this next.

2.4.1 Attacker Votes

We assume that the attacker controls at most α of the total proof-of-work capacity.[2] We model this by splitting the stochastic clock P_λ in two independent parts P_A and P_D. The clock P_A ticks at rate $A = \alpha \cdot \lambda$. With each tick, the attacker gains one *attacker vote* which can be withheld arbitrarily before being sent. The clock P_D ticks at rate $D = (1 - \alpha) \cdot \lambda$ and results in activations of defender nodes which follow the protocol as specified.

Recall that the sum of multiple Poisson processes is another Poisson process with cumulated rate. The probability that an individual tick of $P_\lambda = P_{A+D}$ results in an attacker vote is α.

2.4.2 Malicious Voting

We study withholding attacks by considering two phases. In the **balancing** phase nodes are not yet synchronized on the same preferred value. Recall that without malicious voting, the nodes would synchronize at the first synchronization event (see Prop. 1). A vote-withholding attacker can prevent

[2]The analysis generalizes to multiple attackers: we conservatively assume that all attackers coordinate perfectly and sum up their shares.

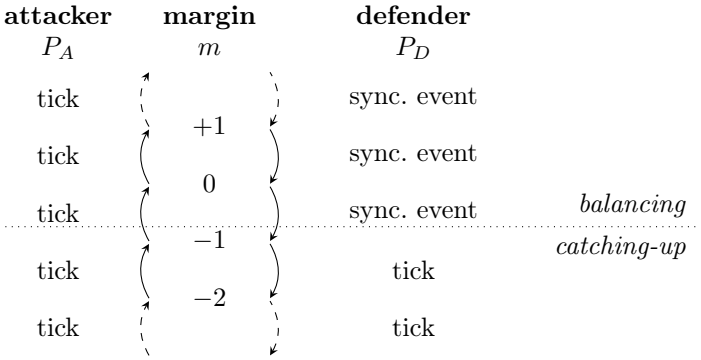


Figure 5: Security of $\mathcal{A}_k$: attacker's margin and the transition between *balancing* and *catching-up* phases.

synchronization by releasing withheld votes around synchronization events. The balancing phase continues while the attacker can balance synchronization events with his stock of withheld votes. If the attacker does not release a withheld vote around a synchronization event, e. g. because his stock is empty, the nodes synchronize. This is when the attack transitions to the *catching-up* phase.

During the **catching-up** phase, all nodes prefer the same value. With each tick of the stochastic clock P_D, the nodes cast one vote for this value and thereby reinforce the synchronization. However, the attacker can destroy the synchronization by releasing sufficiently many votes for a different value. If this happens, the attack transitions back to the *balancing* phase.

Both phases can be characterized with an integer depth. In the *balancing* phase it matters how many votes are currently withheld by the attacker. In the *catching-up* phase it matters how many votes the attacker has to cast in order to destroy the synchronization. Our attack model tracks these depths in a single **margin** variable m. Positive m represent withheld votes during *balancing* and negative m represent number of votes to be *caught-up* (see Fig. 5).

2.4.3 Markov Chain Model

We quantify the safety of $\mathcal{A}_k$ against malicious voting by measuring the space of realizations of the joint clock P_λ where withholding and propagation delays enable inconsistent termination. We generalize the Markov chain model of Section 2.3 to include the states for different margins m. The new state space is $(m, s) \in \mathbb{Z} \times \{\bot, \top\}$. To track the occurrence of synchronization events, we set $s = \top$ if and only if the last activation delay was greater Δ. The initial state is $(0, \top)$: zero votes are withheld and $s = \top$ since $d_1 > \Delta$ by Definition 3.

As before, transitions happen at each tick. For attacker votes (probability α), we increment m. Depending on the phase, the attacker withholds (increasing the stock of withheld votes) or catches up by one; both map to the same transition. For defender activations (probability $1 - \alpha$), we distinguish the two phases. In the *balancing* phase ($m \geq 0$), we only decrement m if the tick is a synchronization event. In the *catching-up* phase ($m < 0$), we always decrement m. Figure 6 illustrates the state transitions and transition probabilities.

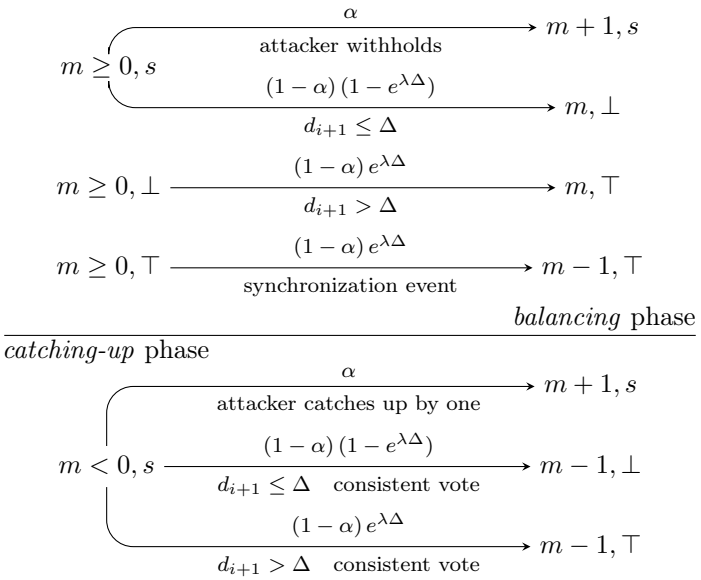


Figure 6: Generalized Markov chain model for attackers who can send and withhold votes. Transition probabilities are annotated above the arrows and interpretations below.

2.4.4 Numerical Solution

In principle, within l steps, the model can reach any state with $-l \leq m \leq l$. Calculating the exact state probabilities after l transitions requires us to raise a square matrix with $n = 2(2l+1)$ rows to the power of l. Each matrix multiplication is $O\left(n^{2.8}\right)$ [67]. Thus, the analysis is infeasible for larger l.

We set a cut-off at $m = \pm 25$ to make the problem tractable. We assume that an attacker who manages to withhold 25 votes during the *balancing* causes inconsistent termination. Similarly, an attacker lagging behind 25 votes in the *catching-up* phase cannot catch up at all. With these assumptions, the number of states is bounded by 102 and the matrix multiplications stay tractable. Using such cut-offs is common practice in the related literature [34, 61].

A second simplification in this model is that it does not track how many votes are cast for each value. Adding this information would blow up the state space excessively. We work around this problem by ignoring the termination rule of $\mathcal{A}_k$ and assume that the nodes continue voting forever. We thus need to rephrase our notions of success and failure for the purpose of this analysis.

Recall that inconsistent commits require at least $2k$ votes. We count an execution as successful if all nodes prefer the same value after $2k$ steps. This is easy to check by inspecting the phase after $2k$ transitions: *catching-up* means success and *balancing* means failure.

We calculate the failure probability of $\mathcal{A}_k$ for different combinations of α, Δ, $\lambda = 1/\bar{d}$, and k by exponentiation of the probability matrix of the generalized Markov chain model. We visualize this in Figure 7, following the setup of Figure 4, but with more lines for different assumptions of attacker strength α. As expected, increasing α pushes up the required k for any given failure bound ε and expected activation delay $\bar{d}$. For example, assuming a proof-of-work puzzle takes 8 times the maximum propagation delay, while without attacker, $k = 3$ were sufficient for 10^{-3}-safety, k must increase to $k = 9$ if an attacker is present and controls 10 % of the proof-of-work capacity; or to

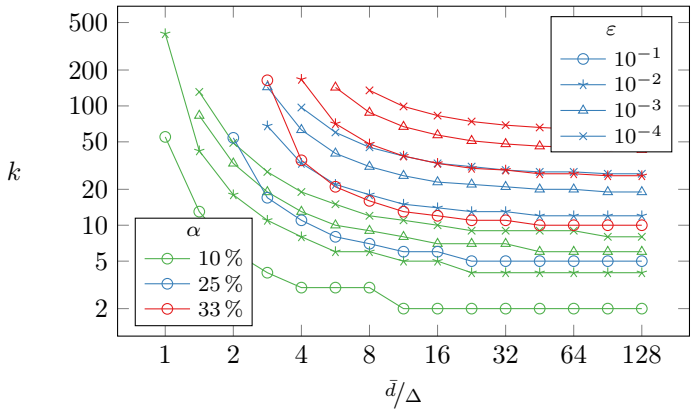


Figure 7: Minimal k such that $\mathcal{A}_k$ satisfies the given failure probability bound ε for a given attacker and expected activation delay $\bar{d}$ as multiple of Δ.

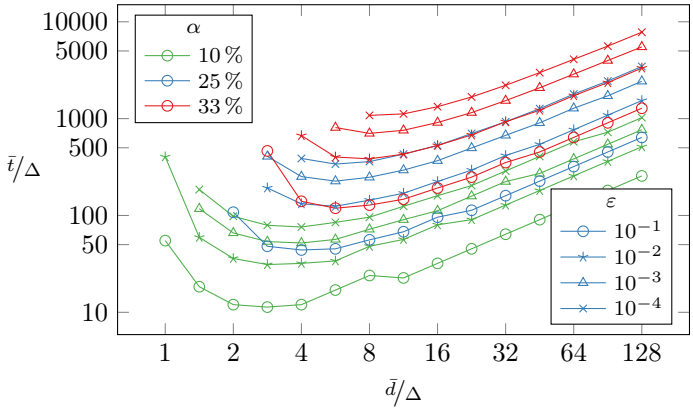


Figure 8: Protocol runtime after choosing the minimal k such that $\mathcal{A}_k$ satisfies the given failure probability bound ε for a given attacker and expected activation delay $\bar{d}$. Both axes show times as multiples of Δ.

$k = 88$ for 33 % attacker strength.

In practice, a protocol designer can adjust the puzzle difficulty and should care about the protocol runtime, to which we turn next.

2.5 Choosing Efficient Parameters

The aim here is to guide the choice of protocol parameters to minimize the protocol runtime for given assumptions about the real world. $\mathcal{A}_k$'s failure probability depends on the synchrony parameter Δ, the proof-of-work rate λ, the attackers compute power α, and the threshold k. The protocol operator can choose λ and k, while Δ and α are worst-case assumptions. Safety increases with k or by decreasing λ. But both options slow down termination: either we wait for more votes or we wait longer for each vote.

Recall that the protocol runtime is stochastic. Termination requires k votes for the same value and thus at least k activations. The time of the k-th activation is the sum of k exponentially

Table 2: Configurations of $\mathcal{A}_k$ minimizing protocol runtime $\bar{t}$.

α	ε	$\bar{d}/\Delta$	k	$\bar{t}/\Delta$
0	10^{-1}	3.0	2	6
0	10^{-2}	2.6	5	13
0	10^{-3}	2.5	8	20
0	10^{-4}	2.7	10	27
$1/10$	10^{-1}	2.8	4	11
$1/10$	10^{-2}	3.3	9	29
$1/10$	10^{-3}	3.2	16	51
$1/10$	10^{-4}	3.8	20	75

α	ε	$\bar{d}/\Delta$	k	$\bar{t}/\Delta$
$1/4$	10^{-1}	4.0	10	40
$1/4$	10^{-2}	5.1	24	123
$1/4$	10^{-3}	5.7	40	226
$1/4$	10^{-4}	5.8	58	339
$1/3$	10^{-1}	6.1	19	115
$1/3$	10^{-2}	7.4	51	375
$1/3$	10^{-3}	7.9	88	699
$1/3$	10^{-4}	8.8	121	1067

We set attacker α and failure bound ε. We optimize activation delay $\bar{d}$ and threshold k. The synchrony parameter Δ serves as unit of time.

distributed delays, i. e., gamma distributed with shape parameter k. Whenever nodes vote for different values—due to network delays or withholding—termination requires more activations and the shape parameter of the gamma distribution increases.

We optimize the protocol runtime for the optimistic case where k activations enable termination. We call $\bar{t} = k \cdot \bar{d} = k/\lambda$ the optimistic expected protocol runtime. Figure 8 shows $\bar{t}$ (in multiples of Δ) for the same parameters as used in Figure 7. Observe that depending on ε and α, different activation delays $\bar{d}$ minimize the protocol runtime.

As the curves in Figure 8 are neither convex nor continuous and expensive to evaluate, we identify the minima using Bayesian optimization [54] and report them in Table 2.

Now we see that in the above example ($\varepsilon = 10^{-3}$, $\alpha = 1/10$), the protocol runtime can be reduced from $\bar{t} = 72\,\Delta$ to $51\,\Delta$ without sacrificing safety by choosing $(k, \bar{d}) = (16, 3.2\,\Delta)$ instead of $(9, 8\,\Delta)$. For perspective, with a network latency bound $\Delta = 2$ seconds, the puzzle difficulty should be adjusted to one solution every 6.4 seconds. The protocol $\mathcal{A}_{16}$ would terminate in about 102 seconds.

2.6 Comparison to Sequential Proof-of-Work

In their contribution to AFT '21, Li et al. [50] provide concrete bounds for the failure probability of Nakamoto consensus. The "achievable security latency function" $\bar{\epsilon}(t)$ for Nakamoto consensus as stated in [50, Theorem 3.5] provides an upper bound for the failure probability after waiting for a given confirmation time t. Our models and assumptions are compatible, but we derive failure probabilities after termination and hence after stochastic runtime.

To enable comparison of sequential and parallel proof-of-work, we fix the time frame to $\bar{t} = 10$ minutes. We also fix the attacker α and propagation delay Δ. For parallel proof-of-work, we optimize k for minimal failure probability of $\mathcal{A}_k$ subject to $\bar{d} \cdot k = \bar{t}$. The resulting configuration exhibits an expected protocol runtime of 10 minutes. For sequential proof-of-work, we optimize the block interval $\bar{d}_{\text{seq}}$ for minimal failure probability after 10 minutes.

Table 3 compares the failure probability ε of $\mathcal{A}_k$ with the achievable security ε_{seq} of Nakamoto consensus for various combinations of Δ and α. Note that Li et al. [50] do not define $\bar{\epsilon}(t)$ for all combinations of α and Δ. We omit the undefined values from the table. Observe that our concrete bounds for parallel proof-of-work consistently outperform the bounds for sequential proof-of-work by

Table 3: Advantage of parallel over sequential proof-of-work.

Parameters		Parallel			Sequential [50]		
Δ	α	k	$\bar{d}$	ε	$\bar{k}_{\text{seq}}$	$\bar{d}_{\text{seq}}$	ε_{seq}
1	$1/10$	77	7.8	$6.3 \cdot 10^{-20}$	192.7	3.1	$9.8 \cdot 10^{-15}$
1	$1/4$	95	6.3	$7.3 \cdot 10^{-7}$	136.6	4.4	$1.8 \cdot 10^{-3}$
1	$1/3$	76	7.9	$1.9 \cdot 10^{-3}$	103.5	5.8	$2.6 \cdot 10^{-1}$
2	$1/10$	76	7.9	$3.9 \cdot 10^{-13}$	96.9	6.2	$3.4 \cdot 10^{-7}$
2	$1/4$	51	11.8	$2.2 \cdot 10^{-4}$	68.8	8.7	$8.8 \cdot 10^{-2}$
2	$1/3$	43	14.0	$1.8 \cdot 10^{-2}$	–	–	–
4	$1/10$	39	15.4	$1.2 \cdot 10^{-7}$	49.0	12.2	$1.8 \cdot 10^{-3}$
4	$1/4$	28	21.4	$5.3 \cdot 10^{-3}$	34.8	17.2	$5.2 \cdot 10^{-1}$
4	$1/3$	24	25.0	$6.9 \cdot 10^{-2}$	–	–	–

Δ propagation delay, seconds
α attacker's compute
k number of votes
$\bar{d}$ activation delay, seconds
ε failure probability of $\mathcal{A}_k$
$\bar{d}_{\text{seq}}$ block interval, seconds
$\bar{k}_{\text{seq}}$ expected number of blocks
ε_{seq} failure prob. after 10 minutes

at least two orders of magnitude.

Remark 1. The true advantage is smaller because Li et al. [50] consider a long-running blockchain protocol, whereas our agreement protocol has a limited time horizon. Moreover, concurrent work presents improved bounds for sequential proof-of-work [33, 37].

3 Proof-of-Work Blockchain

Protocol $\mathcal{A}_k$ solves agreement using parallel proof-of-work. In this section, we propose a replication protocol $\mathcal{B}_k$ that repeatedly runs $\mathcal{A}_k$ to continuously agree on a growing sequence of values. In a nutshell, $\mathcal{B}_k$ is a blockchain protocol where the participants use $\mathcal{A}_k$ to agree on each appended block.

Time is divided in epochs of variable length, determined by the runtime of $\mathcal{A}_k$. Each epoch extends the blockchain by one block and confirms the value of the preceding epoch's block using $\mathcal{A}_k$. The safety guarantees of $\mathcal{A}_k$ imply that possible conflicting block proposals for the current epoch are resolved in the next epoch.

3.1 Prerequisites

In addition to the network assumptions of $\mathcal{A}_k$ (Sect. 2.1.2), we assume interfaces to an application layer and the availability of cryptographic primitives.

3.1.1 Application

$\mathcal{B}_k$ enables state replication and may serve as a basis for different applications [2, 48, 62]. For example, a simple cryptocurrency could append a list of transactions to each block. Jointly, the confirmed blocks would form a distributed ledger. More advanced applications could add scalability layers that only replicate selected decisions using $\mathcal{B}_k$ while handling other state updates separately [24, 46, 58].

We require that the application offers an interface with two procedures. getUpdate returns a valid state update that $\mathcal{B}_k$ can use for block proposals. applyUpdate passes replicated state updates

to the application. The application may have other means to access the broadcast network directly. For example, cryptocurrencies share transactions provisionally before they are written in blocks.

3.1.2 Cryptography

$\mathcal{B}_k$ uses cryptographic hash functions for the hash-linking of blocks and the proof-of-work puzzle. The hash function used for the linking must be cryptographically secure. The hash function used for the proof-of-work puzzle requires the same stronger assumptions as in Nakamoto consensus [2]. In principle, one could separate these concerns and use two different hash functions. For simplicity, we use a single hash function $\mathcal{H}$ satisfying both requirements. The reader can safely assume $\mathcal{H} = \text{SHA3}$.

In addition, $\mathcal{B}_k$ uses a secure digital signature scheme [43, Def. 12.1, p. 442] given by the procedures generateKeyPair, checkSignature, and sign.

3.2 Protocol $\mathcal{B}_k$

We start with presenting core aspects of $\mathcal{B}_k$ in Sections 3.2.1 to 3.2.8 and integrate them into a complete protocol in Section 3.2.9.

3.2.1 Votes

A vote is a triple (r, p, s), with $\mathcal{H}(r, p, s) \leq t_{\text{v}}$. We say r is the value voted for, p is the public key of the voting node, and s is the proof-of-work puzzle solution. The threshold t_{v} represents $\mathcal{B}_k$'s proof-of-work difficulty parameter and is set externally.

3.2.2 Quorums

A k-quorum is a set of k valid votes for the same value. A list $Q = \{(p_i, s_i)\}$ represents a valid k-quorum for value r, if the following conditions hold:

1. $|Q| = k$
2. $\forall\, 1 \leq i \leq k\colon \mathcal{H}(r, p_i, s_i) \leq t_{\text{v}}$
3. $\forall\, 1 \leq i < k\colon \mathcal{H}(r, p_i, s_i) < \mathcal{H}(r, p_{i+1}, s_{i+1})$

The first condition defines the quorum size k. The second condition ensures that all votes are valid. The third condition eliminates duplicates and imposes a canonical order which we use for leader selection. We write $Q[1]$ to address the first vote in the quorum.

Remark 2. The above definitions allow for single nodes providing multiple votes to a single quorum using the same public key. This is intentional. Sybil attacks are mitigated by the scarcity of votes, not by the scarcity of public keys.

3.2.3 Leader Selection

We say that node A is leader for the epoch that produces Q if A contributed the smallest vote $Q[1]$. Only leaders are allowed to propose new blocks. Nodes verify leadership based on the public key p_1, which is part of $Q[1]$.

Remark 3. Leader selection originates from the distributed system literature (e. g. [13, 31, 55, 68]), where it is used to improve performance in the optimistic case that the leader follows the rules. A similar, leader-based performance improvement has been proposed for Nakamoto consensus [24]. Our leader selection mechanism is an optimization as well. It reduces the number of proposals per epoch and thereby improves communication efficiency. Recall that the agreement protocol $\mathcal{A}_k$ resolves conflicting preferences even if all nodes started with their own preferred value (Sect. 2). Thus, $\mathcal{B}_k$ is secure even if leaders equivocate or multiple leaders are selected.

3.2.4 Blocks

A block is a proposed extension to the blockchain. Besides the application payload, a block holds additional values that ensure orderly execution of the agreement $\mathcal{A}_k$ and the leader selection mechanism according to Sections 3.2.1 to 3.2.3. A valid block b contains the following information.

1. parent(b) is either the hash of a previous valid block or equal to the protocol parameter H_0, which characterizes the instance of the protocol;[3]
2. quorum(b) is a valid k-quorum for parent(b);
3. payload(b) is the proposed state update returned from GETUPDATE;
4. signature(b) is a valid signature of the triple (parent(b), quorum(b), payload(b)) signed with the private key corresponding to the public key in quorum(b)[1].

The first condition imposes a sequential order on the list of blocks. The second condition ensures that all nodes agree on the previous block before proposing a new block ($\mathcal{A}_k$, Sect. 2). The forth condition restricts the ability to propose blocks to selected leaders (Sect. 3.2.3).

3.2.5 Local Block Tree

Each node locally maintains a hash-linked *tree* of blocks $\mathcal{T}$. We write $\mathcal{T}[h]$ to access the block b with $\mathcal{H}(b) = h$. For each block b, nodes maintain

1. height(b), the number of predecessors of b in $\mathcal{T}$,
2. state(b), the application state associated with b, and
3. votes(b), the set of votes that confirm b.

3.2.6 Block Preference

Nodes prefer block a over block b if,

1. height(a) > height(b), or
2. height(a) = height(b) $\wedge$ | votes(a)| > | votes(b)|.

In other words, they follow the longest chain (1) between epochs and the voting protocol $\mathcal{A}_k$ within each epoch (2). This rule is ambiguous if there are multiple blocks of equal height and with the same number of confirming votes. In this case, nodes prefer the block first received. The embedded voting protocol $\mathcal{A}_k$ makes the nodes agree on the same parent block until the end of the epoch.

Remark 4. Under normal operation with a constant set of nodes (i. e., no late joining), the longest chain rule will only be invoked to disambiguate the last epoch. The ε-safety guarantee of $\mathcal{A}_k$ ensures that longer forks are unlikely.

[3] In blockchain jargon, H_0 is the hash of the genesis block.

3.2.7 Proof-of-Work Voting

Nodes continuously try to find and share valid votes for their preferred block. Recall that a valid vote $v = (\mathcal{H}(b), p, s)$ satisfies $\mathcal{H}(v) \leq t_{\mathrm{v}}$, where b is the preferred block and p is the node's public key. Due to the properties of the hash function (Sect. 3.1.2), the best solution strategy is iterative trial and error for different values of s. Solving this hash puzzle on physical hardware implements the stochastic clock P_λ presented in Section 2.1.3 for the arrival of votes in a distributed system. Parameter t_{v} must be adjusted to the desired puzzle solving rate λ for a given technology and proof-of-work capacity.

3.2.8 Proposing

Nodes assume leadership whenever possible. I. e., they constantly check whether they can form a quorum Q where the smallest vote $Q[1]$ is their own. If so, they request a state update from the application, integrate it as payload into a new valid block (Sect. 3.2.4), and broadcast it.

3.2.9 Integration

Algorithm 2 integrates Sections 3.2.1 to 3.2.8 into the complete protocol $\mathcal{B}_k$. During initialization, nodes generate a key pair for the digital signature scheme (Sect. 3.1.2) and initialize the empty block tree (ln. 1–3). Two event-handlers process incoming messages (ln. 4 and 6). Valid votes are stored (ln. 10–12) and valid blocks are appended to the local block tree (ln. 13–19). In the background, nodes continuously try to solve proof-of-work puzzles in order to cast votes for their preferred value (ln. 36–42). The path between H_0 and the preferred value in the local block tree represents the current version of the blockchain. Whenever the local block tree changes (ln. 27 triggered from ln. 12 and 40), nodes try to assume leadership and propose a new block (ln. 29–35).

We visualize a typical execution of $\mathcal{B}_k$ in Figure 15 in the Appendix and compare it to sequential proof-of-work.

Remark 5. $\mathcal{B}_1$, i. e., $\mathcal{B}_k$ with $k = 1$, closely resembles Bitcoin as proposed by Nakamoto [52]. However, we highlight one key difference: in Bitcoin, blocks carry a first proof-of-work confirmation of the payload proposed within the block itself. In $\mathcal{B}_k$, the proof-of-work solutions confirm the *previous* block. This enables parallel puzzle solving for $k > 1$.

3.3 Finality

Finality means that the application accepts a commit when it is deemed safe. Deterministic finality is not achievable with stochastic protocols, but our concrete safety bounds help to make an informed decision on when to accept. By implementing $\mathcal{A}_k$ in $\mathcal{B}_k$, we ensure that the commit of the state update in a block with height i is ε-safe as soon as a block with height $i + 1$ is observed.

For example, the configuration $\alpha = 1/4$, $k = 51, \bar{t} = 600$ from Table 3 is 0.0002-safe. This implies that the worst case attacker (within the model) succeeds in causing inconsistent commits in one of 5,000 attempts. In practice, such an attacker would find it easier to temporarily increase the share in compute power above $\alpha = 1/2$, where every system solely based on proof-of-work fails. With proof-of-work capacity being available for rent [9], this turns into an economic argument which is orthogonal to the design assumptions of $\mathcal{B}_k$. This leads us to a brief discussion of incentives.

Algorithm 2 Blockchain protocol $\mathcal{B}_k$

```
1:  upon event ⟨ init ⟩ do
2:      pk, sk ← generateKeyPair( )
3:      height(T[H_0]) ← 0                                  ▷ H_0: hard-coded value
4:  upon event ⟨ deliver | vote v ⟩ do                       ▷ from line 42
5:      count(v)
6:  upon event ⟨ deliver | block b ⟩ do                     ▷ from line 35
7:      for all (p, s) in quorum(b) do                      ▷ count all votes
8:          count(parent(b), p, s)
9:      store(b)
10: procedure count(vote (r, p, s))
11:     if H(r, p, s) ≤ t_v then                            ▷ vote validity, Sect. 3.2.1
12:         votes(T[r]) ← votes(T[r]) ∪ {(p, s)}
13: procedure store(block b)
14:     if b is valid by Sect. 3.2.4 and H(b) ∉ T then
15:         votes(b) ← ∅
16:         a ← T[parent(b)]
17:         height(b) ← height(a) + 1
18:         state(b) ← applyUpdate(state(a), payload(b))
19:         T[H(b)] ← b
20: procedure preferred
21:     Find most preferred block b according to Sect. 3.2.6.
22:     return H(b)
23: procedure leader(r)
24:     Try to form k-quorum Q for r as leader by Sect. 3.2.3.
25:     if possible then return Q
26:     else return ⊥
27: upon change to T do
28:     r ← preferred ; Q ← leader(r)
29:     if Q then                                           ▷ build block according to Sect. 3.2.8
30:         parent(b) ← r
31:         quorum(b) ← Q
32:         payload(b) ← getUpdate(state(T[r]))
33:         signature(b) ← sign(b, sk)
34:         store(b)
35:         broadcast(block b)
36: procedure proofOfWork                                   ▷ background task
37:     loop
38:         r ← preferred ; s ← s + 1
39:         if H(r, pk, s) ≤ t_v then
40:             votes(T[r]) ← votes(T[r]) ∪ {(pk, s)}
41:             if not leader(r) then
42:                 broadcast(vote (r, pk, s))
```

3.4 Incentives

It is possible to motivate participation in $\mathcal{B}_k$ by rewarding puzzle solutions. This requires some kind of virtual asset that can be transferred to a vote's public key. Claiming the reward would depend on the corresponding private key.

$\mathcal{B}_k$ could allot a constant reward per puzzle. As votes occur k times more frequently than blocks, $\mathcal{B}_k$'s mining income would be less volatile than in Nakamoto consensus, making participation more attractive to risk-averse agents with small compute power.

It is tempting to demand that the reward scheme is incentive compatible, i. e., that correct execution is a unique maximum of the nodes' utility function. However, it is not trivial to achieve incentive compatibility because utility of rewards *outside* the system may affect the willingness to participate *in* the system [27]. We do not know any blockchain protocol analysis that solves this problem convincingly. Thus, $\mathcal{B}_k$ is designed to support rewards as a means to encourage participation, but its security intentionally does not depend on incentives. This is a feature, not a bug.

4 Evaluation

We evaluate $\mathcal{B}_k$ by discrete event network simulation. We implement $\mathcal{B}_k$ and the network simulation in OCaml. All results are reproducible with the code provided online [44].

We choose the configuration $k = 51$ and $\lambda = {}^{51}/_{600}$, which is optimized for $\alpha = {}^{1}/_{4}$ and $\Delta = 2''$. Its failure probability is at most $2.2 \cdot 10^{-4}$ (see Sect. 2.6, Tab. 3). The expected block interval is 10 minutes, which enables comparison to Nakamoto consensus, more specifically Bitcoin. For the purpose of this simulation, Bitcoin is equivalent to $\mathcal{B}_1$ with $\lambda = {}^{1}/_{600}$ (see Sect. 3 Remark 5).

While the worst-case propagation delay Δ is specified at design time, realistic network latencies vary. In the simulation, we set an expected network delay δ and use it to draw individual delays for each message delivery from

1. a **uniform** distribution on the interval $[0, 2 \cdot \delta]$, and
2. an **exponential** distribution with rate δ^{-1}.

We also consider that votes may propagate faster than blocks because they are much smaller and their validation does not depend on the application state. To this end we define

1. a **simple** treatment where $\delta = \Delta = 2''$ for all messages, and
2. a **realistic** treatment where blocks propagate with $\delta_{\mathbf{b}} = 2''$ and votes eight times faster, $\delta_{\mathbf{v}} = {}^{1}/_{4}''$.

The cross product of the two distributions and two treatments of small messages gives us four scenarios to be simulated. Note that for all scenarios some delays will be greater than the assumed worst-case propagation delay Δ. For some measurements, we will raise δ beyond Δ to put the protocol under even more pressure.

Unless stated otherwise, measurements are based on a simulated network with 1024 nodes.[4] For each experiment, we average over 64 independent executions up to block height 4096. All figures showing variation do this by plotting ± 1.96 standard deviations around the mean of the 64

[4] Measurements suggest that there are roughly 10 000 Bitcoin nodes, while 80 % of the compute power is held by the top 10 agents [51].

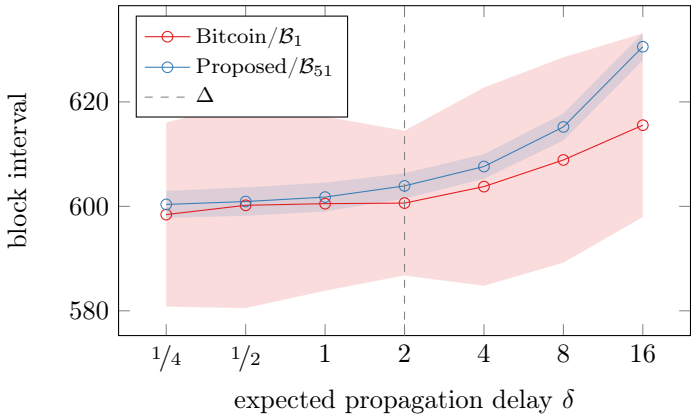


Figure 9: The effect of latency on the average block interval in the **simple/exponential** scenario. Time in seconds.

independent executions. For all executions of $\mathcal{B}_{51}$, we checked for inconsistent commits, which did not occur. As another plausibility check, we verified that the simulated block intervals of $\mathcal{B}_1$ and $\mathcal{B}_{51}$ match the theoretical distributions described in Section 2.5.

4.1 Robustness

We evaluate the robustness of $\mathcal{B}_{51}$ against excessive latency, churn, and leader failure by measuring block intervals. Recall that the simulated protocols are configured for a 10 minute interval in optimal conditions. The puzzle solving rate is constant. Stress implies wasted proof-of-work and hence higher observed block intervals.[5]

4.1.1 Latency

We use the **simple/exponential** scenario and vary the expected propagation delay δ from $1/4$ to 16 seconds. Recall that the choice of $k = 51$ is optimized for $\Delta = 2$ seconds. Larger expected propagation delays put the protocol under stress. Figure 9 shows the effect of latency on the block interval. We observe that even excessive random propagation delays ($\delta = 16$ seconds) slow down $\mathcal{B}_{51}$-consensus by only about 5 %. The **simple/uniform** scenario exhibits similar behavior. We refrain from exploring the **realistic** treatment as it is not obvious how real network latency would affect the ratio of $\delta_{\mathbf{b}}$ and $\delta_{\mathbf{v}}$.

4.1.2 Churn

We simulate churn by muting a fraction of nodes (churn ratio) for one hour each. Muted nodes solve proof-of-work puzzles but do not send or receive messages. Accordingly, the votes and blocks created by muted nodes represent lost work. We expect that the block interval is inversely proportional to the churn ratio: if 50 % of the nodes are muted, the average block interval is twice as long. Figure 10 supports this claim for both protocols.

[5]This metric relates to the orphan rate in the literature on sequential proof-of-work.

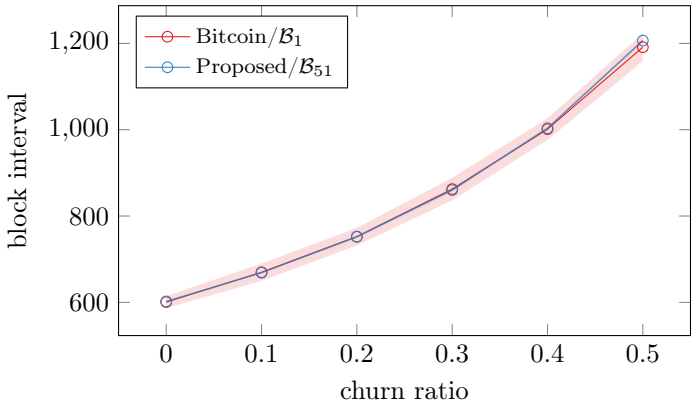


Figure 10: The effect of churn on the average block interval in the **realistic/exponential** scenario. The churn ratio describes how many of the nodes are passive at any given time.

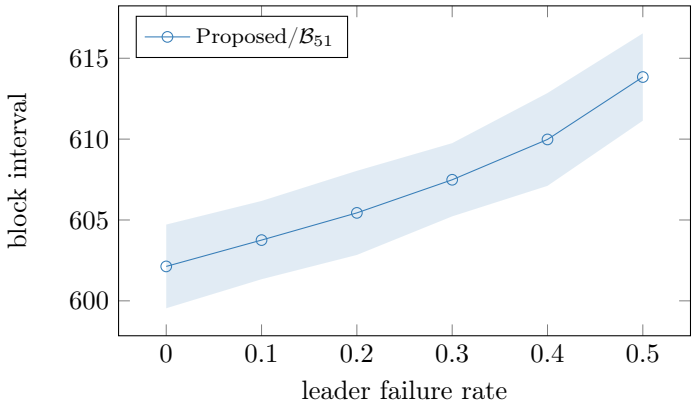


Figure 11: The effect of leader failure on the block interval in the **realistic/exponential** scenario. The leader failure rate is the probability that a selected leader fails to propose a block.

4.1.3 Leader Failure

$\mathcal{B}_k$ separates proof-of-work (votes) from blocks. Leaders selected during the epoch may fail to propose at the end of the epoch. We model such failures by dropping block proposals randomly with constant probability (leader failure rate).

A special property of $\mathcal{B}_k$ is that it can reuse votes for different proposals. Honest nodes reveal at most one new vote with their proposal. Accordingly, a lost proposal wastes at most the work of one vote. Therefore, $\mathcal{B}_{51}$ can recover fast. The results in Figure 11 support this claim. For perspective, the right end of the graph simulates a situation where an attacker can monitor all nodes' network traffic and disconnect leaders at discretion with 50 % success probability. Still, the block interval grows only by about 2.5 %. This effect is similar to the robustness against excessive latencies discussed in Section 4.1.1.

For $\mathcal{B}_1$, voting, leader selection, and proposing happens in a single message. Leader failure is similar to churn and hence has a much stronger effect (see Fig. 10).

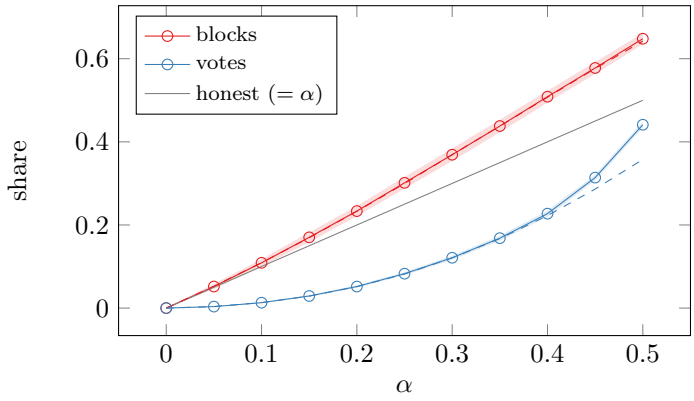


Figure 12: The attacker's share of confirmed blocks and votes as functions of α in the **realistic/exponential** scenario. The attacker uses vote withholding against $\mathcal{B}_{51}$. The gray line shows the expected shares without attack. The dashed lines show results from the Markov chain model. The solid lines show the validation in the network simulator.

4.2 Security

Zhang and Preneel [70] propose to evaluate blockchain protocols with respect to the four security aspects

1. *subversion gain*, to what extent an attacker can rewrite confirmed blocks,
2. *chain quality*, how much of the confirmed blocks are proposed by the attacker,
3. *censorship susceptibility*, how long the attacker can block certain transactions, and
4. *incentive compatibility*, how much reward the attacker can collect by deviating from the protocol.

Our approach is to derive subversion gain from the ε-safety of $\mathcal{A}_k$ and then evaluate chain quality and censorship susceptibility jointly. This is sufficient because both aspects depend on the attacker being selected as a leader. Turning to incentive compatibility, we argue in Section 3.4 why it seems impossible to prove this for realistic utility functions. Zhang and Preneel use a restricted notion in which the attacker utility is the share of rewards assigned by the protocol. We can evaluate their definition of incentive compatibility, along with chain quality and censoring.

4.2.1 Subversion Gain

We provide a consistency analysis for the agreement $\mathcal{A}_k$ in Section 2. The proposed $\mathcal{B}_{51}$ executes $\mathcal{A}_{51}$ for each appended block. The probability that an $\alpha = 1/4$ attacker in a $\Delta = 2''$ synchronous network succeeds in causing inconsistent state updates (e. g., double spend) is $2.2 \cdot 10^{-4}$ (see Tab. 3). The proposed protocol meets this guarantee after one block confirmation, i. e., after about 10 minutes.We assume that applications wait for this confirmation and conclude that subversion gain is not a practical concern for $\mathcal{B}_{51}$.

4.2.2 Chain Quality, Censoring, and Incentives

Chain quality measures the share of confirmed blocks proposed by the attacker. Censoring is possible only if the attacker controls the proposed block payload. Thus, chain quality and censoring reduce to the question of how often an attacker can take leadership.

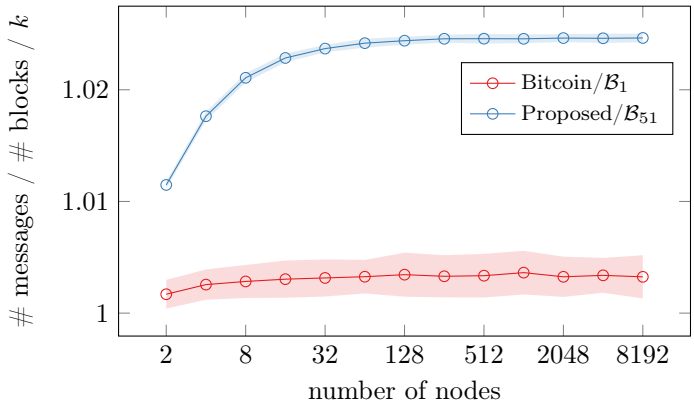


Figure 13: Number of broadcast messages per block divided by k for networks of different size in the **realistic/exponential** scenario.

A common weakness of sequential proof-of-work protocols relates to information withholding. Block withholding, proposed by Eyal and Sirer [25], enables selfish mining against Bitcoin. $\mathcal{B}_k$ is not vulnerable to block withholding because selected leaders who do not propose a block are quickly replaced (see Sect. 4.1.3). The remaining information to be considered in withholding attacks are votes (see Sect. 2.4 for $\mathcal{A}_k$; related [10]). In $\mathcal{B}_k$, the attacker can prolong an epoch by withholding votes until the honest nodes can form a k-quorum themselves. The attacker can use the additional time to mine the smallest vote and be selected as leader.

We first analyze the effectiveness of vote withholding in a single epoch using a Markov chain model (see Appendix A). Then we use the network simulator to confirm the results for executions of the protocol over multiple epochs.

Figure 12 shows the success rate of the attacker in red and his number of confirmed votes in blue. Solid lines originate from the network simulator and dashed lines from the Markov chain model. Both evaluation methods concur in the main result: a withholding attacker can become the leader in about $1.3 \cdot \alpha$ cases (65 % for $\alpha = 50\%$). His advantage in taking leadership comes at the price of fewer confirmed votes. If rewards are proportional to votes, this tells us that vote withholding is disincentivized. For comparison with Nakamoto consensus, block withholding strategies give an $\alpha = 1/3$ attacker an advantage of $1.5 \cdot \alpha$. This factor raises to $2 \cdot \alpha$ for $\alpha = 1/2$ [61]. Moreover, successful selfish miners receive more rewards than without attack. The results indicate that $\mathcal{B}_{51}$ offers higher chain quality, is less susceptible to censorship, and offers fewer incentives to attack than Nakamoto consensus.

4.3 Overhead

Nakamoto consensus requires at least one message broadcast per appended block, namely the block itself, independent of the number of participating nodes. $\mathcal{B}_k$ adds k message broadcasts per block—one for each vote. We evaluate the actual number of sent messages in the network simulator. Figure 13 shows the number of broadcast messages as a function of the number of blocks and k. Observe that $\mathcal{B}_{51}$ plateaus at about $1.025 \cdot k$, i. e., 52 broadcasts per block. This number remains stable as the network scales up.

While the constant factor k may matter for practical networks, it is worth pointing out that vote messages are much smaller than blocks. Under the conservative assumptions of 256 bits each for the block reference and the public key, and 64 bits for the puzzle solution, a vote is as small as 72 B.[6]

The votes also cause a constant storage overhead. $\mathcal{B}_k$ persists the complete quorum of k votes for future verification. Note that the reference r is redundant in all votes and needs to be stored only once. Hence, under the assumptions leading to 72 B message size, the storage overhead of $\mathcal{B}_{51}$ is about 2 kB per block. This is less than 0.2 % of Bitcoin's average block size in April 2022.

4.4 Detecting Network Splits

The assumption of a Δ-synchronous network is unavoidable for proof-of-work protocols: delaying the propagation of a defender's puzzle solution is equivalent to reducing his compute power. With unbounded delays, even a weak attacker can solve sufficiently many puzzles before the defender's solutions propagate [56].

While network splits clearly violate this assumption, we still want to highlight that $\mathcal{B}_k$ allows for faster detection of such events than Nakamoto consensus. In $\mathcal{B}_k$, each vote carries one puzzle solution. The activation delay is exponentially distributed with rate λ (see Sect. 2.1.3). In an intact network, the time between received votes should follow the same distribution. This allows nodes to test the hypothesis of being eclipsed. For $\mathcal{B}_{51}$, a node can distinguish a network split from normal conditions with high confidence after 82 seconds of not receiving a vote (error probability $p = 0.1\,\%$). For comparison, the same hypothesis test would require more than an hour of observation in Bitcoin.

5 Discussion

We discuss our contributions from several perspectives. Section 5.1 compares the security analysis of $\mathcal{A}_k$ to the relevant literature. Section 5.2 positions the family of protocols $\mathcal{B}_k$ in the design space of blockchain protocols. Limitations and directions for future work are discussed in Section 5.3.

5.1 Related Security Analyses

Our security analysis of $\mathcal{A}_k$ is inspired by the literature on Bitcoin security. Table 4 summarizes selected landmark contributions.

The first formal security argument of the so-called "Bitcoin backbone protocol" [30] discretized time in slots. Each slot represents one puzzle trial attempt. Communication is synchronous, i. e., messages are delivered at the end of each slot. Security proofs for consistency and chain quality were given asymptotically in the number of slots. The work formally established the eventual consistency of Nakamoto consensus in synchronous networks.

Follow-up work generalized the main results of [30] for a Δ-synchronous communication model. It allows messages to be delivered in future slots [56]. Further refinements using Markov chain models resulted in tighter, but still asymptotic bounds [45]. Recently, two research groups independently derived optimal bounds [20, 32] for the attacker threshold α. Dembo et al. [20] use continuous time and model proof-of-work as a Poisson process.

[6] Bitcoin shortens public keys to 160 bits and uses solutions of 32 bits. Its blocks are in the order of 1 MB.

Table 4: Comparison of related security analyses.

	Garay et al., 2015 [30]	Pass et al., 2017 [56]	Kiffer et al., 2018 [45]	Gaži et al., 2020 [32]	Dembo et al., 2020 [20]	Li et al., 2021 [50]	this paper
time							
- discrete slots	✓	✓	✓	✓			
- continuous					✓	✓	✓
synchrony	slot	Δ	Δ	Δ	Δ	Δ	Δ
security							
- eventual	✓	✓	✓	✓	✓		
- ε-bounded						✓	✓
Markov chains			✓				✓

All analyses cited above use asymptotic security notions. A recent contribution to AFT '21 breaks with this tradition and provides concrete failure bounds for Nakamoto consensus after waiting for a given confirmation time [50] (comp. Sect. 2.6). Concurrent work improves these bounds [33] and simplifies the analysis [37].

Our analysis of $\mathcal{A}_k$ establishes ε-safety in Δ-synchronous networks. We use Poisson processes to model proof-of-work in continuous time, and Markov chains as an analytic tool.

5.2 Related Protocols

We do not attempt to provide a complete map of the design space for replication protocols since other researchers have specialized on this task [5, 11, 12, 29]. Instead, we compare $\mathcal{B}_k$ to some of its closest relatives along selected dimensions.

Research on agreement and state replication began in the late 1970s, initially only considering benign but unreliable behavior [48, 59]. In the early 1980s, Lamport extended the discussion to adversarial behavior. He coined the term Byzantine Fault Tolerance (BFT) [49], impliying that at most f out of $n = 3f + 1$ identified nodes may deviate from the protocol. Early BFT protocols [e. g., 13, 21] rely on communication between all n nodes for each value agreed upon. This results in quadratic communication complexity and renders the protocols impractical for more than a dozen nodes.

In that light, Bitcoin [52] can be seen as a technical breakthrough [2, 53]. Nodes can join the network without obtaining permission from a central gatekeeper. Active participation, i. e., proposing new state updates, is limited by the nodes' ability to solve proof-of-work puzzles. This yields sub-quadratic communication complexity and allows Bitcoin to scale to thousands of nodes.

However, the sequential proof-of-work scheme proposed with Bitcoin faces a fundamental trade-off between security and throughput. It supports only one state update per block, while the time between blocks must be in the order of seconds to minutes [50]. Bitcoin-NG [24] tries to resolve this

conflict by separating leader selection and transaction processing. The miner of a block becomes responsible for appending multiple consecutive state updates until the next leader emerges with the next mined block. To discipline the leader, Bitcoin-NG relies on incentives.

A number of protocols extend on the idea and try to shift trust from a single to multiple leaders. E. g., Byzcoin [46] selects a committee from the last successful miners, who then run a conventional BFT protocol to agree on the latest state. Similar layered protocols evolved concurrently [18] and afterwards [3, 57, 58]. However, the synchronization between the different consensus layers increases protocol complexity [3, 57, 58] and is a source of concern [18, 46]. Moreover, all layered protocols assume that the attacker cannot corrupt committee members selectively.

$\mathcal{B}_k$ does not layer different consensus mechanisms. It implements state replication directly from proof-of-work and broadcast primitives. As demonstrated in Section 4.1.3, $\mathcal{B}_k$ tolerates selective corruption of committee members.

Other protocols with non-sequential proof-of-work have been proposed. E. g., Phantom [65] replaces the linear blockchain data structure with a directed acyclic graph (DAG) for better scalability and faster first confirmation in latent networks. However, it also increases protocol complexity; in particular, deriving a total order of state updates from the DAG is NP-hard. A related idea is to operate multiple sequential blockchains in parallel [4, 26, 45, 69]. A close relative of $\mathcal{B}_k$ is Bobtail [10]. It uses multiple proof-of-work puzzles per block, like $\mathcal{B}_k$, but binds a preliminary state update to each vote. Votes reference earlier votes, hence Bobtail mixes elements of parallel and sequential proof-of-work. All cited non-sequential protocols lack a principled analysis on when to accept state updates. $\mathcal{B}_k$ is the first non-sequential proof-of-work protocol that comes with concrete bounds for safety.

A separate line of research explores alternatives to the wasteful proof-of-work primitive. Proof-of-stake protocols [e. g., 14, 16, 17] select committee members for participation in an agreement procedure based on the distribution of stake in the system's cryptocurrency. This creates a cyclic dependency which, we think, has not been solved convincingly. Other proof-of-x protocols try to provide useful services like file storage [7] or radio network coverage [38]. However, these protocols seem to rely on heuristics for truthful resource accounting.

In the meantime, research on permissioned protocols caught up. Current BFT-style protocols achieve sub-quadratic communication complexity, enabling deployments with hundreds of nodes [1, 68]. These protocols depend on identities and can scale applications as long as the BFT assumptions hold.

For digital currencies, it might be worth abandoning state machine replication completely and implement digital asset transfer directly from reliable broadcast [36]. Promising proposals exist for the BFT assumptions [6], proof-of-work [64], and proof-of-stake [63]. However, this approach restricts the versatility of the application layer. It cannot support arbitrary smart contract logic.

5.3 Limitations and Future Work

We have presented a permissionless replication protocol that achieves ε-safety in Δ-synchronous networks with computationally bounded attackers. Although our model is widely accepted in the literature [30, 56], it is worth discussing its assumptions.

We assume a fixed puzzle solving rate λ, but in practice agents can add and remove compute power at their discretion. Practical systems try to stabilize λ with a control loop known as difficulty

adjustment algorithm (DAA) [8, 28, 39, 41, 47]. For $\mathcal{B}_k$, the accuracy of a DAA could increase in k as every additional vote provides a data point for the estimation of λ. Turning to the synchronous network assumption, as shown in Section 4.4, the response time to detect network splits decreases for larger k. This relates to the CAP theorem [35], which states that every distributed system has to sacrifice one out of consistency, availability, and partition tolerance. $\mathcal{B}_k$, as specified in Algorithm 2, favors availability over consistency. The trade-off could be changed in favor of consistency by implementing the split detection. Such a variant of the protocol could notify the application layer to withhold commits and trigger out-of-band resolutions.

The perhaps most problematic assumption is that the attacker's share in compute power α is small (see Table 2). Violations, especially $\alpha > 1/2$, are catastrophic, but have been observed in practice [23]. Note that the theory in Section 2.4 does apply for values of $\alpha > 1/3$, but the resulting failure probabilities ε are unattractive. This contrasts with the BFT literature, which requires a hard upper threshold of $n = 3f + 1$ to satisfy an absolute notion of safety.

This leads us to future work. Our evaluation of $\mathcal{B}_k$ is limited to one instance, $\mathcal{B}_{51}$, and uses independent propagation delays on a fully connected graph. We chose the instance for comparability with Bitcoin. Tests of other protocol configurations with more realistic topologies could complete the picture. However, as the literature reports discrepancies between the topology implied at design time and the one observed in practice [19, 51], it is not obvious what topology would be appropriate. A different direction is to explore improvements in the optimistic case by including application-level payloads into $\mathcal{B}_k$'s vote messages. E. g., one could add transactions that do not require consensus [6, 36, 63] or implement a staging of state updates [71]. Finally, as explained in Section 3.4, we refrain from designing an incentive mechanism for $\mathcal{B}_k$. Possible approaches are to search reward-optimizing strategies using Markov Decision Processes (MDPs) [61, 70, 72] or reinforcement learning [40].

6 Conclusion

The proposed family of protocols $\mathcal{A}_k$ proves that unidentified nodes can reach agreement with guaranteed liveness and ε-safety in a Δ-synchronous network using proof-of-work. The family of protocols $\mathcal{B}_k$ shows that parallel proof-of-work enables blockchain protocols with concrete security bounds. With k chosen as described, $\mathcal{B}_k$ enables permissionless state replication that serves certain applications better than existing systems.

It is worth noting that proof-of-work is a wasteful way of establishing agreement. Many alternatives exist if nodes are identifiable. The value of this research is to get better guarantees from protocols when there is no alternative to proof-of-work.

References

[1] Ittai Abraham, T.-H. Hubert Chan, Danny Dolev, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. "Communication Complexity of Byzantine Agreement, Revisited". In: *Symposium on Principles of Distributed Computing*. ACM, 2019, pp. 317–326.

[2] Ittai Abraham and Dahlia Malkhi. "The Blockchain Consensus Layer and BFT". in: *Bulletin of the EATCS* 123 (2017).

[3] Ittai Abraham, Dahlia Malkhi, Kartik Nayak, Ling Ren, and Alexander Spiegelman. *Solida: A Blockchain Protocol Based on Reconfigurable Byzantine Consensus*. 2017. arXiv: `1612.02916 [cs]`.

[4] Vivek Bagaria, Sreeram Kannan, David Tse, Giulia Fanti, and Pramod Viswanath. "Prism: Deconstructing the Blockchain to Approach Physical Limits". In: *Conference on Computer and Communications Security*. ACM, 2019, pp. 585–602.

[5] Shehar Bano, Alberto Sonnino, Mustafa Al-Bassam, Sarah Azouvi, Patrick McCorry, Sarah Meiklejohn, and George Danezis. "SoK: Consensus in the Age of Blockchains". In: *Conference on Advances in Financial Technologies*. ACM, 2019, pp. 183–198.

[6] Mathieu Baudet, George Danezis, and Alberto Sonnino. "FastPay: High-Performance Byzantine Fault Tolerant Settlement". In: *Conference on Advances in Financial Technologies*. ACM, 2020, pp. 163–177.

[7] Juan Benet, David Dalrymple, and Nicola Greco. *Proof of Replication*. Tech. rep. Protocol Labs, 2017.

[8] George Bissias. "Radium: Improving Dynamic PoW Targeting". In: *Workshop on Cryptocurrencies and Blockchain Technology*. Ed. by Joaquin Garcia-Alfaro, Guillermo Navarro-Arribas, and Jordi Herrera-Joancomarti. Vol. 12484. Lecture Notes in Computer Science. Springer, 2020, pp. 374–389.

[9] George Bissias, Rainer Böhme, David Thibodeau, and Brian Levine. "Pricing Security in Proof-of-Work Systems". In: *Workshop on the Economics of Information Security*. 2022.

[10] George Bissias and Brian N. Levine. "Bobtail: Improved Blockchain Security with Low-Variance Mining". In: *Network and Distributed Systems Security Symposium*. Internet Society, 2020.

[11] Joseph Bonneau, Andrew Miller, Jeremy Clark, Arvind Narayanan, Joshua A. Kroll, and Edward W. Felten. "SoK: Research Perspectives and Challenges for Bitcoin and Cryptocurrencies". In: *Symposium on Security and Privacy*. IEEE, 2015, pp. 104–121.

[12] Christian Cachin and Marko Vukolic. "Blockchain Consensus Protocols in the Wild (Keynote Talk)". In: *Symposium on Distributed Computing*. Ed. by Andréa W. Richa. Vol. 91. Leibniz International Proceedings in Informatics. 2017, 1:1–1:16.

[13] Miguel Castro and Barbara Liskov. "Practical Byzantine Fault Tolerance and Proactive Recovery". In: *ACM Transactions on Computer Systems* 20.4 (2002), pp. 398–461.

[14] Jing Chen and Silvio Micali. "Algorand: A Secure and Efficient Distributed Ledger". In: *Theoretical Computer Science*. In Memory of Maurice Nivat, a Founding Father of Theoretical Computer Science - Part I 777 (2019), pp. 155–183.

[15] Mauro Conti, E. Sandeep Kumar, Chhagan Lal, and Sushmita Ruj. "A Survey on Security and Privacy Issues of Bitcoin". In: *IEEE Communications Surveys Tutorials* 20.4 (2018), pp. 3416–3452.

[16] Phil Daian, Rafael Pass, and Elaine Shi. "Snow White: Robustly Reconfigurable Consensus and Applications to Provably Secure Proof of Stake". In: *Financial Cryptography and Data Security*. Ed. by Ian Goldberg and Tyler Moore. Vol. 11598. Lecture Notes in Computer Science. Springer, 2019, pp. 23–41.

[17] Bernardo David, Peter Gazi, Aggelos Kiayias, and Alexander Russell. "Ouroboros Praos: An Adaptively-Secure, Semi-Synchronous Proof-of-Stake Blockchain". In: *Advances in Cryptology – EUROCRYPT 2018*. Ed. by Jesper Buus Nielsen and Vincent Rijmen. Vol. 10821. Lecture Notes in Computer Science. Springer, 2018, pp. 66–98.

[18] Christian Decker, Jochen Seidel, and Roger Wattenhofer. "Bitcoin Meets Strong Consistency". In: *International Conference on Distributed Computing and Networking*. ACM, 2016, pp. 1–10.

[19] Sergi Delgado-Segura, Surya Bakshi, Cristina Pérez-Solà, James Litton, Andrew Pachulski, Andrew Miller, and Bobby Bhattacharjee. "TxProbe: Discovering Bitcoin's Network Topology Using Orphan Transactions". In: *Financial Cryptography and Data Security*. Ed. by Ian Goldberg and Tyler Moore. Vol. 11598. Lecture Notes in Computer Science. Springer, 2019, pp. 550–566.

[20] Amir Dembo, Sreeram Kannan, Ertem Nusret Tas, David Tse, Pramod Viswanath, Xuechao Wang, and Ofer Zeitouni. "Everything Is a Race and Nakamoto Always Wins". In: *Conference on Computer and Communications Security*. ACM, 2020, pp. 859–878.

[21] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. "Consensus in the Presence of Partial Synchrony". In: *Journal of the ACM* 35.2 (1988), pp. 288–323.

[22] Cynthia Dwork and Moni Naor. "Pricing via Processing or Combatting Junk Mail". In: *Advances in Cryptology – CRYPTO '92*. Ed. by Ernest F. Brickell. Vol. 740. Lecture Notes in Computer Science. Springer, 1993, pp. 139–147.

[23] Attah Elikem. *Five Most Prolific 51% Attacks in Crypto: Verge, Ethereum Classic, Bitcoin Gold, Feathercoin, Vertcoin*. `https://cryptoslate.com/prolific-51-attacks-crypto-verge-ethereum-classic-bitcoin-gold-feathercoin-vertcoin/`. 2019.

[24] Ittay Eyal, Adem Efe Gencer, Emin Gün Sirer, and Robbert van Renesse. "Bitcoin-NG: A Scalable Blockchain Protocol". In: *Symposium on Networked Systems Design and Implementation*. USENIX, 2016, pp. 45–59.

[25] Ittay Eyal and Emin Gün Sirer. "Majority Is Not Enough: Bitcoin Mining Is Vulnerable". In: *Financial Cryptography and Data Security*. Ed. by Nicolas Christin and Reihaneh Safavi-Naini. Vol. 8437. Lecture Notes in Computer Science. Springer, 2014, pp. 436–454.

[26] Matthias Fitzi, Peter Gaži, Aggelos Kiayias, and Alexander Russell. "Ledger Combiners for Fast Settlement". In: *Theory of Cryptography*. Ed. by Rafael Pass and Krzysztof Pietrzak. Vol. 12550. Lecture Notes in Computer Science. Springer, 2020, pp. 322–352.

[27] Bryan Ford and Rainer Böhme. *Rationality Is Self-Defeating in Permissionless Systems*. 2019. arXiv: `1910.08820 [cs]`.

[28] Daniel Fullmer and A. Stephen Morse. "Analysis of Difficulty Control in Bitcoin and Proof-of-Work Blockchains". In: *Conference on Decision and Control*. IEEE, 2018, pp. 5988–5992.

[29] Juan Garay and Aggelos Kiayias. "SoK: A Consensus Taxonomy in the Blockchain Era". In: *Topics in Cryptology – CT-RSA 2020*. Ed. by Stanislaw Jarecki. Vol. 12006. Lecture Notes in Computer Science. Springer, 2020, pp. 284–318.

[30] Juan Garay, Aggelos Kiayias, and Nikos Leonardos. "The Bitcoin Backbone Protocol: Analysis and Applications". In: *Advances in Cryptology – EUROCRYPT 2015*. Ed. by Elisabeth Oswald and Marc Fischlin. Vol. 9057. Lecture Notes in Computer Science. Springer, 2015, pp. 281–310.

[31] Hector Garcia-Molina. "Elections in a Distributed Computing System". In: *IEEE Transactions on Computers* C-31.1 (1982), pp. 48–59.

[32] Peter Gaži, Aggelos Kiayias, and Alexander Russell. "Tight Consistency Bounds for Bitcoin". In: *Conference on Computer and Communications Security*. ACM, 2020, pp. 819–838.

[33] Peter Gaži, Ling Ren, and Alexander Russell. *Practical Settlement Bounds for Proof-of-Work Blockchains*. Cryptology ePrint 805. IACR, 2021.

[34] Arthur Gervais, Ghassan O. Karame, Karl Wüst, Vasileios Glykantzis, Hubert Ritzdorf, and Srdjan Capkun. "On the Security and Performance of Proof of Work Blockchains". In: *Conference on Computer and Communications Security*. ACM, 2016, pp. 3–16.

[35] Seth Gilbert and Nancy Lynch. "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services". In: *ACM SIGACT News* 33.2 (2002), pp. 51–59.

[36] Rachid Guerraoui, Petr Kuznetsov, Matteo Monti, Matej Pavlovič, and Dragos-Adrian Seredinschi. "The Consensus Number of a Cryptocurrency". In: *Symposium on Principles of Distributed Computing*. ACM, 2019, pp. 307–316.

[37] Dongning Guo and Ling Ren. "Bitcoin's Latency–Security Analysis Made Simple". In: *Conference on Advances in Financial Technologies*. ACM, 2022.

[38] Amir Haleem, Andrew Allen, Andrew Thompson, Marc Nijdam, and Rahul Garg. *Helium: A Decentralized Wireless Network*. Tech. rep. Helium Systems, 2018, p. 20.

[39] Thomas M. Harding. "Real-Time Block Rate Targeting". In: *Ledger* 5 (2020).

[40] Charlie Hou, Mingxun Zhou, Yan Ji, Phil Daian, Florian Tramèr, Giulia Fanti, and Ari Juels. "SquirRL: Automating Attack Analysis on Blockchain Incentive Mechanisms with Deep Reinforcement Learning". In: *Network and Distributed Systems Security Symposium*. Internet Society, 2021.

[41] Geir Hovland and Jan Kucera. "Nonlinear Feedback Control and Stability Analysis of a Proof-of-Work Blockchain". In: *Modeling, Identification and Control* 38.4 (2017), pp. 157–168.

[42] Ghassan O. Karame, Elli Androulaki, and Srdjan Capkun. "Double-Spending Fast Payments in Bitcoin". In: *Conference on Computer and Communications Security*. ACM, 2012, pp. 906–917.

[43] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. Second. CRC Press, 2014.

[44] Patrik Keller. *Protocol implementation and network simulator*. `https://github.com/pkel/hotpow/tree/ppow`. 2021.

[45] Lucianna Kiffer, Rajmohan Rajaraman, and Abhi Shelat. "A Better Method to Analyze Blockchain Consistency". In: *Conference on Computer and Communications Security*. ACM, 2018, pp. 729–744.

[46] Eleftherios Kokoris Kogias, Philipp Jovanovic, Nicolas Gailly, Ismail Khoffi, Linus Gasser, and Bryan Ford. "Enhancing Bitcoin Security and Performance with Strong Consistency via Collective Signing". In: *Security Symposium*. USENIX, 2016, pp. 279–296.

[47] Daniel Kraft. "Difficulty Control for Blockchain-Based Consensus Systems". In: *Peer-to-Peer Networking and Applications* 9.2 (2016), pp. 397–413.

[48] Leslie Lamport. "Time, Clocks, and the Ordering of Events in a Distributed System". In: *Communications of the ACM* 21.7 (1978), pp. 558–565.

[49] Leslie Lamport, Robert Shostak, and Marshall Pease. "The Byzantine Generals Problem". In: *ACM Transactions on Programming Languages and Systems* 4.3 (1982), pp. 382–401.

[50] Jing Li, Dongning Guo, and Ling Ren. "Close Latency–Security Trade-off for the Nakamoto Consensus". In: *Conference on Advances in Financial Technologies*. ACM, 2021, pp. 100–113.

[51] Sami Ben Mariem, Pedro Casas, Matteo Romiti, Benoit Donnet, Rainer Stütz, and Bernhard Haslhofer. "All That Glitters Is Not Bitcoin – Unveiling the Centralized Nature of the BTC (IP) Network". In: *Network Operations and Management Symposium*. IEEE, 2020.

[52] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. Tech. rep. 2008.

[53] Arvind Narayanan and Jeremy Clark. "Bitcoin's Academic Pedigree". In: *Communications of the ACM* 60.12 (2017), pp. 36–45.

[54] Fernando Nogueira. *Bayesian Optimization: Open source constrained global optimization tool for Python*. `https://github.com/fmfn/BayesianOptimization`. 2021.

[55] Diego Ongaro and John Ousterhout. "In Search of an Understandable Consensus Algorithm". In: *Annual Technical Conference*. USENIX, 2014, pp. 305–319.

[56] Rafael Pass, Lior Seeman, and Abhi Shelat. "Analysis of the Blockchain Protocol in Asynchronous Networks". In: *Advances in Cryptology – EUROCRYPT 2017*. Ed. by Jean-Sébastien Coron and Jesper Buus Nielsen. Vol. 10211. Lecture Notes in Computer Science. Springer, 2017, pp. 643–673.

[57] Rafael Pass and Elaine Shi. "Hybrid Consensus: Efficient Consensus in the Permissionless Model". In: *International Symposium on Distributed Computing*. Ed. by Andréa W. Richa. Vol. 91. Leibniz International Proceedings in Informatics. 2017, 39:1–39:16.

[58] Rafael Pass and Elaine Shi. "Thunderella: Blockchains with Optimistic Instant Confirmation". In: *Advances in Cryptology – EUROCRYPT 2018*. Ed. by Jesper Buus Nielsen and Vincent Rijmen. Vol. 10821. Lecture Notes in Computer Science. Springer, 2018, pp. 3–33.

[59] Marshall Pease, Robert Shostak, and Leslie Lamport. "Reaching Agreement in the Presence of Faults". In: *Journal of the ACM* 27.2 (1980), pp. 228–234.

[60] Ling Ren. *Analysis of Nakamoto Consensus*. Cryptology ePrint 943. IACR, 2019.

[61] Ayelet Sapirshtein, Yonatan Sompolinsky, and Aviv Zohar. "Optimal Selfish Mining Strategies in Bitcoin". In: *Financial Cryptography and Data Security*. Ed. by Jens Grossklags and Bart Preneel. Vol. 9603. Lecture Notes in Computer Science. Springer, 2016, pp. 515–532.

[62] Fred B. Schneider. "Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial". In: *ACM Computing Surveys* 22.4 (1990), pp. 299–319.

[63] Jakub Sliwinski and Roger Wattenhofer. "Asynchronous Proof-of-Stake". In: *Stabilization, Safety, and Security of Distributed Systems*. Ed. by Colette Johnen, Elad Michael Schiller, and Stefan Schmid. Vol. 13046. Lecture Notes in Computer Science. Springer, 2021, pp. 194–208.

[64] Yonatan Sompolinsky, Yoad Lewenberg, and Aviv Zohar. *SPECTRE: A Fast and Scalable Cryptocurrency Protocol*. Cryptology ePrint 1159. IACR, 2016.

[65] Yonatan Sompolinsky, Shai Wyborski, and Aviv Zohar. "Phantom Ghostdag: A Scalable Generalization of Nakamoto Consensus". In: *Conference on Advances in Financial Technologies*. ACM, 2021, pp. 57–70.

[66] Yonatan Sompolinsky and Aviv Zohar. "Secure High-Rate Transaction Processing in Bitcoin". In: *Financial Cryptography and Data Security*. Ed. by Rainer Böhme and Tatsuaki Okamoto. Vol. 8975. Lecture Notes in Computer Science. Springer, 2015, pp. 507–527.

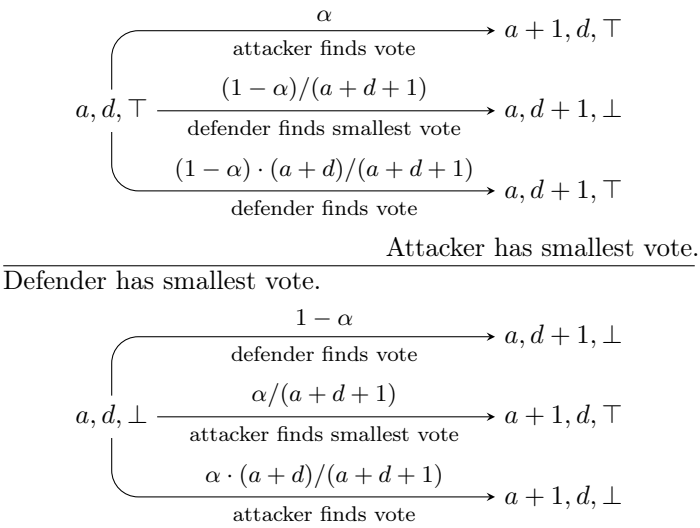


Figure 14: Markov chain of vote withholding to gain leadership in an epoch of $\mathcal{B}_k$. Smallest vote implies leadership.

[67] Volker Strassen. "Gaussian Elimination Is Not Optimal". In: *Numerische Mathematik* 13.4 (1969), pp. 354–356.

[68] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. "HotStuff: BFT Consensus with Linearity and Responsiveness". In: *Symposium on Principles of Distributed Computing*. ACM, 2019, pp. 347–356.

[69] H. Yu, I. Nikolic, R. Hou, and P. Saxena. "OHIE: Blockchain Scaling Made Simple". In: *Symposium on Security and Privacy*. IEEE, 2020, pp. 112–127.

[70] Ren Zhang and Bart Preneel. "Lay down the Common Metrics: Evaluating Proof-of-Work Consensus Protocols' Security". In: *Symposium on Security and Privacy*. IEEE, 2019, pp. 1190–1207.

[71] Ren Zhang, Dingwei Zhang, Quake Wang, Shichen Wu, Jan Xie, and Bart Preneel. "NC-max: Breaking the Security-Performance Tradeoff in Nakamoto Consensus". In: *Network and Distributed Systems Security Symposium*. Internet Society, 2022.

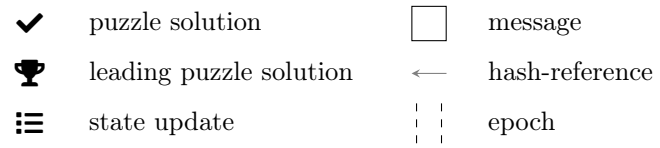
[72] Roi Bar Zur, Ittay Eyal, and Aviv Tamar. "Efficient MDP Analysis for Selfish-Mining in Blockchains". In: *Conference on Advances in Financial Technologies*. ACM, 2020, pp. 113–131.

A Markov Chain Model for Chain Quality and Censoring

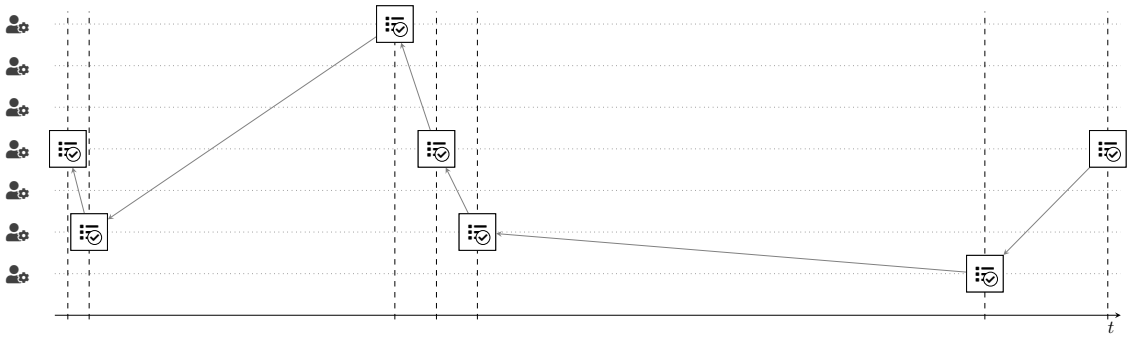
We describe the Markov chain model used in Section 4.2.2. Let the triple (a, d, l) be the current Markov state, where $a \in \mathbb{N}$ denotes the number of withheld attacker votes, $d \in \mathbb{N}$ denotes the number of votes found by the defender, and $l \in \{\bot, \top\}$ is true if the attacker currently holds the smallest vote. The initial state is $(1, 0, \top)$ with probability α and $(0, 1, \bot)$ otherwise.

Figure 14 depicts the probabilistic state transitions. As in Section 2.4, transitions occur at the ticks of the stochastic clock. With probability α, the attacker finds a new vote. The probability that it is the smallest (leading) vote is $1/(a + d + 1)$. This expression follows from $a + d$ old votes cutting the domain into $a + d + 1$ bins. As the hash function's outputs are indistinguishable from a uniform distribution, the expected bin size is $1/(a + d + 1)$. To simplify the figure, we do not show the two terminal states SUCCESS and FAIL. The former is reached when the attacker proposes a valid block $(l \wedge a + d \geq k)$. Conversely, if $\neg l \wedge d \geq k$, the defenders propose a block. In all other cases, the epoch continues.

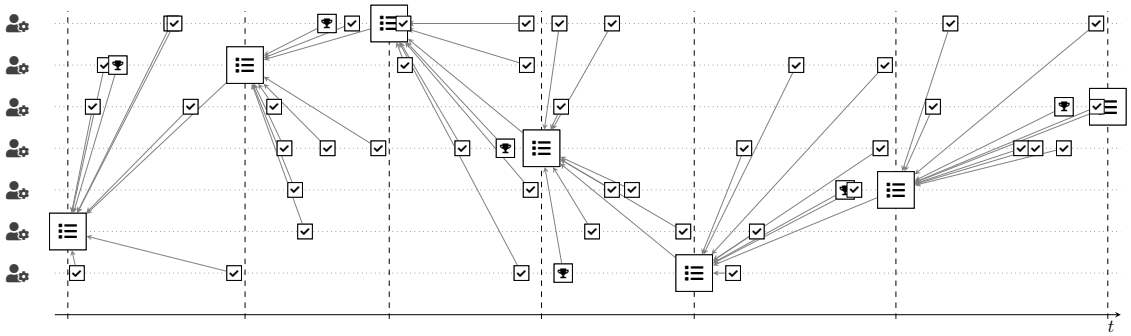
For $k = 51$, the resulting Markov chain has 5204 states. We evaluate it with Monte Carlo simulation for 1 000 000 epochs, $k = 51$ and α in the range $[0, 1/2]$. To validate these results in the context of the protocol and network latency, we implement the same attack in the network simulator and collect data from 64 independent executions by 1024 nodes up to block height 4096. In both cases, we measure chain quality and censorship susceptibility by counting terminations in the state SUCCESS. In addition, we analyze incentive compatibility by counting attacker votes.



(a) Legend.



(b) In Bitcoin, puzzles are solved sequentially. Puzzle solving times are exponentially distributed, implying irregular block intervals.



(c) In $\mathcal{B}_k$, multiple smaller puzzles are solved in parallel for each block. One of the $k = 8$ votes is chosen as leader and the corresponding miner proposes the next block (see Sec. 3.2.3). The embedded agreement protocol $\mathcal{A}_k$ guarantees ε-safety for the previous block (see Sec. 2). Observe that $\mathcal{B}_k$ has more regular block intervals and enables more frequent rewards for miners.

Figure 15: Simulated executions of Bitcoin and $\mathcal{B}_8$ on $n = 7$ nodes (y-axis) over time (x-axis).

Tailstorm:
A Secure and Fair Blockchain for Cash Transactions

Patrik Keller Ben Glickenhaus George Bissias Gregory Griffith

Abstract

Proof-of-work (PoW) cryptocurrencies rely on a balance of security and fairness in order to maintain a sustainable ecosystem of miners and users. Users demand fast and consistent transaction confirmation, and in exchange drive the adoption and valuation of the cryptocurrency. Miners provide the confirmations, however, they primarily seek rewards. In unfair systems, miners can amplify their rewards by consolidating mining power. Centralization however, undermines the security guarantees of the system and might discourage users.

In this paper we present Tailstorm, a cryptocurrency that strikes this balance. Tailstorm merges multiple recent protocol improvements addressing security, confirmation latency, and throughput with a novel incentive mechanism improving fairness. We implement a parallel proof-of-work consensus mechanism with k PoWs per block to obtain state-of-the-art consistency guarantees [29]. Inspired by Bobtail [9] and Storm [4], we structure the individual PoWs in a tree which, by including a list of transactions with each PoW, reduces confirmation latency and improves throughput. Our proposed incentive mechanism discounts rewards based on the depth of this tree. Thereby, it effectively punishes information withholding, the core attack strategy used to reap an unfair share of rewards.

We back our claims with a comprehensive analysis. We present a generic system model which allows us to specify Bitcoin, $\mathcal{B}_k$ [29], and Tailstorm from a joint set of assumptions. We provide an analytical bound for the fairness of Tailstorm and Bitcoin in *honest* networks and we confirm the results through simulation. We evaluate the effectiveness of *dishonest* behaviour through reinforcement learning. Our attack search reproduces known optimal strategies against Bitcoin, uncovers new ones against $\mathcal{B}_k$, and confirms that Tailstorm's reward discounting makes it more resilient to incentive layer attacks. Our results are reproducible with the material provided online [30].

Lastly, we have implemented a prototype of the Tailstorm cryptocurrency as a fork of Bitcoin Cash. The client software is ready for testnet deployment and we also publish its source online [23].

Version Notes

This paper was published in the proceedings of the *5th Conference on Advances in Financial Technologies, AFT 2023, in Princeton, NJ.* I here reproduce the published version, with minimal formatting adjustments applied to fit the body of this work. If possible, I recommend reading the original, open-access document instead: `https://doi.org/10.4230/LIPIcs.AFT.2023.6`

An extended version, with additional protocol specifications and a treatment of selfish mining on short time horizons, is available on arXiv: `https://arxiv.org/abs/2306.12206`





1 Introduction

Proof-of-work (PoW) cryptocurrencies can be thought of as stacked systems comprising four layers. The PoW layer moderates access of weakly identified parties using a mining puzzle: proposing a new block requires finding a small hash. The consensus layer allows all participants to agree on a specific block ordering. The application layer maintains a distributed ledger by writing cryptocurrency transactions into the blocks. Finally, the incentive layer motivates participation in the PoW layer by minting new cryptocurrency for successful miners.

The circular dependencies between the layers result in interdependent failures. Consensus faults are inevitable when individual miners gain too much control [19]. Unreliable consensus enables double spending and erodes confidence in the cryptocurrency. Devaluation of the currency renders the mining rewards worthless as well. Lastly, misaligned incentives encourage centralization among the miners, eventually allowing the strongest one to break consensus.

In this paper, we introduce and analyze Tailstorm, a new cryptocurrency that strengthens two layers of the stack. We employ an innovative incentive mechanism as well as a state-of-the-art consensus mechanism, while retaining the PoW and application layers of Bitcoin. We draw inspiration from the organization of delta blocks in the Storm protocol [4] as well as the use of partial PoW in the Bobtail [9] and $\mathcal{B}_k$ protocols [29].

Bitcoin's consensus [39] uses a *sequential* PoW mechanism where each block references a single parent block. The blocks form a tree and the participants mine new blocks that extend the longest branch according to the *longest chain rule*. Blocks off the longest branch are discarded. Evidently, attackers who possess more than 50 % of the hash rate pose a threat to the system: they can execute double-spend attacks by mining their own branch until it eventually becomes the longest. But even less mining power might suffice since honest participants also discard the blocks of other honest miners if they are not on the same branch. This happens naturally in realistic networks due to propagation delays. If an attacker can induce and exploit communication delays, all discarded blocks may benefit the attacker; effectively increasing their strength [24].

In contrast, Tailstorm implements a *parallel* PoW consensus mechanism that largely avoids discarding blocks. For this, we closely follow the approach taken by Keller and Böhme at AFT '22 [29]. Their protocol, $\mathcal{B}_k$, confirms each block with k *votes*. $\mathcal{B}_k$ blocks do not require a PoW, only votes do. Notably, votes confirming the same block can be mined in parallel because they do not depend on each other. Discarding only occurs when there are more than k votes for the same block. Even then, individual discarded votes only account for $1/k$ of a block's PoW. As Keller and Böhme [29] argue, this makes consensus more robust against consensus layer attacks.

But it is futile to analyze the consensus without considering incentives. Ideally, rewards are distributed fairly, which means that a miner's expected reward is proportional to its hash rate. Arnosti and Weinberg [3] show that even small inequalities in reward allocation encourage substantial centralization of hash rate which ultimately poses a threat to consensus and the cryptocurrency itself [19].

Unfairness arises from natural network delays and dishonest behavior. Both of these factors affect Bitcoin. In latent networks, two blocks mined around the same time may refer to the same parent, even if both miners follow the longest chain rule. One of the blocks will be discarded, the other rewarded. The stronger miner has more hash rate to support their own block, hence the weaker miner is worse off. Network-level attackers can additionally exploit latency to manipulate impartial

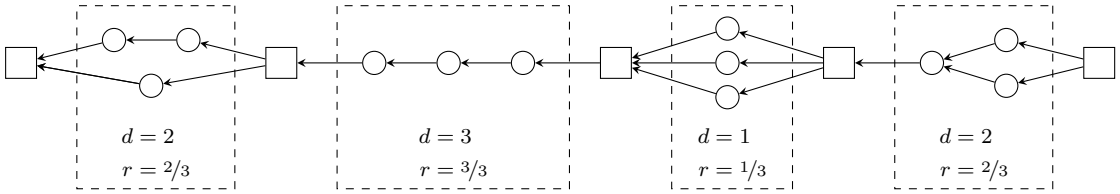


Figure 1: Example of a Tailstorm blockchain with $k = 3$ subblocks per summary. Squares are summary blocks, circles are subblocks, arrows indicate hash-references. Dashed rectangles mark subblock trees with depth d and discounted subblock reward r. Reduced depth implies lower rewards.

participants in their favor. But even without delays, Bitcoin miners with more than one third of the hash rate can reap an unfair share of rewards by temporally withholding blocks instead of acting honestly [16, 21, 40, 47]. As we will demonstrate, $\mathcal{B}_k$ suffers from the same problem, partly due to its leader election mechanism.

Tailstorm addresses the unfairness problem through an innovative reward scheme that punishes withholding. The Tailstorm blockchain consists of *subblocks* and *summary* blocks. Similar to $\mathcal{B}_k$, summaries do not require a PoW, but subblocks do. Assembling a new summary requires k subblocks that confirm the same parent summary. To preserve the security properties of $\mathcal{B}_k$, subblocks confirming the same summary are conflict-free, and hence can be mined in parallel. Taking inspiration from Bobtail [9], subblocks *optionally* refer to another subblock instead of a summary, and hence form a tree. With Tailstorm, we propose to discount rewards based on the depth of this tree, as depicted in Figure 1. Mining subblocks in private causes branching of the tree, reduces its depth, and ultimately leads to lower rewards.

We support our claims with a comprehensive analysis and make the following contributions.

1. We formulate a generic system model for PoW cryptocurrencies. The models abstracts PoW and communication, defines a joint set of assumptions, and enables valid comparisons between different consensus protocols and incentive mechanisms.
2. We specify Bitcoin, $\mathcal{B}_k$, and Tailstorm in the joint model. To isolate the effect of Tailstorm's discount reward scheme and $\mathcal{B}_k$'s leader election mechanism, we additionally specify a hybrid protocol, TS/const, modelling Tailstorm without discounting and $\mathcal{B}_k$ without leader election.
3. We provide an upper bound for the orphan rate of Tailstorm in honest networks. Compared to Bitcoin [43], Tailstorm creates less orphans and hence presents less opportunity for unfairness.
4. We implement the system model as a simulator and show that Tailstorm is more fair than Bitcoin in honest but realistic networks with propagation delays. We confirm Bitcoin's inherent tradeoff: while short block intervals are desirable for fast confirmations and a less volatile stream of rewards, they also bias rewards in favour of strong miners. In Tailstorm, these concerns are largely separated by configuring long *summary* block intervals for fairness and short *subblock* intervals for fast confirmations and frequent rewards.
5. We evaluate multiple hard-coded attack strategies against the specified protocols, finding that attacks which are profitable against $\mathcal{B}_k$ are less profitable against Tailstorm, with the TS/const protocol lying in between.
6. We follow Hou et al. [26] and search optimal attack strategies using reinforcement learning. Our search reproduces optimal strategies against Bitcoin [21, 47] and generally matches or outperforms the hard-coded strategies. The regularity of our results indicates that we indeed

found near-optimal strategies against all protocols and enables the conclusion that Tailstorm is less susceptible to incentive layer attacks than the other protocols.

7. We describe the Tailstorm application layer, which implements a cryptocurrency on top of the proposed consensus protocol. It preserves Bitcoin's transaction logic while enabling faster confirmations. Transactions are stored in subblocks in much the same way that the Storm protocol [4] stores transactions in delta blocks.
8. Lastly, we implement a prototype of Tailstorm which is ready for testnet deployments and make the code available online [23].

We structure the paper in order of our contributions. Section 2 defines the system model and Section 3 presents the specification of Tailstorm. In Section 4, we evaluate the protocols in an honest network with propagation delays. In Section 5, we evaluate hard-coded attack strategies, and in Section 6 we conduct the search for optimal policies with reinforcement learning. Section 7 presents the Tailstorm cryptocurrency and our prototype implementation. In Section 8, we discuss related work, limitations and future work. Section 9 concludes.

2 System Model

Recall the layered view on PoW cryptocurrencies presented in the introduction: The PoW layer moderates access using a mining puzzle, the consensus layer establishes a specific block ordering, the application layer writes cryptocurrency transactions into the blocks, and the incentive layer mints new cryptocurrency for successful miners. We now present a system model that abstracts PoW and communication to enable concise specification of the consensus layer. Application and incentives are considered in later sections.

In practice, PoW consensus protocols are executed as distributed systems, where independent nodes communicate over a P2P network. Messages exchanged between the nodes may be subject to natural or potentially malicious delays. To facilitate specification, we abstract this distributed system and model it algorithmically. We define a virtual environment that emulates distributed protocol execution in a single thread of computation. Within the virtual environment, nodes are represented as numbers, and blocks are represented as vertices in a directed acyclic graph (DAG). Mining is simulated as a loop with random delays, while communication is modeled by restricting the visibility of blocks to a subset of the nodes. The behaviour of nodes is defined by functions, which can be customized to model different protocols.

We define the virtual environment in Algorithm 1. The environment maintains a DAG where each *vertex* represents one block. Each block b has an associated list of parent blocks that constitute the outgoing *edges* in the DAG. We denote this list $\texttt{parents}(b)$. In practice, edges arise from hash references pointing to other blocks in the blockchain. We say that block b is a *descendant* of block b', if b' is either a parent of b, or is connected transitively by the `parents` relationship. In this case, we say b' is an *ancestor* of b. Each block has *properties* which are assigned as the protocol unfolds. For example, the virtual environment uses the Boolean property $\texttt{pow}(b)$ to track whether block b has a PoW or not.

We label the participating nodes with integers ranging from 1 to n. For each node, the virtual environment maintains a local view of the DAG and a preferred tip of the chain. In Lines 10 and 21, we restrict the local view of node i to blocks where $\texttt{visibility}(b, i)$ was set to true. Initially, local

Algorithm 1: Virtual Environment

```
1  t ← Root();                                          // Root: protocol's genesis block template
2  b ← block obtained from appending template t to the block DAG;   // reification
3  for i = 1, ..., n do                                // n: number of nodes
4      tip(i) ← b;                                      // tip(i): state of node i
5      visibility(b, i) ← true;                         // visibility(b, i): whether node i sees block b

6  while true do                                        // concurrent PoW loop
7      sample delay d_pow ~ Expon(rate λ);              // λ: PoW rate
8      sample node i ~ Discrete(1, ..., n with weights κ_1, ..., κ_n);   // κ_1, ... κ_n: relative hash rates
9      wait for d_pow seconds, handling other tasks concurrently;
10     with local view of node i do                     // partial and immutable view on block DAG
11         tmpl ← Extend(tip(i));                       // Extend: protocol's mining rule
12     b ← block obtained from appending template tmpl to the block DAG;   // reification
13     pow(b') ← true;                                  // mark block as mined
14     visibility(b, ·) ← false;                        // block not yet propagated
15     if Validate(b) then Deliver(b, i);               // Validate: protocol's chain structure

16 function Deliver(block b, node i) is                 // Turn block visible for node ...
17     for p ∈ parents(b) do                            // ... in topological order ...
18         wait until visibility(p, i) is true;
19     if not visibility(b, i) then                     // ... and at most once.
20         visibility(b, i) ← true;
21         with local view of node i do                 // partial and immutable view on block DAG
22             (tip(i), share, append) ← Update(tip(i), b);   // Update: protocol's state update rule
23         for b' ∈ share and j = 1, ..., n, j ≠ i do   // handle block broadcast requests
24             pick message delay d_net according to network assumptions;
25             Deliver(b', i) in d_net seconds;
26         for tmpl ∈ append do                         // handle requests to append blocks without PoW
27             b' ← block obtained from appending template tmpl to the block DAG;   // reification
28             pow(b') ← false;                         // block has no PoW
29             visibility(b', ·) ← false;
30             if Validate(b') then Deliver(b', i);     // Validate: protocol's chain structure
```

views are empty and new blocks are not visible to any node. We denote as $\texttt{tip}(i)$ the preferred *tip* of node i, the block to which a new block from i will point. We describe the behaviour of nodes as pure functions. These functions are called by the environment to obtain instructions from the node which the environment then interprets according to our assumptions. This makes all modifications of the DAG and all communication explicit in Algorithm 1. In particular, nodes do not directly append blocks to the DAG; they return block templates, which the environment then reifies by appending a new block to the DAG.

A protocol is fully specified through four functions: `Root` and `Validate` define the structure of the blockchain, while `Update` and `Extend` define the behavior of honest nodes.

The `Root` function takes no argument and returns a single block, which we call *genesis*. Initially, the genesis is the only block in the DAG. `Validate` takes a block as argument and returns `true` if the block is valid and `false` otherwise. E. g., Bitcoin's `Validate` checks the `pow` property and that there is exactly one parent. The environment enforces block validity during the reification of blocks, while deployed protocols would reject invalid blocks in the communication layer. The genesis is not subject to the validity rule.

The `Update` function specifies how nodes react to newly visible blocks, after they are mined locally with PoW, appended locally without PoW, or received from the network. The function takes

two arguments: the node's currently preferred tip and the new block. The function returns the new preferred tip, a list of blocks the node intends to *share* with other nodes, and a list of block templates it wants to *append* to the chain *without* PoW. On the other hand, the function `Extend` defines how nodes grow the chain *with* PoW. It takes a single argument, a node's currently preferred tip, and returns a template for the block that the node intends to mine.

We follow related work [15, 29, 37, 43, 51] and model the mining process in continuous time. The virtual environment generates independent mining delays from the exponential distribution $\texttt{Expon}(\lambda)$, with rate λ measured in expected number of proofs-of-work per second. Accordingly, the expected value of the distribution, $1/\lambda$, is called the *mining interval* and is measured in seconds. After each mining delay, the environment randomly selects a successful miner, obtains a block template from `Extend`, and reifies the block by appending it to the DAG. We support arbitrary hash rate distributions among the nodes by setting the weights $\kappa_1, \ldots, \kappa_n$ in Line 8 accordingly.

The `Deliver` function captures the process of making blocks visible to nodes. The specified protocols have in common that block validation requires knowledge of all referenced blocks. We avoid a lot of boilerplate code in the specification, by ensuring that parent blocks are delivered before their children. Upon delivery, the virtual environment first invokes the `Update` function to obtain the node's new preferred tip, a list of blocks the node wants to share, and a list of block templates the node intends to append without PoW. It then handles the node's requests to share and append. Communication is modelled through delayed delivery, while appends happen immediately.

3 The Tailstorm Protocol

This section specifies the Tailstorm consensus protocol and reward mechanism using the algorithmic model described in Section 2. The specification serves as the basis for our theoretical analyses, network simulations, and attack search in subsequent sections. We first describe Tailstorm's chain structure in Section 3.1. We then specify the behaviour of honest nodes in Section 3.2. As a point of reference, we also specify Bitcoin and $\mathcal{B}_k$ protocols in the extended version of the paper. In this section, we assume that the application layer implements a cryptocurrency which we can use to pay rewards. We defer the description of Tailstorm's application layer to Section 7. Throughout this section, we focus on honest miners who follow the protocol as intended. Later sections will consider dishonest behaviour.

3.1 Chain Structure

Algorithm 2 defines Tailstorm's chain structure. Each block is either a summary or a subblock. Subblocks must have PoW and they must have exactly one parent. Summaries do not require PoW and reference k subblocks, each confirming the same ancestor summary.

Each block b has two integer properties, $\texttt{height}(b)$ and $\texttt{depth}(b)$. The genesis has height and depth zero. Subblocks inherit the height of their parent and they increment the depth by one. Summaries increment the height and reset the depth to zero. Figure 1 in Section 1 illustrates a valid Tailstorm chain for $k = 3$. Note that the subblocks confirming the same summary form a tree and that the `depth` property tracks the depth of this tree. The `height` property on the other hand counts the number of trees that have been summarized.

Algorithm 2: Tailstorm: Chain Structure

```
1  function Root() is
2      return block template b with summary(b) = true and height(b) = depth(b) = 0;
3  function Validate(block b) is
4      if summary(b) then
5          p ← last summary before b;
6          S ← subblocks between b and p;
7          return (|S| = k) ∧ (depth(b) = 0) ∧ (height(b) = height(p) + 1);
8      else                                                          // b is subblock
9          p ← first parent of b;
10         return pow(b) ∧ (|parents(b)| = 1) ∧ (depth(b) = depth(p) + 1) ∧ (height(b) = height(p));
```

Algorithm 3: Tailstorm: Node

```
1  function Preference(summary s, summary b) is
2      hs, hb ← height(s), height(b);
3      ns, nb ← size of subblock tree confirming s and b;
4      rs, rb ← reward obtained from (s, b);
5      return (hb > hs) ∨ (hb = hs ∧ nb > ns) ∨ (hb = hs ∧ nb = ns ∧ rb > rs);
6  function Extend(tip t) is                          // t: currently preferred block
7      if t has children then p ← maximum-depth subblock after t else p ← t;
8      return subblock template b with
9          parents(b) = [p], height(b) = height(p), and depth(b) = depth(p) + 1;
10 function Update(tip t, block b) is                 // t: currently preferred block // b: new block
11     pref, share, append ← t, [b], [];              // new preferred tip, blocks to broadcast, templates to append
12     if summary(b) then
13         if Preference(pref, b) then pref ← b;
14     else                                                          // b is subblock
15         p ← last summary before b;
16         if Preference(pref, p) then pref ← p;
17         S ← all subblocks in subblock tree confirming p;
18         if |S| ≥ k then
19             L ← result of Algorithm 4: Subblock Selection;
20             create summary template s with parents(s) = L;
21             append ← [s];
22     return pref, share, append;
```

To incentivize participation, Tailstorm allocates rewards to the miners of subblocks. The reward size is proportional to the depth of the subblock tree: let b be a summary block, and S be the set of subblocks in the corresponding subblock tree. Then all subblocks in S are allocated the same reward

$$\texttt{discount}(b) = \frac{c}{k} \cdot \max_{x \in S}(\texttt{depth}(x)) \,, \tag{1}$$

where c represents a tunable upper limit on the subblock reward. In Figure 1 we show the rewards for $c = 1$. Note that the reward scheme punishes non-linearities in the blockchain, and this punishment affects all included subblocks equally.

3.2 Honest Nodes

Algorithm 3 specifies the behaviour of honest nodes. The algorithm revolves around a preference order (ln. 1-5) that ranks summaries first by height, then by number of confirming subblocks, and finally by potential personal reward for the individual node. Nodes set the highest ranked summary

Algorithm 4: Tailstorm: Subblock Selection

```
1  R ← ∅;                                                  // selected subblocks
2  while |R| < k do                                        // select one subblock per iteration
3      C ← S \ R;                                          // S: candidate subblocks
4      for x ∈ C do
5          B_x ← x and all ancestors of x in S;
6          B'_x ← B_x \ R;                                 // newly referenced blocks
7          r_x ← number of node's own subblocks in B'_x;   // reward
8      C ← {x ∈ C : |R| + |B'_x| ≤ k};                     // enforce tree size k
9      y ← arg max_{x∈C} r_x;                              // select candidate with highest reward
10     R ← R ∪ B'_y;                                       // track referenced subblocks
11 return leaves in subblock tree R;                       // invariant: |R| = k
```

as their preferred summary (ln. 13+16) and they mine subblocks (ln. 6-9) that confirm their preferred summary. To maximize the depth of the subblock tree, nodes append their subblocks to the longest existing branch.

Whenever nodes learn about a new block (ln. 10-22), they share it with the other nodes and they update their preference. As soon as there are k subblocks (ln. 18) confirming the preferred summary, nodes assemble the next summary. When there are more than k subblock candidates for the next summary, nodes choose the subblocks to maximize their own rewards. We present a greedy algorithm for subblock selection in Algorithm 4. Note that nodes may choose different subblocks and thereby create conflicting summaries. Such conflicts create temporary forks of the blockchain which are resolved quickly according to the preference order described above. We analyze the implications on fairness in Section 4.

3.3 Difficulty Adjustment

A major goal for most blockchains, including Tailstorm, is for the blockchain itself to grow at a constant rate so as to maintain constant transactional throughput. However, in any deployed blockchain, the puzzle solving rate λ changes over time because nodes may come and go or may add or remove mining hardware. The changes in solving rate lead to changes of the growth rate. Adjusting for the fluctuations requires feedback from the consensus to the PoW layer. Typically, blockchains adjust the puzzle solving *difficulty* depending on the observed chain growth using a dynamic *difficulty adjustment algorithm* (DAA).

There exists a rich body of prior work concerning DAA design and analysis [8, 18, 25, 27, 35], but a deep investigation of ideal DAAs for the Tailstorm protocol is beyond the scope this paper. We note, however, that existing DAAs for Bitcoin can be adapted to Tailstorm by counting the number of subblocks where Bitcoin DAAs use the length of the blockchain. For any Tailstorm block b (summary or subblock), we define

$$\texttt{progress}(b) = k \cdot \texttt{height}(b) + \texttt{depth}(b)\,, \tag{2}$$

which counts the number of PoWs included in the chain. Any Tailstorm DAA should adjust the puzzle solving difficulty such that `progress` grows at a constant rate.

3.4 Protocol Variant With Constant Rewards

Tailstorm discounts rewards proportional to the depth of the subblock tree. We see the discounting mechanism as a core contribution of this paper. To isolate the effect of discounting, we introduce a protocol variant without discounting, which we call TS/const. While Tailstorm pays out *at most* c units of reward per subblock, the TS/const protocol pays out *exactly* c units of reward per subblock. In all other aspects, TS/const is identical to Tailstorm.

Note, that the TS/const protocol does not use the subblock tree structure, neither for consensus nor for incentives. In that regard, TS/const resembles the parallel PoW protocol $\mathcal{B}_k$ [29] where all subblocks refer to a summary, never another subblock. The only difference with $\mathcal{B}_k$ is that TS/const does not implement leader election: while $\mathcal{B}_k$ restricts the creation of the next summary to the miner of the subblock with the smallest hash, any TS/const node may (re-)create valid summaries locally.

4 Fairness Under Protocol Compliance

A fair PoW protocol rewards miners in proportion to the amount of work they do. Disproportionate or inconsistent allocation discriminates against weak miners and encourages the formation of pools and centralization. In this section, we explore the causes of unfairness in Tailstorm under the assumption that all miners are honest. We measure work in number of hashes evaluated. The system is fair if rewards are proportional to the miners' hash rates.

The root cause of unfairness is the inherent asymmetry in reward loss when multiple PoW solutions are discovered in short order. In Tailstorm, if more than k subblocks are produced, all having the same height, then all but k of them will be discarded. The discarded blocks are commonly called *orphans* and receive no rewards. Typically, miners do not orphan their own blocks. Since miners with a high hash rate are more often able to choose which blocks will be orphaned, miners with a relatively low hash rate lose a disproportionate amount of rewards. In Bitcoin, this effect is amplified because orphaning can occur for each block (set $k = 1$ in the above argument).

4.1 Analytical Orphan Rate Analysis

In this section, we develop an analytical model for the orphan rate and use it to compare the fairness of Bitcoin and Tailstorm. Let B be a random variable denoting the number of subblocks orphaned during the production of a summary block in Tailstorm or the number of orphans per block in Bitcoin. The orphan rate is given by $\rho = E[B]/k$, which represents the expected number of PoW solutions orphaned for every PoW solution confirmed. In the remainder of this section, we will derive a bound on ρ.

Following Rizun's orphan rate analysis for Bitcoin [43], we model block propagation delays as

$$\tau(\tau_0, z, Q) = \tau_0 + zQ, \tag{3}$$

where τ_0 represents network latency (seconds), z represents bandwidth (bytes per second), and Q represents block size (bytes). In order to adapt this expression for Tailstorm, we assume that the transactions included in a summary block are spread evenly across the k subblocks, so the subblock size is Q/k. The expected summary block interval is T and the subblock mining rate is $\lambda = k/T$.

	Bitcoin	Tailstorm		
block interval	$k = 1$	$k = 5$	$k = 10$	$k = 15$
$T =$ 75 seconds	10.08 %	7.35 %	7.01 %	6.89 %
$T = 150$ seconds	5.04 %	3.67 %	3.50 %	3.45 %
$T = 300$ seconds	2.52 %	1.84 %	1.75 %	1.72 %
$T = 600$ seconds	1.26 %	0.92 %	0.88 %	0.86 %

Table 1: Upper bounds on orphan rate obtained from Theorem 1 assuming various combinations of k and expected block (Bitcoin) or summary block (Tailstorm) intervals T.

Theorem 1. *For network parameters τ_0, z, and Q, summary block interval T, and subblock count k, Tailstorm's expected orphan rate ρ is bounded from above by:*

$$\rho(\tau_0, z, Q, T) \leq \frac{\tau_0}{T} + \frac{zQ}{kT}.$$

Proof. Assume there are $k + X$ subblocks, all pointing to the same summary block. Then, k of these subblocks will be included in the next summary and X will be orphaned. Recall from Algorithm 3 that honest miners proceed to the next summary as soon as they learn of k subblocks. So, if X subblocks are orphaned, they must have been mined before the kth subblock has propagated to all nodes. Additionally, subblocks k through $k + X$ must originate from different miners, since no miner orphans its own subblock.

Let Y_t be a random variable representing the number of subblocks generated during an arbitrary time interval $t > 0$ under the assumption, introduced in Section 2, that mining intervals are exponentially distributed with rate $\lambda = k/T$. A standard result from the theory of Poisson processes shows that $Y_T \sim \texttt{Poisson}(k)$ [46, ch. 5]. The same theory shows that $Y_t \sim \texttt{Poisson}(t\frac{k}{T})$ for arbitrary $t > 0$. According to our network model, the kth subblock (as well as the $k - 1$ before) is fully propagated after $\tau(\tau_0, z, Q/k)$ seconds. Thus, X is equal to $Y_{\tau(\tau_0,z,Q/k)}$ in distribution. $E[X]$ bounds the expected number of subblocks orphaned per summary block. The orphan rate, that is subblocks orphaned per subblocks included, is bounded by

$$\rho(\tau_0, z, Q, T) \leq \frac{E[X]}{k} = \frac{\tau(\tau_0, z, Q/k)\frac{k}{T}}{k} = \frac{\tau_0}{T} + \frac{zQ}{kT}. \tag{4}$$

□

Table 1 shows how the upper bound varies depending on the expected summary block interval T and the subblock count k. The orphan rate for Bitcoin [43] appears in the column for $k = 1$. In this analysis we assume that $Q = 32\,\text{MB}$, $z = 100\,\text{MB/s}$, and $\tau_0 = 5\,\text{s}$. Generally, we can see that the orphan rate decreases with both expected summary block interval T and k. This suggests that, when summary block intervals are constant across protocols, Tailstorm with high k increases fairness over Bitcoin and Tailstorm with lower k.

4.2 Measuring Fairness in Simulation

Section 4.1 provides an analytical bound for Tailstorm's orphan rate. This bound is only tight if all subblocks originate from different miners, which is unlikely to happen in practice. To obtain a more

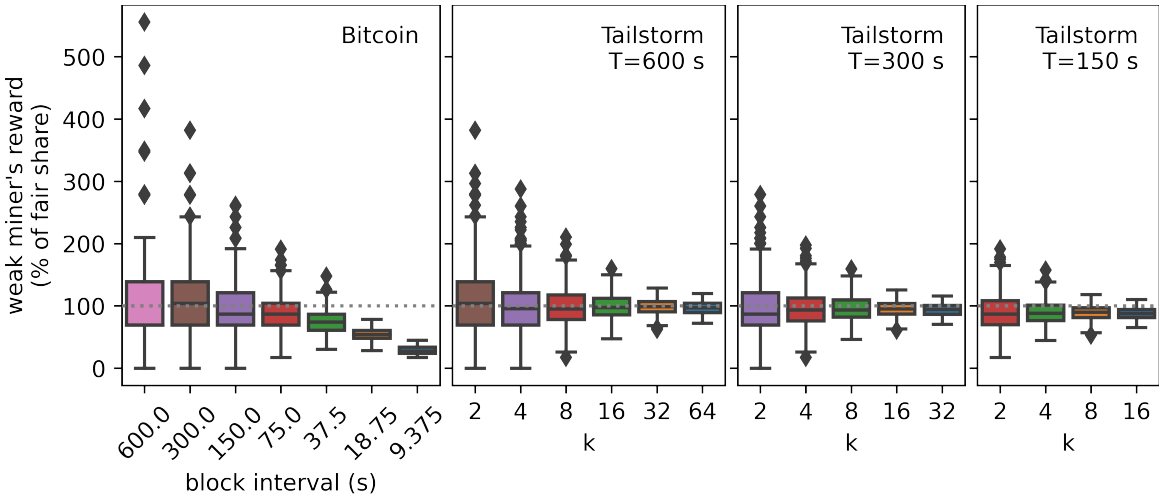


Figure 2: Observed fairness for different configurations of Bitcoin (leftmost facet) and Tailstorm (right facets) with expected summary interval T ranging from 600 to 150 seconds. Colors represent expected subblock mining intervals.

realistic analysis, albeit in a more specific setting, we now consider strong miners who are likely to produce multiple blocks within a single propagation delay. Since the mathematical analysis for this scenario is complex, we rely on simulation instead. Additionally, instead of using the orphan rate metric, itself only a proxy for fairness, we directly measure a miner's deviation from their fair share of rewards.

We implement the virtual environment as described in Section 2, Algorithm 1 and measure how the choice of protocol—Bitcoin and Tailstorm under various parameterizations—affects rewards. The network is configured with one *weak* and one *strong* miner operating with 1 % and 99 % of the total hash rate, respectively. All messages are delayed for 6 seconds.

We simulate one million PoW solutions per configuration. We split the simulation into *independent observations*, each representing one day of protocol execution. Specifically, with T denoting the expected summary block interval, we terminate individual executions when $\lfloor 24 \times 3600 \times k/T \rfloor$ PoWs are mined. For each execution, we identify the longest chain of blocks and calculate the accumulated rewards for both miners. Since each observation covers one day of mining, the variance represents daily volatility of rewards.

A miner's fair share of reward equals its relative hash rate. To measure fairness, we compare actual rewards to the fair share. Figure 2 shows the weak miner's relative reward as a percentage of its fair share. The leftmost facet shows the Bitcoin protocol for expected block intervals ranging from roughly 9 seconds to 600 seconds. The remaining three facets show Tailstorm with *summary* block intervals T of 600, 300, and 150 seconds for k ranging from 2 to 64. Like colors represent such configurations where *subblocks* are generated at the same frequency $\lambda = k/T$. We omit the configurations where the expected subblock interval T/k is lower than the network propagation delay of 6 seconds.

The Figure illustrates the tradeoff between reward fairness and volatility in Bitcoin and Tailstorm. In the leftmost facet, it shows how increasing the expected block interval improves fairness but also increases volatility in Bitcoin. In contrast, the right facets show how Tailstorm can be configured

to independently control both reward fairness (by adjusting the summary block interval T) and volatility (by adjusting k). In particular, choosing higher k leads to more frequent rewards and lower volatility, while longer summary block intervals tend to increase fairness. The latter observation aligns with the results in Table 1, which also shows that fairness tends to increase with the summary block interval.

5 Attack Evaluation

We now extend the virtual environment defined by Algorithm 1 in Section 2 to model adversarial behavior. We then specify and evaluate several attack strategies against Tailstorm.

To account for the possibility of collusion among multiple dishonest parties, we pessimistically assume that they indeed collude. Therefore, we model a single but strong node deviating from the protocol. Throughout the remaining sections we refer to the single dishonest node as the *attacker* and to the honest nodes as *defenders*. The attacker *observes* the virtual environment's state and reacts to any updates accordingly. We describe attacks as *policies*, which determine the *action* to take based on the observed state.

5.1 Network

We deploy a virtual environment comprising n nodes. The first node represents the attacker and is assigned fraction α of the total hash rate. The remaining hash rate is distributed evenly among the $n-1$ defenders. Henceforth, we refer to α as the *attacker's strength*.

In Bitcoin, blocks form a linear chain. If there are two blocks with the same height, only one will be included in the blockchain long term. Since honest Bitcoin nodes prefer and mine on the block first received, nodes with better connectivity obtain an advantage. The same *block race* situation can arise in Tailstorm. Here, there can be only one *summary* at each height and the nodes' preference of summaries (Alg. 3) likewise depends on the order they are received. Attackers able to send blocks more quickly might be able to manipulate the defenders' preference to suit their needs.

We follow Eyal and Sirer [16] and Sapirshtein et al. [47] who model the *block race advantage* with a single network parameter γ, which defines the proportion of defenders, weighted by hash rate, that will opt for the attacker's block in the case of a block race. We pessimistically assume that the attacker receives all blocks without delay. Defender nodes can send blocks to one another with a negligible delay of ε. When the attacker sends a block to a defender, we introduce a random delay, which we draw independently from a uniform distribution on the interval $[0, \frac{n-2}{n-1}\frac{\varepsilon}{\gamma}]$.

Note that we chose these delays to enable comparison with related work [16, 47] on selfish mining against Bitcoin. Other constructions with different distributions are possible but redundant: $\gamma = 0$ implies an inferior network level attacker who wins *no* block races, $\gamma = 1$ models a superior attacker who wins *all* block races, and intermediate attackers can be modeled by choosing γ accordingly. The effect of non-negligible delays among the defenders was analyzed in Section 4.

Theorem 2. *Let $\gamma \in [0, 1]$ be given. For any choice of $\varepsilon > 0$ and $n > \frac{1}{1-\gamma} + 1$, our definition of γ is equal to the attacker's block race advantage in [16, 47].*

Proof. Let A be the event that a randomly chosen honest miner accepts the attacker block over honest and A_i the event that a specific miner accepts the attacker block. By the law of total probability

we have $P[A] = \frac{1}{n-1}\sum_{i=1}^{n-1} P[A_i]$. Note also that if we assume w.l.o.g. that A_{n-1} was the miner of the competing block, then $P[A_{n-1}] = 0$. Since defenders communicate with each other with delay ε while the attacker communicates with defenders with uniform random delay up to time $\frac{n-2}{n-1}\frac{\varepsilon}{\gamma}$, it must be the case that

$$P[A_i] = \varepsilon \left(\frac{n-2}{n-1} \frac{\varepsilon}{\gamma} \right)^{-1}, \quad \forall i < n-1\,, \tag{5}$$

provided that $\frac{n-2}{n-1}\frac{\varepsilon}{\gamma} > \varepsilon$. Choosing $n > \frac{1}{1-\gamma} + 1$ satisfies this constraint. Equality in the constraint is not valid because it forces $P[A_i] = 1$ for all γ. Therefore,

$$\begin{aligned} P[A] &= \frac{1}{n-1} \sum_{i=1}^{n-1} P[A_i] && (6)\\ &= \frac{1}{n-1} \sum_{i=1}^{n-2} P[A_i] && (7)\\ &= \frac{n-2}{n-1} \frac{\varepsilon}{\frac{n-2}{n-1}\frac{\varepsilon}{\gamma}} && (8)\\ &= \gamma. && (9) \end{aligned}$$

□

Constraint $n > \frac{1}{1-\gamma} + 1$, from Theorem 2, is fundamental; if n is smaller, then it is not possible for an honest miner to communicate a block to another honest miner faster than the attacker can. The constraint implies that $\gamma < \frac{n-2}{n-1}$. Hence, we can model $\gamma = 1$ only in the limit that the number of miners approaches infinity, which is not possible in practice. Other authors [16, 47] have directly considered $\gamma = 1$, but we feel that the restriction to $\gamma < 1$ is natural. Given that the miner of the defending block in a block race will never adopt the attacker's competing block, $\gamma = 1$ implies that individual miners have zero hash rate, which is not realistic.

5.2 Observation Space

The virtual environment maintains complex state. It tracks the full history of blocks, delayed messages, and the partial views of all nodes. Some of this information is not available to attackers in practice, other parts are not relevant. To enable concise policies, we follow Sapirshtein et al. [47] and restrict what the attacker can see.

Figure 3 illustrates our notation. At any given time, let $b_\mathrm{a} = \texttt{tip}(1)$ denote the attackers preferred summary block, and let b_d denote the best block (according to `Preference` in Alg. 3) among the preferred blocks of the defenders: $\texttt{tip}(2), \ldots, \texttt{tip}(n)$. We use b_c to refer to the best summary block among the common ancestors of b_a and b_d. In other words, b_c is the latest block that *all* nodes agree upon. For any summary block b, we use $R(b)$ to identify the subblocks which are both a descendant of b and have the same height as b. Note that Tailstorm's chain structure (Alg. 2) implies that $R(b)$ and b together span a tree. Section 3 defines the *depth* of a subblock to be its depth within this tree. We define $R'(b)$ as the largest subset of $R(b)$ such that all subblocks are mined by the attacker and $R'(b) \cup \{b\}$ is still connected. In the following, we use $|S|$ to refer to the cardinality of a set S and $\texttt{depth}[S]$ to refer to the maximum depth among all blocks in S. The attacker observes

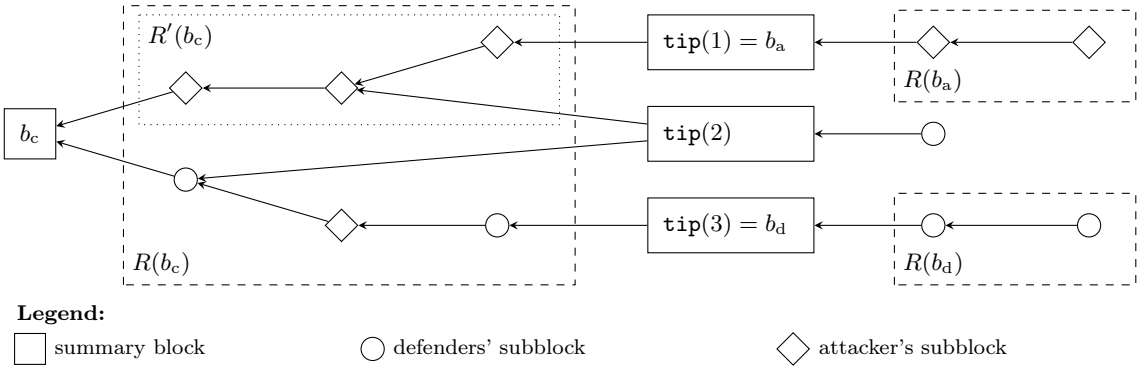


Figure 3: Notation in Section 5.2 Observation Space. This example uses three nodes and $k = 3$.

Algorithm 5: Tailstorm: Attacker

```
1  function Update(tip t, block b) is                    // t: currently preferred block // b: new block
2      pref, share, append ← t, [], [];                  // new preferred tip, blocks to broadcast, templates to append
3      if b is own summary block then pref ← b;
4      W ← set of withheld blocks (= blocks not shared before);
5      withhold, extend ← P(h_a, h_d, s_a, s'_a, s_d, d_a, d'_a, d_d);   // P: attack policy
6      switch withhold do
7          case Adopt do  pref ← b_d                     // b_d: defender's block, see Sec. 5.2 ;
8          case Match do share ← {x ∈ W | progress(x) ≤ progress(b_d)} ;
9          case Override do share ← {x ∈ W | progress(x) ≤ progress(b_d) + 1} ;
10         case Wait do nothing;
11     switch extend do
12         case Inclusive do
13             if |R(pref)| ≥ k then                     // R(b): full subblock tree confirming b, see Sec. 5.2
14                 append ← summary template using R(pref);
15         case Exlusive do
16             if |R'(pref)| ≥ k then                    // R'(b): partial subblock tree confirming b, see Sec. 5.2
17                 append ← summary template using R'(pref);
18     return pref, share, append;
```

$h_\mathbf{a}$: the attacker's height advantage, $\texttt{height}(b_a) - \texttt{height}(b_c)$,
$h_\mathbf{d}$: the defenders' height advantage, $\texttt{height}(b_d) - \texttt{height}(b_c)$,
$s_\mathbf{a}$: the attacker's inclusive subblock count, $|R(b_a)|$,
$s'_\mathbf{a}$: the attacker's exclusive subblock count, $|R'(b_a)|$,
$s_\mathbf{d}$: the defenders' subblock count, $|R(b_d)|$,
$d_\mathbf{a}$: the attacker's inclusive depth, $\texttt{depth}[R(b_a)]$,
$d'_\mathbf{a}$: the attacker's exclusive depth, $\texttt{depth}[R'(b_a)]$, and
$d_\mathbf{d}$: the defenders' depth, $\texttt{depth}[R(b_d)]$.

The example shown Figure 3 implies $(h_a, h_d, s_a, s'_a, s_d, d_a, d'_a, d_d) = (1, 1, 2, 2, 2, 2, 2, 2)$.

5.3 Action Space

The virtual environment (Alg. 1) calls a node's `Update` function whenever this node learns about a new block: after it was mined locally with PoW, appended locally without PoW, or received from the network. We implement the attacker's *dishonest* `Update` function in Algorithm 5. We allow for

generic attacks by letting the attacker choose from a set of potentially dishonest actions. We assume that there is a policy $\mathcal{P}$ which maps observations (see Sect. 5.2) to action tuples of the form (*withhold*, *extend*). The *withhold* action type controls the preferred tip of the chain and the withholding of blocks. We hereby follow closely the actions used by Sapirshtein et al. [47] for selfish mining against Bitcoin [16].

`Wait` Continue mining on b_a and withhold new blocks.
`Match` Release just enough blocks to induce a block race between b_d and an attacker block.
`Override` Release just enough blocks to make the defenders discard their block b_d.
`Adopt` Abort attack, prefer defenders' block b_d, and discard b_a.

Recall from Section 2 that "blocks" refers to vertices in the block DAG. This makes the *withhold* action protocol-agnostic. For Tailstorm, blocks can be subblocks or summaries. Note that while `Adopt` and `Wait` are always feasible, `Match` and `Override` are not. If the attacker's chain is too short, the defenders will not consider adopting it. In such cases, `Match` and `Override`, as implemented in Algorithm 5, fall back to releasing all withheld blocks.

The second action type, *extend*, is specific to Tailstorm. It controls how the attacker assembles new summary blocks, more specifically, which subblocks it considers for selection with Algorithm 4.

`Inclusive` Use all available subblocks to create new summaries.
`Exclusive` Use only subblocks which were mined by the attacker.

Note that the attacker never delays the next summary block longer than necessary. As soon as there are enough subblocks for an inclusive or exclusive summary (according to the chosen action), this summary will be created.

5.4 Reference Policies

We evaluate the following policies. In each, the attacker assembles `Inclusive` summaries.

Honest Emulate Algorithm 3: adopt the longest chain and release all blocks:
`Adopt` if $h_\mathrm{d} > h_\mathrm{a}$. Otherwise `Override`.

Get Ahead Withhold own subblocks, release own summaries:
`Adopt` if $h_\mathrm{d} > h_\mathrm{a}$. `Override` if $h_\mathrm{d} < h_\mathrm{a}$. Otherwise `Wait`.

Minor Delay Withhold own subblocks, override defender summaries as they come out:
`Adopt` if $h_\mathrm{d} > h_\mathrm{a}$. `Wait` if $h_\mathrm{d} = h_\mathrm{c}$. Otherwise `Override`.

To evaluate the policies above, we reuse the simulator from Section 4.2, configuring the network as described in Sections 5.1 to 5.3. We set block race advantage $\gamma \in \{5, 50, 95\}\,\%$ and allow the attacker strength α to range from 20 % to 45 %. We run one hundred simulations per configuration, and stop each as soon as the DAG contains 2048 blocks. At the end of each simulation we select the longest chain of blocks and calculate its rewards. We configure Tailstorm's discount reward scheme, as defined in Section 3.1, with $c = 1$ such that at most one unit of reward is minted per unit of chain progress.

Some policies cause more orphans than others. Assuming effective difficulty adjustment according to Section 3.3, the amount of simulated time depends on the attacker's behaviour. The attacker's

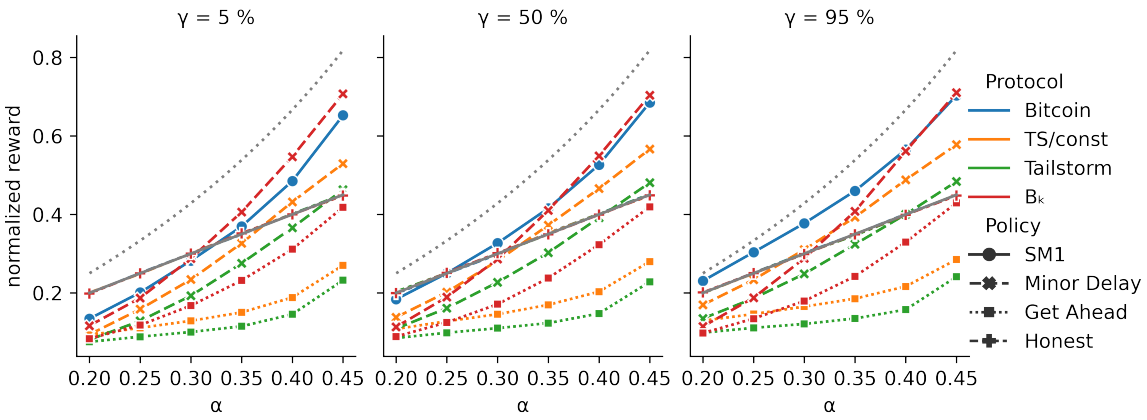


Figure 4: Observed normalized reward as a function of attacker strength α for different protocols (color) and reference policies (line style and markers). The three facets represent the network assumptions $\gamma \in \{5, 50, 95\}$ %. We set protocol parameter $k = 8$ where applicable. TS/const is Tailstorm without reward discounting. We add a gray reference line for the fair reward α (overlaps with honest policy) and a gray dotted curve for the known upper bound $\alpha/(1-\alpha)$.

mining cost is proportional to the time spent on the attack. To facilitate comparison across policies, we normalize rewards with respect to simulated time. Formally, we define the *normalized reward* as the attacker's reward up to block b divided by $\texttt{progress}(b)$. Note that relative reward, a metric commonly used for Bitcoin [16, 47, 54, 55], is not sufficient to account for Tailstorm's reward discounting.

Figure 4 reports the average normalized reward (across 100 simulations) on the y-axis for the varying α on the x-axis and with γ varying by facet. The green curves represent the different reference policies against Tailstorm. We also evaluate the reference policies against the TS/const protocol as defined in Section 3.4 (orange curves) and against $\mathcal{B}_k$ (red curves). In addition, we evaluate the SM1 policy against Bitcoin (blue curve) as described by Sapirshtein et al. [47]. For orientation we include their upper bound $\alpha/(1-\alpha)$ as a gray dotted curve and add a solid gray line for α, the expected reward of honest behaviour.

The figure supports multiple conclusions. First, the curves for the *Honest* policy coinciding with the line for α indicates that the policy indeed replicates honest behavior in all evaluated protocols. Second, comparing the measurements for Tailstorm with the ones for TS/const shows clearly that the discounting of rewards reduces efficacy of all dishonest reference policies. Third, *Minor Delay* is generally the most effective reference policy. Forth, *Minor Delay* produces less reward in Tailstorm than SM1 does in Bitcoin. Fifth, for low γ, *Minor Delay* is more profitable against $\mathcal{B}_k$ than SM1 is against Bitcoin.

Note however, that *Minor Delay* might not be the best strategy against Tailstorm, just like SM1 is not always optimal for Bitcoin [47]. This motivates to search for optimal policies.

6 Attack Search

Previous research has identified optimal attacks against Bitcoin and Ethereum through the use of Markov Decision Processes (MDP) and exhaustive search [21, 47, 55]. However, for more complex

protocols, the state space of MDPs can become prohibitively large, and exhaustive search becomes impractical [26, 29, 54]. As a result, many authors in the past have resorted to evaluating hard-coded attacks instead of searching for optimal policies [9, 13, 17, 29, 34].

We adopt the approach introduced by Hou et al. [26] and employ reinforcement learning (RL) to search for attacks. The observation and action spaces described in Section 5 readily define a partially observable MDP. To search for optimal attacks, we replace the hard-coded policy P with an RL agent that learns to select actions based on past observations.

We utilize off-the-shelf RL tooling by first exposing our simulator and attack space as an OpenAI Gym [10]. Next, we deploy Proximal Policy Optimization (PPO) [48] as our agent. To support reproduction and future research, we release the Gym as Python package [28] and include all training scripts in our open source repository [30].

The modularity of our simulator allows us to apply the same RL pipeline to different protocols. We insert the protocol specifications along with their associated observation and action spaces, while reusing the virtual environment and network assumptions across protocols. For Bitcoin, we adopt the attack space defined by Sapirshtein et al. [47]. Details about the attack spaces for Bitcoin and $\mathcal{B}_k$ are provided in the extended version of this paper.

We use Bitcoin as a reference protocol to measure the completeness of our approach. Previous research by Sapirshtein et al. [47] has yielded the optimal policy. As we will demonstrate, our search mechanism reproduces these results closely.

We search for optimal policies for all possible combinations of attacker strength $\alpha \in \{20, 25, 30, \ldots, 45\}\,\%$, and block race advantage $\gamma \in \{5, 50, 95\}\,\%$. We evaluate four protocols: Bitcoin, $\mathcal{B}_k$, Tailstorm, and TS/const with $k = 8$ where applicable. In total, this amounts to $7 \cdot 3 \cdot 4 = 84$ different learning problems.

For each learning problem, we conduct multiple training runs with varying hyperparameters. For the objective function we choose *normalized reward*, as evaluated for the reference policies in Figure 4. From the training we obtain 1292 policies.

We select the best trained policy for each problem by simulating 100 independent protocol executions for each and proceeding as follows. As in Section 5.4, we stop individual executions as soon as there are 2048 blocks. We then select the longest chain of blocks and observe the attacker's normalized reward. This reward is averaged over the 100 observations per policy to determine the policy with the highest average reward. For reference, we apply the same filtering method to select the best policy among the reference policies from Section 5.4.

Figure 5 shows the performance of the selected policies, with γ varying by facet, α on the x-axis, and average normalized reward on the y-axis. The solid colored curves represent the best trained policies, while the dashed colored curves represent the best reference policies. As in Figure 4, we include gray reference curves for Bitcoin's known upper bound $\alpha/(1-\alpha)$ (dotted) and fair share α (solid). Note that for $\gamma = 50\,\%$, Nakamato and $\mathcal{B}_k$ visually overlap.

In comparing the performance of different protocols, we say that protocol A performs *better* (or *worse*) than B in the event that A returns fewer (alternatively *more*) rewards to the attacker than does B. Figure 5 supports the following conclusions. First, $\mathcal{B}_k$ performs worse than Bitcoin for low γ, but better than Bitcoin for high γ. The break-even point seems to be at $\gamma = 50\,\%$. Second, Tailstorm consistently outperforms Bitcoin and $\mathcal{B}_k$, even with constant rewards (TS/const). Moreover, discounting of rewards consistently makes Tailstorm less susceptible to selfish mining and similar

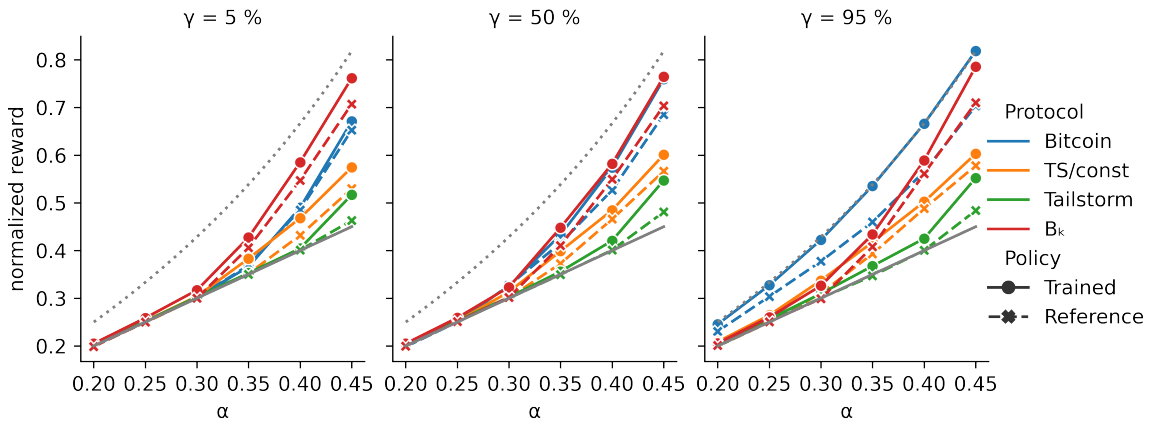


Figure 5: Observed normalized reward of the *best known policy* with the axes set up like in Figure 4. Solid curves with •-markers represent trained reinforcement learning models. Dashed curves with ×-markers represent the reference policies defined in Section 5.4.

incentive layer attacks. Third, the learned policies consistently match or outperform the hard-coded reference policies. This finding is important because anything else would indicate deficiencies in the training. Forth, the learned policies are close to optimal for Bitcoin. This can be seen in two ways. Firstly, Sapirshtein et al. showed that the SM1 policy is close to optimal for $\gamma = 0$ [47, Fig. 1a]. Our figure for $\gamma = 5\%$ shows that the learned policy matches SM1 (the dashed blue curve with ×-markers). Secondly, the authors showed that the optimal policy reaches the upper bound $\alpha/(1-\alpha)$ for $\gamma = 1$ [47, Fig. 1c]. Our figure for $\gamma = 95\%$ shows that the learned policy matches this bound as well.

Until now, we have evaluated the efficacy of policies in absolute terms. We now take a different perspective and ask: how strong must an attacker be for dishonest behaviour to pay off? Recall that honest behaviour implies an expected normalized reward of α (solid gray line in Figures 4 and 5). To answer the question, we calculate break-even points, which represent the minimum relative hash rate α, such that following the policy produces more than α of normalized reward. We start from the optimal policies presented in Figure 5 and select only those policies that feature dishonest behavior. Recall that each of the remaining policies was trained on a fixed relative hash rate α. For each protocol and choice of γ, we select the dishonest policy trained for the lowest α. We then evaluate this policy against alternative α values ranging from 5 % to 50 %. Our simulator provides noisy observations and the reward distribution varies stochastically. Hence, we use Bayesian optimization to minimize the difference between the observed reward and α, i. e., to get as close to the break-even point as possible. Table 2 reports the results. Observe that for the trained policies, the break-even points are consistently either lower or close to the break-even points of the hard-coded reference policies. $\mathcal{B}_k$, TS/const, and Tailstorm are also less sensitive to changes in γ than Bitcoin. Tailstorm has the highest break-even points among all protocols, indicating that it is most resilient to incentive layer attacks.

Table 2: Break-even points: minimal α (in %) where dishonest behaviour (Figure 5) pays off.

	Trained Policy			Reference Policy		
γ	5 %	50 %	95 %	5 %	50 %	95 %
Bitcoin	33.6	26.5	≤ 5	32.7	24.5	≤ 5
TS/const	28.9	28.6	21.7	37.9	32.6	28.4
Tailstorm	41.6	35.8	35.8	43.7	41.0	39.3
$\mathcal{B}_k$	22.2	27.8	25.3	31.3	31.1	30.7

7 Tailstorm Cryptocurrency

So far, we have focused on Tailstorm's consensus and incentive mechanisms. Throughout our analysis, we have assumed a cryptocurrency on the application layer that facilitates the creation and distribution of rewards to participants of the consensus layer. In this section, we describe and discuss the Tailstorm cryptocurrency and our prototype implementation.

7.1 Transaction Handling

Tailstorm implements the unspent transaction output (UTXO) model as it is used in Bitcoin [39]: Each UTXO represents a designated amount of cryptocurrency. Transactions consume and create UTXOs, but they never create more cryptocurrency than they consume. Consumed UTXOs cannot be consumed again. Ownership and transfer of value follows from restricting the consummation of UTXOs to the holders of specific cryptographic keys.

Tailstorm uses the same public key cryptography as Bitcoin, and it also supports Bitcoin's UTXO scripting facility. The dissemination and processing of transactions follows Bitcoin's approach as well. However, as we describe next, Tailstorm deviates from Bitcoin in how transactions are recorded in the blockchain.

Taking inspiration from Storm [4], each subblock contains a list of transactions. Let $\texttt{tx}_\text{a}$ be a transaction listed at position $\texttt{idx}_\text{a}$ in subblock $\texttt{sub}_\text{a}$ summarized in summary block $\texttt{sum}_\text{a}$ and let $\texttt{tx}_\text{b}$, $\texttt{idx}_\text{b}$, $\texttt{sub}_\text{b}$, and $\texttt{sum}_\text{b}$ be defined similarly. Assume $\texttt{tx}_\text{a}$ and $\texttt{tx}_\text{b}$ are incompatible, e. g., because they spend the same UTXO twice. If $\texttt{tx}_\text{a}$ and $\texttt{tx}_\text{b}$ are listed in the same subblock, i. e., $\texttt{sub}_\text{a} = \texttt{sub}_\text{b}$ and $\texttt{idx}_\text{a} \neq \texttt{idx}_\text{b}$, then this subblock is marked invalid. We proceed similarly if $\texttt{tx}_\text{a}$ and $\texttt{tx}_\text{b}$ are summarized by different summaries: due to Tailstorm's chain structure (see Alg. 2) $\texttt{sum}_\text{a}$ and $\texttt{sum}_\text{b}$ must differ in height and we mark the higher one as invalid. Honest nodes ignore invalid blocks and all their descendants at the consensus layer. Hence, under the above circumstances, incompatible transactions are not persisted in the blockchain, as is the case in Bitcoin.

However, special attention is required for incompatible transactions in different subblocks summarized by the same summary block, i. e., $\texttt{sub}_\text{a} \neq \texttt{sub}_\text{b}$ and $\texttt{sum}_\text{a} = \texttt{sum}_\text{b}$. A critical assumption of our analyses as well as parallel PoW in general [29] is that subblocks confirming the same summary are compatible and thus can be mined independently. We thus cannot mark the summary invalid in this case and the incompatible transactions are persisted in the blockchain. To resolve this conflict, Tailstorm executes only one of the transactions at the application layer and ignores the other according the following rules. Let $\texttt{dst}_\text{a}$ denote the distance of $\texttt{sub}_\text{a}$ to $\texttt{sum}_\text{a}$ in the block DAG, and let $\texttt{dst}_\text{b}$ be defined similarly. If $\texttt{dst}_\text{a} < \texttt{dst}_\text{b}$, then $\texttt{tx}_\text{a}$ is ignored. Ties are broken by the PoW hash

function, i. e., if $\texttt{dst}_a = \texttt{dst}_b$ and $\texttt{hash}(\texttt{sub}_a) > \texttt{hash}(\texttt{sub}_b)$, then $\texttt{tx}_a$ is ignored. If $\texttt{tx}_a$ is ignored then all dependent transactions (i.e. those spending the UTXOs produced by $\texttt{tx}_a$) are ignored as well. Note that even though incompatible transactions are persisted in the blockchain, the double-spending semantics are equivalent to Bitcoin: offending transactions off the longest branch are ignored.

7.2 Fast Confirmations

To reap the full consistency guarantees of parallel PoW [29], prudent cryptocurrency users should wait for one summary block confirmation before accepting their transactions as final. For example, if their transaction is included in subblock $\texttt{sub}_a$ and first summarized in $\texttt{sum}_a$, then they should wait for another valid summary $\texttt{sum}_b$ with $\texttt{height}(\texttt{sum}_b) = \texttt{height}(\texttt{sum}_a) + 1$. Assuming a 10 minute summary block interval and large k, the full confirmation will likely occur in 10 to 20 minutes, depending on whether the transaction was included early in the subblock tree or later.

If time is short and the transacted value is low, e. g. if the user is selling a cup of coffee to go, they can consider waiting for a number of subblock confirmations instead. For example, consider Tailstorm with $k = 60$ and a 10 minute summary block interval. Subblocks will be mined every 10 seconds in expectation. The heuristics are similar to a fast version of Bitcoin: If the seller waits for 6 subblock confirmations, i. e., for a subblock $\texttt{sub}_b$ with $\texttt{depth}(\texttt{sub}_b) = \texttt{depth}(\texttt{sub}_a) + 6$, the settlement will take about one minute. Invalidation of the payment, e. g. due to a double spend, implies a fork of the subblock tree of length 6 or more. Tailstorm discounts mining rewards according to the depth of the subblock tree; whereas the tree could have achieved depth 60, it now can achieve depth at most 54. One tenth of the minted rewards is lost, and the cost of the coffee is dwarfed.

7.3 Tailstorm Prototype

We have implemented Tailstorm and make the code available online [23]. We started from a fork Bitcoin Unlimited's implementation of Bitcoin Cash. Our fork implements the Tailstorm consensus layer and incentive mechanism as specified in Section 3. The prototype uses $k = 3$, but is easily configurable by changing a compile-time flag. The DAA is calibrated for a target summary block interval of ten minutes. Subblocks are expected to arrive every 200 seconds.

To minimize propagation delays, we have implemented several network compression mechanisms. First, we adapt the Graphene protocol [41] to avoid redundant transmission of transactions. Second, we adapt the Compact Block protocol [12] to reduce the size of summary blocks. Third, the transaction list is encoded bit-efficiently in each subblock.

We leave the implementation of Tailstorm's application layer, i. e. the transaction handling as described in Section 7.1, for future work. Finally, we note that our implementation has minimal testing and, being a prototype, is not ready for production use.

8 Discussion

Tailstorm inherits the consistency guarantees of parallel PoW consensus [29]. To increase fairness, we discount rewards based on the tree structure of subblocks which itself is inspired from Bobtail [9]. To obtain fast confirmations, we write transactions into subblocks like Storm [4]. Alongside these foundational influences, we incorporate insights from a variety of related works, which we here can

only list partially. For more details we refer to the surveys of Garay and Kiayias [19], describing the different ways of defining the consensus problem, and Bano et al. [6], focusing on the different solution approaches and protocols.

Tailstorm improves fairness by avoiding orphans and discounting rewards. The former is not new [2, 50, 51], however, the motivation differs. Sompolinsky and Zohar [51] propose to improve Bitcoin's *consistency* by referencing blocks that would otherwise be orphaned (called *uncles*) and then counting both blocks and uncles (GHOST-rule) whereas Bitcoin counts only the blocks (longest chain rule). However, the authors explicitly avoid rewarding uncles and hence do not improve *fairness*. Follow-up work [2, 50] does not discuss rewards at all. In Ethereum PoW, a deployed variant of GHOST [51], and Kaspa, a deployed variant of GHOSTDAG [50], uncles receive *partial* rewards and the blocks that include uncles get *additional* rewards. The unfair dynamic of Bitcoin, where included blocks receive the full reward and orphans none, is largely preserved. We think that GHOST-like protocols can become more fair by applying Tailstorm's ideas: discounting of rewards, applied equally to all involved miners.

Pass and Shi [42] explicitly set out to increase fairness in PoW. Their Fruitchains protocol uses two kinds of blocks like Tailstorm. Blocks record fruits and fruits record transactions. Unlike in Tailstorm, both fruits and blocks require PoW. Applying the *2-for-1 trick* of Garay et al. [20], both are mined with the same hash puzzle observing different bits of the output. Fruits have a lower difficulty and thus are created more frequently. The rewards are distributed evenly across recent fruits while blocks receive nothing. According to Zhang and Preneel [54], Fruitchains are more vulnerable to incentive layer attacks than Bitcoin for block race advantage $\gamma = 0$, and thereby less fair. Furthermore, Fruitchains suffer from a tradeoff between fairness and transaction confirmation time [54]. A related line of research [5, 53] extends the *2-for-1* into an *n-for-1* trick and makes the miners work on multiple chains in parallel. The chains are then interleaved to form a single coherent transaction ledger. Unlike Fruitchains, however, Prism [5] and OHIE [53] focus on consistency, liveness, and throughput. We leave for future work to investigate whether and how the fairness of these protocols can be improved by applying Tailstorm's reward discounting.

Kiffer and Rajaraman [33] propose to discount rewards inversely proportional to the overall hash rate participating in the system. This reduces centralization in their model, but we worry that motivating lower hash rates might harm the primary goals of consensus: consistency and liveness. Their mechanism poses an alternative to Bitcoin's halving mechanism, which statically reduces block rewards by 50 % roughly once every 4 years, to avoid long term inflation of the cryptocurrency. In contrast, Tailstorm's reward discounting is focused on the short term and punishes non-linearities within a single subblock tree.

But our approach is not without limitations. In Section 4, we analyze the impact of message propagation delays on fairness. We assume that these delays are uniform, affecting all miners equally. However, in practice, propagation delays depend on the connectivity of individual miners. Some inequalities arise from economies of scale, larger miners can invest more in low latency connections, others from the underlying physics of information propagation: miners located in the same region or on the same continent implicitly form are cartel, whereas joining the network from a distant location puts them at a disadvantage. We leave for future work to analyze how Tailstorm is affected by more realistic network topologies [1, 14, 38, 44, 45].

In Section 6, we perform a search for optimal attack strategies. Our search algorithm is based on

reinforcement learning (RL), which means it is not exhaustive. It is possible that the RL agent did not discover the optimal strategy against certain protocols. This challenges our conclusion that Tailstorm is less susceptible to incentive layer attacks than the other protocols, Bitcoin and $\mathcal{B}_k$. To encourage further exploration we release the RL Gym environment on PyPI [28], allowing others to discover more effective attacks. Assuming the absence of better strategies, our results can be confirmed by encoding our attack space as a Markov Decision Process (MDP) and employing exhaustive search techniques, as has been done for Bitcoin [21, 47], Ethereum [55] and other longest chain protocols [54]. However, to the best of our knowledge, such techniques have not yet been successfully applied to DAG-based protocols like Tailstorm.

Another limitation is that our action space might not cover all possible attacks. We closely follow the modeling of the selfish mining attack space against Bitcoin [47], where we observe consent that the four actions `Adopt`, `Match`, `Override`, and `Wait` are indeed complete. We are convinced that adding two actions for summary formation, `Inclusive` and `Exclusive`, is enough to represent the most profitable attacks. However, our conclusions can be challenged by presenting a more effective incentive layer attack against Tailstorm, which cannot be expressed in our attack space. We encourage this endeavour by making all analytical code available online [30]. After modifying the attack space, new strategies can be drafted, evaluated, and optimized, while the virtual environment, protocol specifications, and RL attack search can be reused without changes.

On a separate note, Carlsten et al. [11] demonstrate that selfish mining becomes more profitable when considering transaction fees in addition to mining rewards. They present a strategy targeting Bitcoin which leverages transaction fees to outperform honest behavior for any $\alpha > 0$ and $\gamma < 1$. Similar attacks are likely feasible against all PoW cryptocurrencies, including Tailstorm, however they also exceed the scope of this paper.

Lastly, Arnosti and Weinberg [3] show that even small cost imbalances or economies of scale lead to highly centralized PoW mining ecosystems. While, Tailstorm reduces the imbalances compared to other PoW protocols we studied, it cannot fully mitigate centralization. Our notion of fairness revolves around rewarding miners in proportion to their hash rate. This definition is not new [1, 7, 31, 42, 54]. However, from an economic standpoint, to prevent centralization, miners should be rewarded in proportion to their operational mining costs [3]. Unfortunately, achieving this goal seems to be impossible. Mining costs primarily depend on the price and the availability of energy and specialized mining hardware. Purchasing energy in large quantities tends to be more cost-effective, and strong miners may even consider operating their own power plants. Scaling up operations also enables miners to develop and deploy more efficient mining hardware, and then selling it to weak miners after it has become outdated [52]. Given that these unfair scaling effects do not only affect Tailstorm, but PoW cryptocurrencies in general, one might be lead to consider alternative approaches like proof-of-stake [22, 32, 49]. But even then, it remains questionable whether any permissionless system can fully avoid centralization [36].

9 Conclusion

Tailstorm integrates parallel PoW [29], partial transaction confirmation [4, 9], and a novel incentive mechanism—reward discounting—into a PoW cryptocurrency that provides fast and secure confirmations for its users and fair rewards for its miners. We thereby solve a long standing issue of longest

chain protocols, where the operator has to choose between either a short block interval with fast but less reliable confirmations and unfair rewards, or a long block interval with slow confirmations and more fair but infrequent rewards. Our prototype demonstrates that Tailstorm can replace Bitcoin not only theoretically but also in practice.

Contribution Statement

George originally developed the idea of reward discounting as a way to combat withholding attacks in Bobtail. George and Gregory refined that idea into an early version of Tailstorm suitable for a hard fork of Bitcoin Cash. They also contributed to the prototype implementation. Patrik contributed the protocol specification, simulation-based analyses, and reinforcement learning interface. George contributed the analytical orphan rate analysis. George and Patrik contributed the hard-coded reference policies. Ben and Patrik conducted the attack search with reinforcement learning. Guided by earlier stages of the analyses, Patrik provided improvements to Tailstorm consensus and its transaction handling. Patrik and George wrote this paper.

Acknowledgements

We wish to thank Bitcoin Unlimited for their financial and technical support as well as Michael Fröwis for his review of this work and for the helpful suggestions he provided.

References

[1] Mohamed Alzayat, Johnnatan Messias, Balakrishnan Chandrasekaran, Krishna P. Gummadi, and Patrick Loiseau. *Modeling Coordinated vs. P2P Mining: An Analysis of Inefficiency and Inequality in Proof-of-Work Blockchains*. Preprint. 2021. https://arxiv.org/abs/2106.02970.

[2] Ignacio Amores-Sesar, Christian Cachin, and Anna Parker. "Generalizing weighted trees: a bridge from Bitcoin to GHOST". in: *AFT '21: 3rd ACM Conference on Advances in Financial Technologies, Arlington, Virginia, USA, September 26 - 28, 2021*. Ed. by Foteini Baldimtsi and Tim Roughgarden. ACM, 2021, pp. 156–169. https://doi.org/10.1145/3479722.3480995.

[3] Nick Arnosti and S. Matthew Weinberg. "Bitcoin: A Natural Oligopoly". In: *Manag. Sci.* 68.7 (2022), pp. 4755–4771. https://doi.org/10.1287/mnsc.2021.4095.

[4] awemany. *Storm*. https://github.com/awemany/storm-sim/blob/master/whitepaper/. 2019.

[5] Vivek Kumar Bagaria, Sreeram Kannan, David Tse, Giulia Fanti, and Pramod Viswanath. "Prism: Deconstructing the Blockchain to Approach Physical Limits". In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*. Ed. by Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz. ACM, 2019, pp. 585–602. https://doi.org/10.1145/3319535.3363213.

[6] Shehar Bano, Alberto Sonnino, Mustafa Al-Bassam, Sarah Azouvi, Patrick McCorry, Sarah Meiklejohn, and George Danezis. "SoK: Consensus in the Age of Blockchains". In: *Proceedings of the 1st ACM Conference on Advances in Financial Technologies, AFT 2019, Zurich, Switzerland, October 21-23, 2019*. ACM, 2019, pp. 183–198. https://doi.org/10.1145/3318041.3355458.

[7] Georgios Birmpas, Elias Koutsoupias, Philip Lazos, and Francisco J. Marmolejo Cossío. "Fairness and Efficiency in DAG-Based Cryptocurrencies". In: *Financial Cryptography and Data Security - 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10-14, 2020 Revised Selected Papers*. Ed. by Joseph Bonneau and Nadia Heninger. Vol. 12059. Lecture Notes in Computer Science. Springer, 2020, pp. 79–96. https://doi.org/10.1007/978-3-030-51280-4_6.

[8] George Bissias. "Radium: Improving Dynamic PoW Targeting". In: *Data Privacy Management, Cryptocurrencies and Blockchain Technology - ESORICS 2020 International Workshops, DPM 2020 and CBT 2020, Guildford, UK, September 17-18, 2020, Revised Selected Papers*. Ed. by Joaquín García-Alfaro, Guillermo Navarro-Arribas, and Jordi Herrera-Joancomartí. Vol. 12484. Lecture Notes in Computer Science. Springer, 2020, pp. 374–389. https://doi.org/10.1007/978-3-030-66172-4_24.

[9] George Bissias and Brian Neil Levine. "Bobtail: Improved Blockchain Security with Low-Variance Mining". In: *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society, 2020. https://www.ndss-symposium.org/ndss-paper/bobtail-improved-blockchain-security-with-low-variance-mining/.

[10] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. *OpenAI Gym*. Preprint. 2016. https://arxiv.org/abs/1606.01540.

[11] Miles Carlsten, Harry A. Kalodner, S. Matthew Weinberg, and Arvind Narayanan. "On the Instability of Bitcoin Without the Block Reward". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*. Ed. by Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi. ACM, 2016, pp. 154–167. https://doi.org/10.1145/2976749.2978408.

[12] Matt Corallo. *BIP152: Compact Block Relay*. https://github.com/bitcoin/bips/blob/master/bip-0152.mediawiki. 2016.

[13] Francisco J. Marmolejo Cossío, Eric Brigham, Benjamin Sela, and Jonathan Katz. "Competing (Semi-)Selfish Miners in Bitcoin". In: *Proceedings of the 1st ACM Conference on Advances in Financial Technologies, AFT 2019, Zurich, Switzerland, October 21-23, 2019*. ACM, 2019, pp. 89–109. https://doi.org/10.1145/3318041.3355471.

[14] Sergi Delgado-Segura, Surya Bakshi, Cristina Pérez-Solà, James Litton, Andrew Pachulski, Andrew Miller, and Bobby Bhattacharjee. "TxProbe: Discovering Bitcoin's Network Topology Using Orphan Transactions". In: *Financial Cryptography and Data Security - 23rd International Conference, FC 2019, Frigate Bay, St. Kitts and Nevis, February 18-22, 2019, Revised Selected Papers*. Ed. by Ian Goldberg and Tyler Moore. Vol. 11598. Lecture Notes in Computer Science. Springer, 2019, pp. 550–566. https://doi.org/10.1007/978-3-030-32101-7_32.

[15] Amir Dembo, Sreeram Kannan, Ertem Nusret Tas, David Tse, Pramod Viswanath, Xuechao Wang, and Ofer Zeitouni. "Everything is a Race and Nakamoto Always Wins". In: *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*. Ed. by Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna. ACM, 2020, pp. 859–878. https://doi.org/10.1145/3372297.3417290.

[16] Ittay Eyal and Emin Gün Sirer. "Majority Is Not Enough: Bitcoin Mining Is Vulnerable". In: *Financial Cryptography and Data Security - 18th International Conference, FC 2014, Christ Church, Barbados, March 3-7, 2014, Revised Selected Papers*. Ed. by Nicolas Christin and Reihaneh Safavi-Naini. Vol. 8437. Lecture Notes in Computer Science. Springer, 2014, pp. 436–454. https://doi.org/10.1007/978-3-662-45472-5_28.

[17] Chen Feng and Jianyu Niu. "Selfish Mining in Ethereum". In: *39th IEEE International Conference on Distributed Computing Systems, ICDCS 2019, Dallas, TX, USA, July 7-10, 2019*. IEEE, 2019, pp. 1306–1316. https://doi.org/10.1109/ICDCS.2019.00131.

[18] Daniel Fullmer and A. Stephen Morse. "Analysis of Difficulty Control in Bitcoin and Proof-of-Work Blockchains". In: *57th IEEE Conference on Decision and Control, CDC 2018, Miami, FL, USA, December 17-19, 2018*. IEEE, 2018, pp. 5988–5992. https://doi.org/10.1109/CDC.2018.8619082.

[19] Juan A. Garay and Aggelos Kiayias. "SoK: A Consensus Taxonomy in the Blockchain Era". In: *Topics in Cryptology - CT-RSA 2020 - The Cryptographers' Track at the RSA Conference 2020, San Francisco, CA, USA, February 24-28, 2020, Proceedings*. Ed. by Stanislaw Jarecki. Vol. 12006. Lecture Notes in Computer Science. Springer, 2020, pp. 284–318. https://doi.org/10.1007/978-3-030-40186-3_13.

[20] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. "The Bitcoin Backbone Protocol: Analysis and Applications". In: *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part II*. ed. by Elisabeth Oswald and Marc Fischlin. Vol. 9057. Lecture Notes in Computer Science. Springer, 2015, pp. 281–310. https://doi.org/10.1007/978-3-662-46803-6_10.

[21] Arthur Gervais, Ghassan O. Karame, Karl Wüst, Vasileios Glykantzis, Hubert Ritzdorf, and Srdjan Capkun. "On the Security and Performance of Proof of Work Blockchains". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*. Ed. by Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi. ACM, 2016, pp. 3–16. https://doi.org/10.1145/2976749.2978341.

[22] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. "Algorand: Scaling Byzantine Agreements for Cryptocurrencies". In: *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*. ACM, 2017, pp. 51–68. https://doi.org/10.1145/3132747.3132757.

[23] Greg Griffith and George Bissias. *Tailstorm Node Implementation*. https://gitlab.com/georgebissias/BCHUnlimited/-/tree/tailstorm_prototype. 2023.

[24] Dongning Guo and Ling Ren. "Bitcoin's Latency-Security Analysis Made Simple". In: *Proceedings of the 4th ACM Conference on Advances in Financial Technologies, AFT 2022, Cambridge, MA, USA, September 19-21, 2022*. Ed. by Maurice Herlihy and Neha Narula. ACM, 2022, pp. 244–253. https://doi.org/10.1145/3558535.3559791.

[25] Thomas M. Harding. "Real-Time Block Rate Targeting". In: *Ledger* 5 (2020). https://doi.org/10.5195/ledger.2020.195.

[26] Charlie Hou, Mingxun Zhou, Yan Ji, Phil Daian, Florian Tramèr, Giulia Fanti, and Ari Juels. "SquirRL: Automating Attack Analysis on Blockchain Incentive Mechanisms with Deep Reinforcement Learning". In: *28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21-25, 2021*. The Internet Society, 2021. https://www.ndss-symposium.org/ndss-paper/squirrl-automating-attack-analysis-on-blockchain-incentive-mechanisms-with-deep-reinforcement-learning/.

[27] Geir Hovland and Jan Kucera. "Nonlinear Feedback Control and Stability Analysis of a Proof-of-Work Blockchain". In: *Modeling, Identification and Control* 38.4 (2017), pp. 157–168. https://doi.org/10.4173/mic.2017.4.1.

[28] Patrik Keller. *PyPI Release of our Reinforcement Learning Environment as OpenAI Gym*. https://pypi.org/project/cpr-gym/0.7.0/. 2023.

[29] Patrik Keller and Rainer Böhme. "Parallel Proof-of-Work with Concrete Bounds". In: *Proceedings of the 4th ACM Conference on Advances in Financial Technologies, AFT 2022, Cambridge, MA, USA, September 19-21, 2022*. Ed. by Maurice Herlihy and Neha Narula. ACM, 2022, pp. 1–15. https://doi.org/10.1145/3558535.3559773.

[30] Patrik Keller and Ben Glickenhaus. *Source Code for our Simulator, Evaluations, and Attack Search*. https://github.com/pkel/cpr/tree/aft23. 2023.

[31] Aggelos Kiayias, Elias Koutsoupias, Maria Kyropoulou, and Yiannis Tselekounis. "Blockchain Mining Games". In: *Proceedings of the 2016 ACM Conference on Economics and Computation, EC '16, Maastricht, The Netherlands, July 24-28, 2016*. Ed. by Vincent Conitzer, Dirk Bergemann, and Yiling Chen. ACM, 2016, pp. 365–382. https://doi.org/10.1145/2940716.2940773.

[32] Aggelos Kiayias, Alexander Russell, Bernardo David, and Roman Oliynykov. "Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol". In: *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part I*. ed. by Jonathan Katz and Hovav Shacham. Vol. 10401. Lecture Notes in Computer Science. Springer, 2017, pp. 357–388. https://doi.org/10.1007/978-3-319-63688-7_12.

[33] Lucianna Kiffer and Rajmohan Rajaraman. "HaPPY-Mine: Designing a Mining Reward Function". In: *Financial Cryptography and Data Security - 25th International Conference, FC 2021, Virtual Event, March 1-5, 2021, Revised Selected Papers, Part II*. ed. by Nikita Borisov and Claudia Díaz. Vol. 12675. Lecture Notes in Computer Science. Springer, 2021, pp. 250–268. https://doi.org/10.1007/978-3-662-64331-0_13.

[34] Lucianna Kiffer, Rajmohan Rajaraman, and Abhi Shelat. "A Better Method to Analyze Blockchain Consistency". In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*. Ed. by David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang. ACM, 2018, pp. 729–744. https://doi.org/10.1145/3243734.3243814.

[35] Daniel Kraft. "Difficulty control for blockchain-based consensus systems". In: *Peer-to-Peer Netw. Appl.* 9.2 (2016), pp. 397–413. https://doi.org/10.1007/s12083-015-0347-x.

[36] Yujin Kwon, Jian Liu, Minjeong Kim, Dawn Song, and Yongdae Kim. "Impossibility of Full Decentralization in Permissionless Blockchains". In: *Proceedings of the 1st ACM Conference on Advances in Financial Technologies, AFT 2019, Zurich, Switzerland, October 21-23, 2019*. ACM, 2019, pp. 110–123. https://doi.org/10.1145/3318041.3355463.

[37] Jing Li, Dongning Guo, and Ling Ren. "Close latency-security trade-off for the Nakamoto consensus". In: *AFT '21: 3rd ACM Conference on Advances in Financial Technologies, Arlington, Virginia, USA, September 26 - 28, 2021*. Ed. by Foteini Baldimtsi and Tim Roughgarden. ACM, 2021, pp. 100–113. https://doi.org/10.1145/3479722.3480992.

[38] Sami Ben Mariem, Pedro Casas, Matteo Romiti, Benoit Donnet, Rainer Stütz, and Bernhard Haslhofer. "All that Glitters is not Bitcoin - Unveiling the Centralized Nature of the BTC (IP) Network". In: *NOMS 2020 - IEEE/IFIP Network Operations and Management Symposium, Budapest, Hungary, April 20-24, 2020*. IEEE, 2020, pp. 1–9. https://doi.org/10.1109/NOMS47738.2020.9110354.

[39] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. https://bitcoin.org/bitcoin.pdf. 2008.

[40] Kevin Alarcón Negy, Peter R. Rizun, and Emin Gün Sirer. "Selfish Mining Re-Examined". In: *Financial Cryptography and Data Security - 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10-14, 2020 Revised Selected Papers*. Ed. by Joseph Bonneau and Nadia Heninger. Vol. 12059. Lecture Notes in Computer Science. Springer, 2020, pp. 61–78. https://doi.org/10.1007/978-3-030-51280-4_5.

[41] A. Pinar Ozisik, Gavin Andresen, Brian Neil Levine, Darren Tapp, George Bissias, and Sunny Katkuri. "Graphene: efficient interactive set reconciliation applied to blockchain propagation". In: *Proceedings of the ACM Special Interest Group on Data Communication, SIGCOMM 2019, Beijing, China, August 19-23, 2019*. Ed. by Jianping Wu and Wendy Hall. ACM, 2019, pp. 303–317. https://doi.org/10.1145/3341302.3342082.

[42] Rafael Pass and Elaine Shi. "FruitChains: A Fair Blockchain". In: *Proceedings of the ACM Symposium on Principles of Distributed Computing, PODC 2017, Washington, DC, USA, July 25-27, 2017*. Ed. by Elad Michael Schiller and Alexander A. Schwarzmann. ACM, 2017, pp. 315–324. https://doi.org/10.1145/3087801.3087809.

[43] Peter R. Rizun. "Subchains: A Technique to Scale Bitcoin and Improve the User Experience". In: *Ledger* 1 (2016), pp. 38–52. https://ledgerjournal.org/ojs/index.php/ledger/article/view/40.

[44] Elias Rohrer and Florian Tschorsch. "Blockchain Layer Zero: Characterizing the Bitcoin Network through Measurements, Models, and Simulations". In: *46th IEEE Conference on Local Computer Networks, LCN 2021, Edmonton, AB, Canada, October 4-7, 2021*. IEEE, 2021, pp. 9–16. https://doi.org/10.1109/LCN52139.2021.9524930.

[45] Elias Rohrer and Florian Tschorsch. "Kadcast: A Structured Approach to Broadcast in Blockchain Networks". In: *Proceedings of the 1st ACM Conference on Advances in Financial Technologies, AFT 2019, Zurich, Switzerland, October 21-23, 2019*. ACM, 2019, pp. 199–213. https://doi.org/10.1145/3318041.3355469.

[46] Sheldon M. Ross. *Introduction to Probability Models*. Elsevier, 2014. https://doi.org/10.1016/C2012-0-03564-8.

[47] Ayelet Sapirshtein, Yonatan Sompolinsky, and Aviv Zohar. "Optimal Selfish Mining Strategies in Bitcoin". In: *Financial Cryptography and Data Security - 20th International Conference, FC 2016, Christ Church, Barbados, February 22-26, 2016, Revised Selected Papers*. Ed. by Jens Grossklags and Bart Preneel. Vol. 9603. Lecture Notes in Computer Science. Springer, 2016, pp. 515–532. https://doi.org/10.1007/978-3-662-54970-4_30.

[48] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. *Proximal Policy Optimization Algorithms*. Preprint. 2017. https://arxiv.org/abs/1707.06347.

[49] Jakub Sliwinski and Roger Wattenhofer. "Asynchronous Proof-of-Stake". In: *Stabilization, Safety, and Security of Distributed Systems - 23rd International Symposium, SSS 2021, Virtual Event, November 17-20, 2021, Proceedings*. Ed. by Colette Johnen, Elad Michael Schiller, and Stefan Schmid. Vol. 13046. Lecture Notes in Computer Science. Springer, 2021, pp. 194–208. https://doi.org/10.1007/978-3-030-91081-5_13.

[50] Yonatan Sompolinsky, Shai Wyborski, and Aviv Zohar. "PHANTOM GHOSTDAG: a scalable generalization of Nakamoto consensus: September 2, 2021". In: *AFT '21: 3rd ACM Conference on Advances in Financial Technologies, Arlington, Virginia, USA, September 26 - 28, 2021*. Ed. by Foteini Baldimtsi and Tim Roughgarden. ACM, 2021, pp. 57–70. https://doi.org/10.1145/3479722.3480990.

[51] Yonatan Sompolinsky and Aviv Zohar. "Secure High-Rate Transaction Processing in Bitcoin". In: *Financial Cryptography and Data Security - 19th International Conference, FC 2015, San Juan, Puerto Rico, January 26-30, 2015, Revised Selected Papers*. Ed. by Rainer Böhme and Tatsuaki Okamoto. Vol. 8975. Lecture Notes in Computer Science. Springer, 2015, pp. 507–527. https://doi.org/10.1007/978-3-662-47854-7_32.

[52] Aviv Yaish and Aviv Zohar. "Pricing ASICs for Cryptocurrency Mining". In: *Proceedings of the 5th ACM Conference on Advances in Financial Technologies, AFT 2022, Princeton, NJ, USA, October 21-25, 2023*. LIPIcs, 2023.

[53] Haifeng Yu, Ivica Nikolic, Ruomu Hou, and Prateek Saxena. "OHIE: Blockchain Scaling Made Simple". In: *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*. IEEE, 2020, pp. 90–105. https://doi.org/10.1109/SP40000.2020.00008.

[54] Ren Zhang and Bart Preneel. "Lay Down the Common Metrics: Evaluating Proof-of-Work Consensus Protocols' Security". In: *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 175–192. https://doi.org/10.1109/SP.2019.00086.

[55] Roi Bar Zur, Ittay Eyal, and Aviv Tamar. "Efficient MDP Analysis for Selfish-Mining in Blockchains". In: *AFT '20: 2nd ACM Conference on Advances in Financial Technologies, New York, NY, USA, October 21-23, 2020*. ACM, 2020, pp. 113–131. https://doi.org/10.1145/3419614.3423264.

Automated Selfish Mining Analysis for DAG-Based PoW Consensus Protocols

Patrik Keller

Abstract

Selfish mining is strategic rule-breaking to maximize rewards in proof-of-work protocols [4]. Markov Decision Processes (MDPs) are the preferred tool for finding optimal strategies in Bitcoin [7, 16] and similar linear chain protocols [22]. Protocols increasingly adopt DAG-based chain structures [19], for which MDP analysis is more involved [3]. To date, researchers have tailored specific MDPs for each protocol [3, 7, 8, 12, 16, 22]. Protocol design suffers long feedback loops, as each protocol change implies manual work on the MDP. To overcome this, we propose a generic attack model that covers a wide range of protocols, including Ethereum Proof-of-Work, GhostDAG [17], and Parallel Proof-of-Work [11]. Our approach is modular: we specify each protocol as a concise program, and our tooling then derives and solves the selfish mining MDP automatically.

Version Notes

This paper was published in the proceedings of the *23nd International Conference on Applied Cryptography and Network Security, ACNS 2025, in Munich, Germany.* I here reproduce the latest version, with minimal formatting adjustments applied to fit the body of this work, but I recommend reading the freely available preprint instead: `https://arxiv.org/abs/2501.10888`



The preprint is available in three versions. The latest version 3 corresponds closely to the published version, with one exception: the preprint corrects an error in the specification of the incentive scheme of Ethereum's Byzantium upgrade in Listing 3, Line 19. After re-evaluating the protocol, I conclude this change does not affect the results presented here.

1 Introduction

In the context of proof-of-work cryptocurrencies, "selfish mining" entails the deliberate deviation from the protocol to maximize personal rewards, e.g., by temporarily withholding blocks. The first selfish mining strategy was proposed by Eyal and Sirer in 2014 against Bitcoin [4]. Soon after, other researchers started to employ Markov Decision Processes (MDPs) to identify and evaluate optimal behavior [7, 16]. Using these strategies, attackers controlling more than a third of the system's hash rate can reap excessive rewards. Depending on the communication assumptions, the threshold is even lower.

In 2019, Zhang and Preneel [22] comprehensively mapped and analyzed various defense mechanisms. Their findings indicate that all designs for improving incentive compatibility compromise at least one of the other security properties. Many recent protocol attempt to close this gap by adopting a directed acyclic graph (DAG) structure [1, 11, 12, 15, 17]. But MDP-modeling requires enumerating

all feasible states, and doing this for arbitrary DAGs is not tractable. Hence, the DAG-based protocols have, so far, escaped MDP-based selfish mining analysis.

Fortunately, recent research has provided new options for analysing complex protocols. Probabilistic Termination [3] enables the *exact* solution of selfish mining MDPs with a tenfold efficiency improvement. When this approach meets its limits, one can turn to *approximating* reinforcement learning techniques [2, 8, 12], which can handle arbitrarily large state spaces, including infinite ones [18].

The remaining bottleneck arises from tailoring the MDPs. So far, this involved a lot of manual work, basically starting from scratch for each protocol [3, 7, 8, 12, 16, 22]. To overcome this, we propose a generic attack model that covers a wide range of DAG protocols. We specify each protocol as a concise program. Our tooling then generates the selfish mining MDP automatically.

We demonstrate our tooling by specifying and analyzing multiple protocols. Bitcoin allows to validate our approach against existing results. Ethereum Proof-of-Work and its Byzantium update serve as examples for (mostly) linear-chain protocols that carefully introduce DAG structure to adjust the reward scheme and chain preference rule. GhostDAG [17] and Parallel Proof-of-Work [11] represent the newer class of DAG-protocols that, so far, has been considered out of reach for MDP-based analysis. Notable results are that the optimal selfish mining policies against Ethereum, Byzantium, and GhostDAG involve creating more than one fork, and that relatively weak miners can benefit from selfish mining in GhostDAG. The most resilient protocol is Parallel Proof-of-Work.

We proceed as follows: Section 2 provides relevant background information on MDPs; Section 3 enumerates our assumptions and sketches the approach; Section 4 explains how we specify protocols; Section 5 documents how we search optimal selfish mining policies in automatically derived MDPs; Section 6 introduces two optimizations reducing the size of the state space; Section 7 documents all experiments and presents the results; Section 8 discusses our work and concludes.

2 Markov Decision Processes (MDP)

Finding incentive attacks against proof-of-work consensus protocols involves sequential decision-making, where an attacker must make a series of choices over time. The outcomes of these decisions are non-deterministic due to the inherently stochastic nature of mining. We use Markov Decision Processes (MDP) to model the problem and find the most profitable behavior.

An MDP is a probabilistic state machine. The *agent*, observes the current state of the system and chooses an *action*. The MDP defines probabilistic state transitions, specifying the probability distribution for the next state based on the current state and the chosen action. In our case, the agent models the attackers. The defenders are not agents, they react passively to the actions of the attacker.

A sequence of state transitions, starting from the (probabilistic) initial state until the process ends, is called an *episode*. Each state transition may have an effect, such as assigning a reward. The agent's goal is to maximize a function of the effects (objective function), in our case, expected mining rewards per time.

The strategy an agent uses to decide which action to take in each state is called a *policy*. Solving an MDP is about finding a policy that maximizes the objective function. For us, the optimal policy is the worst-case attack.

A good, textbook-sized introduction to the topic is provided by Sutton and Barto [18]. The book offers a formal mathematical description of MDPs and covers solving algorithms from first principles. These methods can be broadly categorized as follows:

Linear programming: The Bellman equations provide an optimality condition for the policy. This induces a linear programming problem which can be solved exactly.

Dynamic programming: Algorithms such as value iteration and policy iteration use the Bellman equations to define an iterative optimization step. Repeatedly applying this step converges to the optimal policy. When problem sizes become too large for linear programming, these methods offer practical solutions without losing much accuracy.

Approximate methods: Modern reinforcement learning (RL) techniques, such as Q-learning or policy gradient methods, train a simplified surrogate model by observing the probabilistic state machine. The resulting policies may not be optimal in the original MDP, but these methods are highly versatile: they are applicable when exact solutions are infeasible and effective even when the underlying MDP is unknown or changes over time.

Dynamic programming has been successfully applied to Bitcoin [3, 7, 16] and Ethereum Proof-of-Work [3]. Approximating methods have been applied to more complex protocols [8] or objective functions [2].

All these have in common that the researchers manually craft one MDP per problem and protocol. Our goal is to derive MDPs automatically from the protocol specification. We will use dynamic programming to provide optimal baseline results for small problem instances. Applying RL to solve bigger problem instances approximately is an option for future work.

3 Protocol-Generic Attack Space

We propose a protocol-generic state and action space (attack space) that is tractable for MDP-based analysis while still covering a useful range of attacks.

Assumptions

We achieve tractability by exploiting the following assumptions.

A1. Participants (miners) exchange blocks via reliable broadcast, and this is the only form of communication.

A2. Blocks reference other blocks by their content, inducing a directed acyclic graph (BlockDAG) with a single root block (genesis).

A3. Appending a new block to the DAG requires a proof-of-work. There is only one type of proof-of-work and the solving time is exponentially distributed.

A4. The broadcast primitive delivers the blocks in topological order, i.e., a block is delivered only after all its parents have been delivered.

A5. Honest behavior follows deterministically from the structure of the BlockDAG and the order in which the individual blocks were delivered.

A6. We focus on consensus protocols: in ideal conditions, excluding attacks and communication delays, all miners agree on an ordering of (a subset of) the blocks which we call *linear history*.

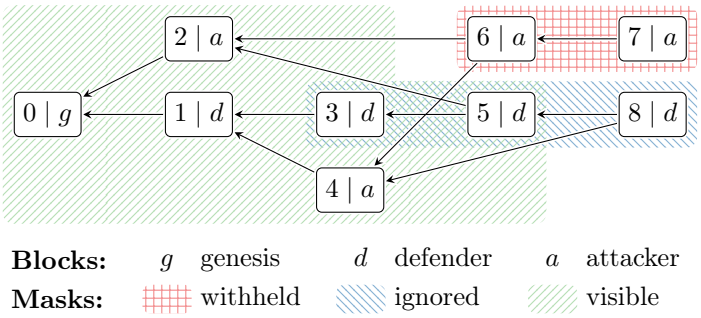


Figure 1: Example state. Vertices are blocks and arrows indicate parents. The available actions are `Release(6)`, `Consider(3)`, and `Continue`. Block 8 was just mined; it will become visible during the next `Continue` action.

A7. We model all dishonest participants as a single agent, the *attacker*.

A8. The attacker may withhold and ignore blocks but otherwise behaves honestly.

A9. We analyse selfish mining in the equilibrium state where the difficulty adjustment algorithm (DAA) has fully adjusted to the available hash rate and the attacker's policy.

State Space

The assumptions **A1**–**A3** and **A7**–**A9** allow us to track the relevant state in a few variables:

1. An adjacency list, describing the structure of the BlockDAG.
2. A vector describing which blocks where mined by whom.
3. An ignorance mask, describing which blocks are ignored by the attacker.
4. A withholding mask, describing which blocks are withheld by the attacker.
5. A visibility mask, describing which blocks have been delivered to the defender.
6. Protocol-specific local states of the defender and the attacker.

We do not consider the full cross product of these variables, as most combinations are invalid. Due to topological delivery (**A4**), a block can become visible only after all parents are visible. Similarly, a block must remain ignored while any parent is ignored. Naturally, the attacker can only withhold their own blocks. As the blocks are delivered to the defender in topological order anyways, we apply the topological constraints to the withholding mask as well. Additionally, the protocol-specific local states follow deterministically from the stepwise modification of the other variables (**A5**).

Figure 1 shows an example state. We visualize the BlockDAG and the three masks for ignoring, withholding, and visibility. We omit the miners' local states. Figure 7 in Appendix B shows another example with our attack space instantiated for the Bitcoin protocol.

Action Space

The original selfish mining MDP for Bitcoin uses only four actions (see Appendix B). We here use an (a-priori) unbounded number of actions and group them as follows.

Consider(b) to stop ignoring block b.
Release(b) to stop withholding block b.

Continue to wait until the next block is mined.

Induced by the topological restrictions of the state space, only of subset of actions is available at a time.

Bounding the State and Action Spaces

Exact MDP solving techniques work only if the number of states is finite. This is clearly not the case, as the BlockDAG grows continuously. We address this by truncating parts of the BlockDAG that are irrelevant for future protocol execution: stale blocks and the prefix of the linear history that all miners already agree upon (common history). Moreover, we define a limit on the number of blocks in the BlockDAG. In states that meet this limit, we force the attacker to behave honestly taking away the options to withhold and ignore blocks. By the consensus assumption **A6**, the miners will agree on a linear history. The common history will be truncated and the policy loops back to a smaller state. This bounds the number of actions as well.

Discussion of the Assumptions

Most of the assumptions **A1**–**A9** are uncontroversial and backed by related work. We here give special attention to the last three and defer the rest of our arguments to Appendix A.

Assumption **A7** restricts our model to a single attacking agent. This is conservative, as it allows dishonest participants to collude and coordinate their attack. This assumption is common practice since defenders typically cannot rule out collusion among the attackers [6]. To address scenarios where attackers do not collude, our model can be expanded for multiple agents. We think the main challenge here lies in defining a suitable scenario. In the most obvious case where all participants act rationally, security collapses entirely [5].

In Assumption **A8**, we force the attacker to base their attack on ignoring and withholding blocks alone. We not think this is overly restrictive:

First, as we lay out in Appendix B, our attack space fully covers selfish mining against Bitcoin. We confirm this numerically in Section 7.2.

Second, we deem it necessary from a practical point of view. It allows us to derive the structure of the BlockDAG from honest behavior. The alternative approach, starting from arbitrary blocks, checking them against the protocol's block validity rule, and then enumerating all valid choices, is not tractable.

Third, using local information alone, withholding is indistinguishable from benign network delays. In the absence of network delays and withholding, local knowledge is global knowledge and consensus is easily solved. We think, dealing with withholding is the key aspect of proof-of-work consensus.

Assumption **A9** encapsulates the details of difficulty adjustment. The DAA observes timestamps in the linear history and, in response, adjusts the difficulty of the proof-of-work puzzle. The goal is to maintain a (roughly) constant growth rate for the linear history, even though the available hash rate changes over time. This creates a feedback loop from the result of consensus (linear history, time) to the communication primitive (rate-limited by proof-of-work).

Expectedly, the attacker's actions affect the difficulty in the long term. In fact, selfish mining in Bitcoin becomes profitable (expected reward per time) only after the DAA has adjusted to the policy.

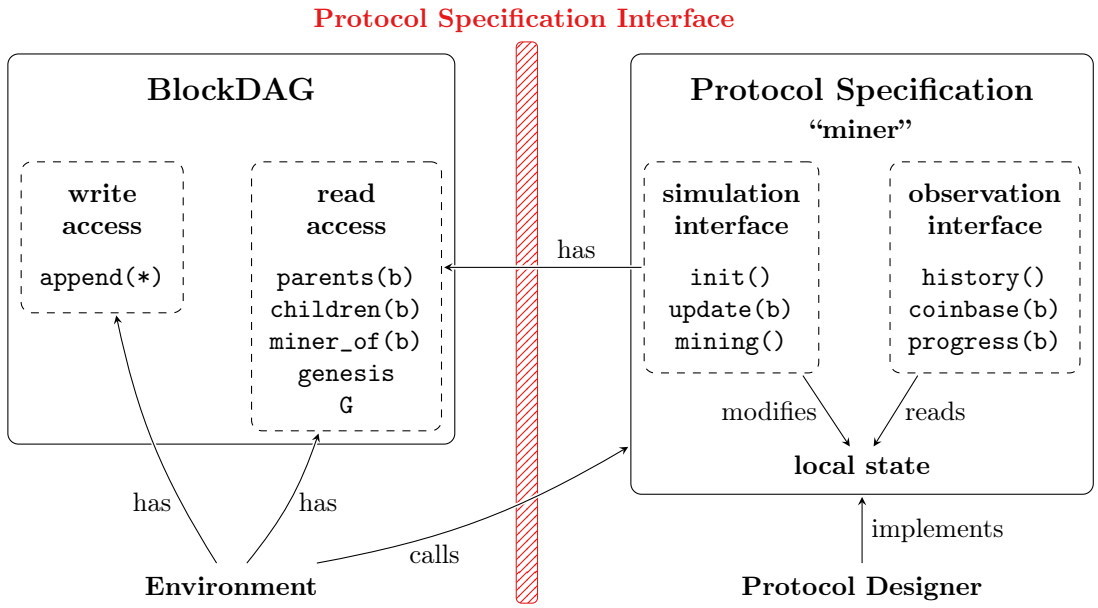


Figure 2: The protocol designer specifies the protocol according to the protocol specification interface (Section 4.1). Our tooling (the environment) derives an MDP and searches the optimal policy for selfish mining (Section 5).

We are aware that the DAA can be gamed, e.g., by faking time stamps [20] or tactically leaving and re-joining the system [14, 21]. Following related work [3, 4, 16], we keep our model tractable by assuming the DAA is in a stable state.

4 Specification of Protocols

Our analysis framework is generic with respect to the protocol. We here explain how the protocols are specified. The other sections assume the protocol specification as an input.

The *protocol designer* specifies the protocol by implementing the mining software in a constrained programming *environment* which provides essential functionality like proof-of-work, reliable broadcast, and the BlockDAG. The *protocol implementation interface* is depicted in Figure 2.

4.1 Protocol Specification Interface

The protocol designer specifies the miner's behavior as Python functions, which the environment calls during execution. We chose Python over pseudocode as this avoids ambiguity, enables automated testing and formatting, and closes the gap between the paper and the analytical code: the listings in this paper are the actual protocol implementations we feed into the automated analysis.

4.1.1 BlockDAG

The specification can access the BlockDAG as follows.

`parents(b)`, `children(b)` return the parents and children of block `b`.
`miner_of(b)` returns the miner of block `b` and `me` identifies the miner themselves.

`genesis` is the unique root block and `G` is the set of all blocks.
`height(b)` returns the distance of block `b` to `genesis`.

Later, we use additional DAG-related terminology:

- The *past* and *future* refer to the transitive closures of the parent and children relationships. The past of block B includes the parents of B as well as the parents of any other block in the past of B. The future is defined respectively for the children relationship.
- We say block A confirms block B to imply that B is in the past of A.

Recall from Section 3, that we restrict the visibility of blocks for the defender and let the attacker ignore blocks. This happens in a topological manner: `parents(b)` always returns all parents, `children(b)` and `G` might be subsets.

Also note that this interface does not allow direct modifications of the BlockDAG—only the environment can append new blocks.

4.1.2 Simulation Interface

The protocol designer implements three functions: `init`, `update`, and `mining`, which together model the miner's behavior. These functions can persist state across calls by setting attributes on the `state` object. Attribute names can be chosen freely, but we assume the values are hashable.

`init()` initializes the miner's local `state`. The environment calls this at the beginning of the protocol execution. The function has no return value.

`update(b)` updates the miner's local `state` with respect to a new block `b`. The environment calls this function to deliver blocks right after they become visible or stop being ignored. The function has no return value.

`mining()` returns the parents of the next block to be mined. This specifies how the miner *intends to* extend the BlockDAG. The actual extension is left to the environment and happens only if the miner was successful at proof-of-work.

Reflecting our assumptions in Section 3, the protocol designer assumes that new blocks are automatically and reliably dissipated to all miners (**A1**). Delivery happens in a topological order (**A4**).

4.1.3 Example (Bitcoin)

In Bitcoin, the miner extends the longest chain of blocks, one block at a time. The `init()` function initializes `state.head = genesis`. The `update(b)` function checks whether the new block `b` has a longer history than `state.head`, and if so, updates `state.head = b`. The `mining()` function returns `state.head` as the single parent for the new block. We specify this in Listing 1, Lines 1–9.

4.1.4 Observation Interface

The protocol designer implements three functions: `history`, `coinbase`, and `progress`, which inform the analyst about the miner's state. These functions can read but not modify the `state` object.

`history()` returns a list of blocks, the miner's opinion on the linear history.

Listing 1 Specification of the Bitcoin Protocol

```
1  def init():
2      state.head = genesis
3
4  def mining():
5      return {state.head}
6
7  def update(block):
8      if height(block) > height(state.head):
9          state.head = block
10
11 def history():
12     def history_of(block):
13         if block == genesis:
14             return [genesis]
15         else:
16             return history_of(parents(block).pop()) + [block]
17
18     return history_of(state.head)
19
20 def progress(block):
21     return 1
22
23 def coinbase(block):
24     return [(miner_of(block), 1)]
```

`coinbase(b)` defines the rewards allocated by block b. It returns a list of tuples `(r, v)`, where `r` denotes the recipient and `v` the size of the reward.

`progress(b)` defines the feedback for difficulty adjustment. It returns a non-negative number defining the progress of block b. We assume the linear history grows at a constant rate, denoted in (expected) progress per time (**A9**).

4.1.5 Example (Bitcoin, cont'd)

In Bitcoin, the linear history is the longest chain of blocks. For each block `b` in the linear history, `coinbase(b)` assigns a unit reward to the miner of `b`. Similarly, there is one unit of progress per block. We specify this in Listing 1, Lines 11–24. Note, `set.pop()` removes and returns an arbitrary element of `set`. This is deterministic because the set is singleton.

4.2 Overview of the Specified Protocols

Besides Bitcoin, we have specify Ethereum Proof-of-Work, its Byzantium update, GhostDAG [17], and Parallel Proof-of-Work [11]. We here provide short descriptions for each, shortly highlighting the core design ideas, and defer the full listings to Appendix C.

4.2.1 Ethereum

can be seen as an extension to Bitcoin. Like in Bitcoin, miners append their blocks to the longest chain, the history is the longest chain, and each block on the longest chain contributes one unit of progress for difficulty adjustment. New is that miners reference blocks off the longest chain as uncles

and that these uncles get a full block reward. This compensates the miner's of blocks that would have missed their reward in Bitcoin.

Ethereum sets a time horizon h, expressed in numbers of blocks added to the longest chain, before which uncles have to be merged into the main chain. Otherwise, they become orphans like in Bitcoin. The Ethereum cryptocurrency was deployed with $h = 7$, we set $h = 3$ throughout the paper.

We provide the full specification in Listing 2 in Appendix C.

4.2.2 Byzantium

was deployed as an upgrade to the Ethereum cryptocurrency. The overall chain structure stays intact, with the sole exception that a block can reference at most two uncles. Besides that, Byzantium modifies the consensus rule, progress, and rewards. Miners now extend the *heaviest* chain, where uncles add the same weight as regular blocks. Similarly, uncles add the same progress as regular blocks. Uncle rewards are now discounted based on the distance to the block merging them into the chain, while the merging block gets a small bonus reward for each uncle. Like Ethereum, Byzantium was deployed with horizon $h = 7$. We set horizon $h = 3$ throughout the paper. The full specification is available in Listing 3 in Appendix C.

4.2.3 GhostDAG

breaks with the notion of a longest or heaviest chain—miners just confirm all unconfirmed blocks. The total order is established after the fact, for all blocks in the DAG. GhostDAG comes with an heuristic for detecting longer phases of uncooperative behavior (private mining). Blocks contributed by the (honest) majority are colored blue, the others red. In the linear history, blue blocks take precedence over red blocks. Blue blocks allocate one unit of reward to their miner, blue blocks none. Similarly, blue blocks contribute one unit of progress to the chain, red blocks none. Note that the publication [17] does not specify rewards and progress—we fill these gaps.

The heuristic for coloring blocks can be tuned through a natural number parameter k. Higher k allows for more parallelism in the DAG without punishing the minority miners. The extreme setting $k = 0$ roughly resembles Bitcoin. We set $k = 3$ throughout the paper. The full specification is available in Listings 4 and 5 in Appendix C.

4.2.4 Parallel Proof-of-Work

uses two categories of blocks: votes and summaries. Appending the next summary requires referencing k votes confirming the parent summary. The summaries form a linear chain and this chain constitutes the linear history. As long as there are not enough votes for the next summary, miners append votes confirming the longest chain. In case of ties, miners vote for the chain with the most votes already. All blocks, both votes and summaries, assign a unit reward to their miner and contribute a unit progress to the difficulty adjustment.

Like in GhostDAG, higher k allows for more parallelism. The minimal setting $k = 1$ roughly resembles Bitcoin. We set $k = 3$ throughout the paper.

In the original proposal [11] summaries are not mined. Instead there is a form of leader election: only the miner of the smallest vote (by hash) may append the next summary. Later, the author has proposed a simplification [9] that works without public key cryptography. Here, the summaries are

mined like votes and this proof-of-work is the only form of authorization. This is what we specify and analyse in this paper. The full specification is available in Listing 6 in Appendix C.

5 Automated Attack Search

We now integrate the protocol-generic attack space from Section 3 with the protocol specification in Section 4. We will derive and solve MDPs that allow us to study the protocol's susceptibility to selfish mining, in a fully automated way.

Recall from Section 2 that an MDP is a probabilistic state machine, where the inputs are called actions and the outputs are effects. On each step of execution, the attacker observes the state and chooses the next action. The attacker's objective is a function of the effects, in our case, maximizing rewards per time.

Assuming the protocol specification is given as input, the automated analysis pipeline is as follows. We first define the transition function, which takes the current state and the chosen action as input and returns a discrete probability distribution for the next state (Section 5.1). After bounding the state space (Section 5.2) and defining the transitions' effects (Section 5.3), we derive an explicit tabular MDP (Section 5.4). Lastly, we define the objective function (Section 5.5) and find the optimal policy (Section 5.6).

5.1 State Transition Function

Recall from Section 3, that we model two miners, attacker and defender. The attacker ignores and withholds blocks using three types of actions `Consider(b)`, `Release(b)`, and `Continue`. We describe the associated state transitions as a function that modifies the current state in response to the chosen action.

If the attacker chooses to `Consider(b)`, we remove block `b` from the set of ignored blocks and deliver it by calling the attacker's `update` function with argument `b`. This transition is deterministic. The action is only possible, if `b` is currently ignored and none of the parents of `b` are ignored.

`Release(b)` removes block `b` from the set of withheld blocks. This is only possible, if `b` is currently withheld and none of the parents of `b` are withheld. The transition is deterministic. Note, the defender does not learn about `b` yet because communication is delayed.

`Continue` advances time until the next block is mined. This action is always available. It induces one out of four probabilistic transitions, depending on whether the attacker communicates quickly (probability γ) and who mines the next block (probability α for the attacker).

During the `Continue` action, the defender learns about all blocks that are neither withheld nor yet visible. Depending on the attacker's luck regarding message reordering, we modify the order of delivery. With probability γ, we deliver the attacker's blocks before the defender's blocks. Otherwise, with probability $1 - \gamma$, we deliver the defender's blocks first. We deliver the blocks by calling the defender's `update` function. This always happens in topological order (**A4**).

After communication, we append a new block. With probability α, we call the attacker's `mining` function and mark the new block withheld and ignored. Otherwise, with probability $1 - \alpha$, we call the defender's `mining` function and mark the new block ignored. Note, even if the defender mines the new block, it only becomes visible during the next round of communication. This enables us to model all honest system participants as a single miner.

As we lay out in Appendix B, our communication assumptions align with the traditional selfish mining model against Bitcoin [16]. Similarly, the traditional actions, *Wait*, *Adopt*, *Match*, and *Override*, can be reproduced with a combination of our actions, `Consider(b)`, `Release(b)`, and `Continue`.

5.2 State Space Cutoff

The state transition function induces an infinite number of states: the BlockDAG grows monotonically, one block per `Continue` action. Blocks are not removed and, throughout an episode, no state is visited twice.

Exact MDP solving techniques assume a finite number of states. We thus impose a limit on the size of the BlockDAG (number of blocks). In states that meet this limit, we force *honest behavior*. We make sure that all remaining policies loop within the restricted state space, by *removing stale blocks* and *truncating the common history*. The details are as follows.

Honest Behavior

We define the honest policy as follows:

1. If any block b is withheld, then `Release(b)`.
2. If any block b is ignored, then `Consider(b)`.
3. Else, `Continue`.

In states where the BlockDAG meets the size limit, we enforce this policy by taking away all other actions. The attacker becomes honest and, by assumption **A6**, the miners will agree on the linear history.

Truncating the Common History

After applying the state transition function (Section 5.1), we identify irrelevant parts of the BlockDAG and remove them. Our heuristic is as follows:

1. Obtain the linear histories of both miners, attacker and defender.
2. Find the longest common prefix of the linear histories (common history).
3. Find the last block `b` in the common history such that none of the blocks in `past(b)` have any children outside of $\{\texttt{b}\} \cup \texttt{past(b)}$.
4. Remove all blocks in `past(b)` from the BlockDAG.

Point 3 preserves the semantics of the BlockDAG: block `b` becomes the new genesis (**A2**) and none of the remaining blocks lose any of their parents.

Some protocols leave behind unconfirmed (stale) blocks, e.g., orphans in Bitcoin. Stale blocks violate the condition in Point 3 and effectively prevent common history truncation. We hence remove them in advance.

Removing Stale Blocks

We identify and remove state blocks as follows:

1. Mark all blocks that are invisible to either miner, `Honest` or `Attacker`.
2. Mark all blocks that either miner would reference next (`mining` function).
3. Mark all blocks that are in the past of any block marked before.
4. Remove all *unmarked* blocks from the BlockDAG.

Point 1 is a safeguard against removing blocks not relevant now but in the future.

5.3 Effects: Reward and Progress

Each state transition has two effects: reward and progress. We measure these on the common history, just before truncating it. We iterate the blocks on the common history, apply the `coinbase` and `progress` functions to each, and accumulate the results. The transition's reward is the accumulated reward assigned to the attacker. The transition's progress is the accumulated progress.

5.4 State Exploration

After state space truncation, the state transition function spans a finite attack space. We now explore all reachable states, noting the available actions, and tabulating the probabilistic state transitions together with their effects. This operation yields an *explicit* (or tabular) MDP suitable for policy search.

Importantly, we evaluate the (expensive) state transition function once for each input. Our attack search uses the cached results. Also note, that we can update the model parameters, α and γ, in the explicit MDP without re-evaluating the state transition function, because only five distinct probabilities are involved.

5.5 Objective Function: Reward per Progress

Selfish mining is about maximizing the expected reward per time, after the DAA has adjusted to the attack. As discussed in Section 3, the DAA creates a complex feedback loop between the miners' clocks, the overall hash rate, and the attacker's behavior. Assumption **A9** allows us break this loop: the DAA is in a stable state already and the expected progress per time fixed. We hence can treat *reward per progress* as a proxy for reward per time.

Note that Bitcoin, GhostDAG, and Parallel, assign exactly one unit of reward per unit of progress. For these protocols, reward per progress equals relative reward, the objective function used in the traditional models [3, 16].

We use reward per progress to capture the differences to Ethereum and Byzantium. In Ethereum, uncles are assigned full rewards but they do not count for progress. Byzantium discounts uncle rewards, partially reallocating some of the discount to the merging block, while all blocks induce the same amount of progress. For both protocols, relative reward does not capture all details.

5.6 Probabilistic Termination Optimization

Exact MDP solving algorithms expect that the objective is a linear function of the transition effects [3, 16]. Our objective function, reward per progress, is not linear. However, as the same is true for relative reward, we can simply turn to the established solving techniques for the Bitcoin MDP [3, 16].

We use probabilistic termination optimization (PTO) [3]. In short, the trick is to transform the (source) MDP, where all policies loop, into an MDP where all episodes terminate. In the terminating MDP, the expected progress per episode (horizon) is sufficiently independent from the policy. The authors prove that, as the horizon grows to infinity, any reward-maximizing policy in the terminating MDP also maximizes the reward per progress in the source MDP.

We derive terminating MDPs with horizon 100. Then we apply a standard dynamic programming algorithm, value iteration, to obtain an 10^{-4}-optimal policy. We then apply this policy to the *source* MDP and find its steady state using linear equations. After calculating the expected reward *and* progress at the steady state, we obtain the expected reward *per* progress by dividing those numbers. This is what we report in Section 7.

6 State Space Optimizations

We propose two optimizations to reduce the size of the state space: canonizing the BlockDAG and forcing the attacker to consider their own blocks.

Canonization

We use adjacency lists to track the evolution of the BlockDAG over time. The blocks are enumerated in the order in which they were mined (compare Figure 1). However, for proof-of-work consensus protocols, the exact mining order does not matter. In fact, the true mining order is unknown to the participants; they rely on consensus to establish an order in the first place.

So, our default BlockDAG implementation is bound to create redundant states. E.g., the blocks in Figure 1 can be relabelled in 69 ways without breaking the topological constraints (parents have lower ids than their children) and there are 35 options for the blocks in Figure 7 in Appendix B.

We optionally merge these states during state exploration (Section 5.4) by relabelling the blocks canonically: We first derive a vertex coloring from the relevant block properties (who mined it and whether it is visible/ignored/withheld). We then find a canonical and color-preserving vertex labeling using appropriate third-party software, Nauty [13]. The protocol designer may provide assistive input for canonization by implementing a function that colors blocks depending on the miner's local state.

Note that the GhostDAG protocol assumes a deterministic topological ordering of blocks. We specify this using Python's builtin `hash` function (Lines 13 and 18 of Listing 4 in Appendix C). Canonization does not preserve these hashes (blocks are integers) and thus is an invalid operation for GhostDAG.

Force Consider

The traditional selfish mining model against Bitcoin [16] allows at most two distinct chains (one fork). In our attack model, the attacker can create many forks, by temporally ignoring their own blocks.

We optionally adopt the traditional behavior and force the attacker to consider their own blocks. This happens during the same transition in which the new block has been mined (as opposed to enforcing a consider action after the transition).

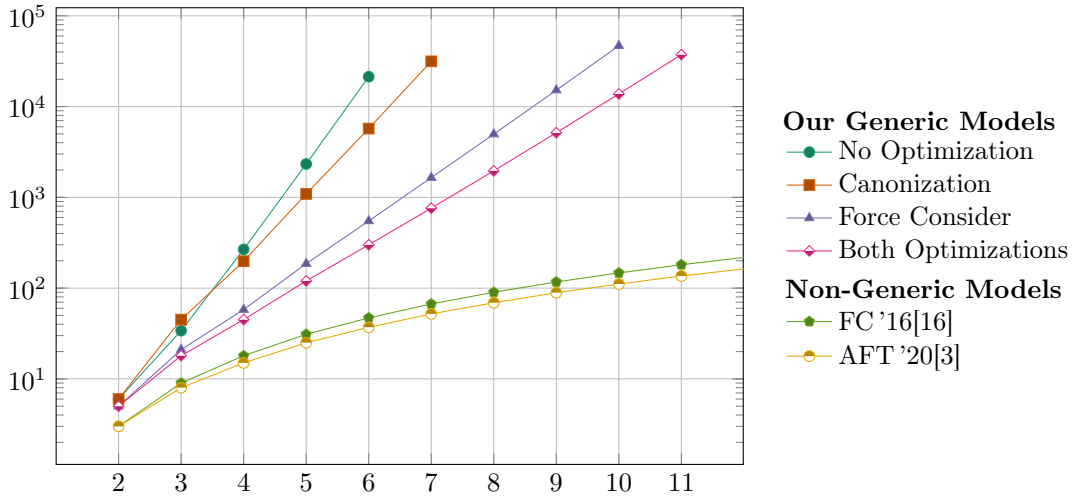


Figure 3: Number of states of the Bitcoin MDP (y-axis) as a function of the BlockDAG's size limit (x-axis, number of blocks). We compare our model with optimizations turned on and off and add the traditional non-generic models [3, 16] as a reference.

7 Results

We now apply the automatic analysis pipeline (Section 5) to the specified protocols (Section 4.2). We first focus on the size of the state space (Section 7.1). Then we validate our modeling choices (Section 3 and 5) against established models for Bitcoin (Section 7.2). This is followed by a comparison of the protocols to determine which is the least susceptible to selfish mining (Section 7.3). Lastly, we evaluate whether and how our state space optimizations (Section 6) affect the results on selfish mining (Section 7.4).

Reference Models

Throughout this section, we compare our generic model to the traditional models specific to Bitcoin [3, 16]. We obtain the reported numbers from implementing the state transition functions and effects as described in the respective papers (comp. Section 5.1 and 5.3). The remaining parts of the analysis are the same across models: we restrict the state space based on the number of blocks (Section 5.2), build a tabular MDP (Section 5.4), and optimize the policy for selfish mining (Sections 5.5 and 5.6).

7.1 Size of the State Space

Recall from Section 5.2, that we limit the number of blocks in the BlockDAG to bound the number of states in the MDP. We expect the state size to grow as we increase the BlockDAG size limit. We now evaluate how fast.

We do this, by applying the analysis pipeline to the specified protocols, skipping the attack search itself. Instead we observe the number of states in the resulting MDP. We start with a BlockDAG size limit of 2 and increase it one-by-one, until the state size exceeds 100 000 states. This upper limit ensures that we can reproduce all results, including the following sections, within five hours on our consumer grade laptop ($\approx$ \$ 1 000).

Figure 3 visualizes the growth of the state space for Bitcoin, with state space optimizations turned on and off (Section 6). The optimizations are effective: when applying them both, we exceed the state size limit after 11 instead of 6 blocks. As a reference, we compare our generic model to the traditional models specific to Bitcoin [3, 16]. Expectedly, the specific models have much smaller state sizes. Note that the plot is truncated on the right: the traditional models exceed the state size limit after 239 and 283 blocks respectively.

Table 1 expands this analysis for the other protocols, excluding the invalid optimization, canonization on GhostDAG. We report the maximum number of blocks explored ($s_{\max}$) before exceeding the state size limit of 100 000 states, and the number of states (n_6) when limiting the BlockDAG size to 6 blocks. For reference, we could explore the traditional model [16] up to $s_{\max} = 239$ before reaching 100 000 states, and it has $n_6 = 47$ states when limiting the BlockDAG to 6 blocks. For the second reference model [3], $s_{\max} = 283$ and $n_6 = 37$.

7.2 Validation Against Traditional Models for Bitcoin

We now validate our protocol-generic model against existing models specific to Bitcoin [3, 16]. We take 30 MDPs from Figure 3: our model with both optimizations turned on, the two reference models, and BlockDAG size limits ranging from 2 to 11 blocks. We then apply the remaining analysis, policy optimization and evaluation (Section 5.6), with model parameters $\alpha = 0.33$ and $\gamma \in \{0.33, 0.66\}$. Figure 4 reports the results, denoted in (expected) reward per progress.

We make three observations: First, reward per progress grows monotonically for increasing BlockDAG size limit, independent of the model. This indicates that the BlockDAG size limit restricts the attacker. In a real setting, there is no BlockDAG size limit and the attacker's reward per progress could be even higher. Second, using the FC '16 model [16] as baseline, the error of our generic model is comparably sized to the error of the AFT '20 model [3]. Third, the effect of increasing the BlockDAG size limit by one, e.g. from 10 to 11, dominates the remaining differences between the models. All three models appear to converge on the same result.

7.3 Comparison of Protocols

We now evaluate selfish mining across all specified protocols: Bitcoin, Ethereum, Byzantium, GhostDAG, and Parallel Proof-of-Work. We take the 5 MDPs from Table 1, where the BlockDAG

Table 1: State size scaling for all protocols and state space optimizations.

	No Opt.		Canonization		Force Consider		Both Opt.	
Protocol	$s_{\max}$	n_6	$s_{\max}$	n_6	$s_{\max}$	n_6	$s_{\max}$	n_6
Bitcoin	6	21327	7	5724	10	549	11	300
Ethereum	6	32961	7	10293	9	1179	10	596
Byzantium	6	33016	7	9879	9	1109	10	572
GhostDAG	6	46966			8	1527		
Parallel	7	9122	8	1654	8	2050	12	462

We grow the BlockDAG size limit (number of blocks), until the resulting model exceeds 100 000 states. We here report the maximum BlockDAG size limit ($s_{\max}$) before reaching this limit, as well as the number of states (n_6) at a fixed size limit of 6 blocks. We omit the results for the invalid optimization, canonization on GhostDAG.

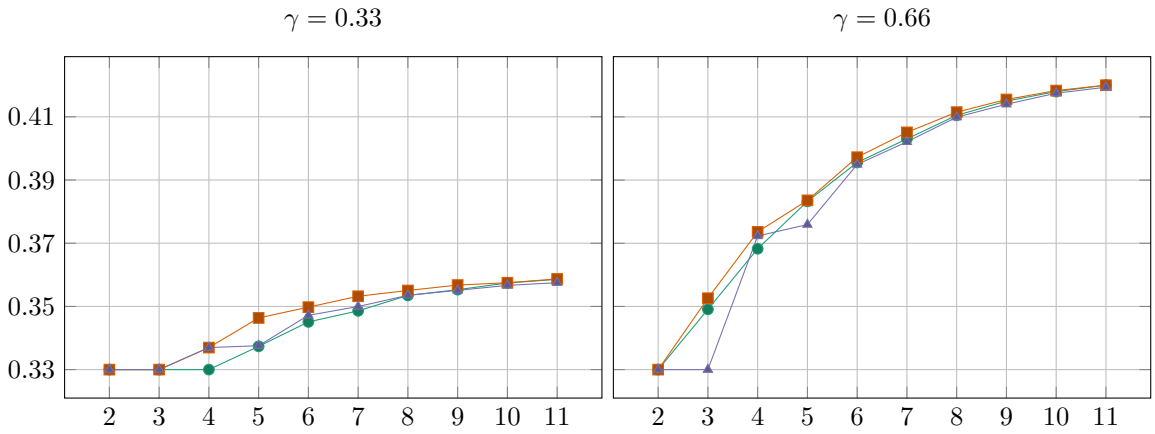


Figure 4: Reward per progress of the optimal selfish mining policy against Bitcoin (y-axis) as a function of the BlockDAG size limit (x-axis, number of blocks). We compare our model (—●—) to the non-generic models presented at FC '16 [16] (—■—) and AFT '20 [3] (—▲—). We set the attacker's relative hash rate $\alpha = 0.33$ and communication advantage $\gamma \in \{0.33, 0.66\}$ (left/right). On our model, we enable BlockDAG canonization and force the attacker to consider their own blocks. Line $y = 0.33$ represents honest behavior.

is limited to 6 blocks (n_6 column) and both optimizations are turned off. We then apply policy optimization and evaluation (Section 5.6), with model parameters $\alpha \in \{0.05, 0.1, \ldots, 0.45, 0.5\}$ and $\gamma \in \{0.33, 0.66\}$. Figure 5 reports the results, denoted in (expected) reward per progress minus α. This represents the *surplus* of selfish mining as compared to honest behavior. Note that we truncate the $\alpha = 0.5$ result for Ethereum, to give more detail to the other protocols. The truncated result can be read from Figure 6 of the next experiment.

This comparison creates important insights for protocol design. First, the first Ethereum protocol is clearly off the table. Second, the fixes deployed with the Byzantium upgrade are effective. Compared to Bitcoin, Byzantium is less susceptible to selfish mining for most combinations of α and γ, and the difference is relatively small if not. Third, GhostDAG seems to be incentive incompatible for relatively weak attackers (low α) although it clearly outperforms Bitcoin and Byzantium for strong attackers. Forth, Parallel Proof-of-Work seems to hit a sweet spot, showing the lowest reward per progress for all parameter combinations.

This warrants further research. We are particularly interested in how GhostDAG's and Parallel's k-parameters affect the results and to which extent the results are skewed by our low BlockDAG size limit. We here set $k = 3$, while the designers of both protocols propose much higher values.

7.4 State Space Optimizations

We have presented two attack space optimizations in Section 6: canonization of the BlockDAG and forcing the attacker to consider their own blocks. We now check whether these optimizations affect the reward per progress after policy optimization. We take the 18 MDPs of Table 1, where the BlockDAG is limited to 6 blocks (all four n_6 columns). We then apply policy optimization and evaluation (Section 5.6), with model parameters $\alpha \in \{0.05, 0.1, \ldots, 0.45, 0.5\}$ and $\gamma \in \{0.33, 0.66\}$.

We do not observe substantial differences with the *Canonization* optimization turned on or off.

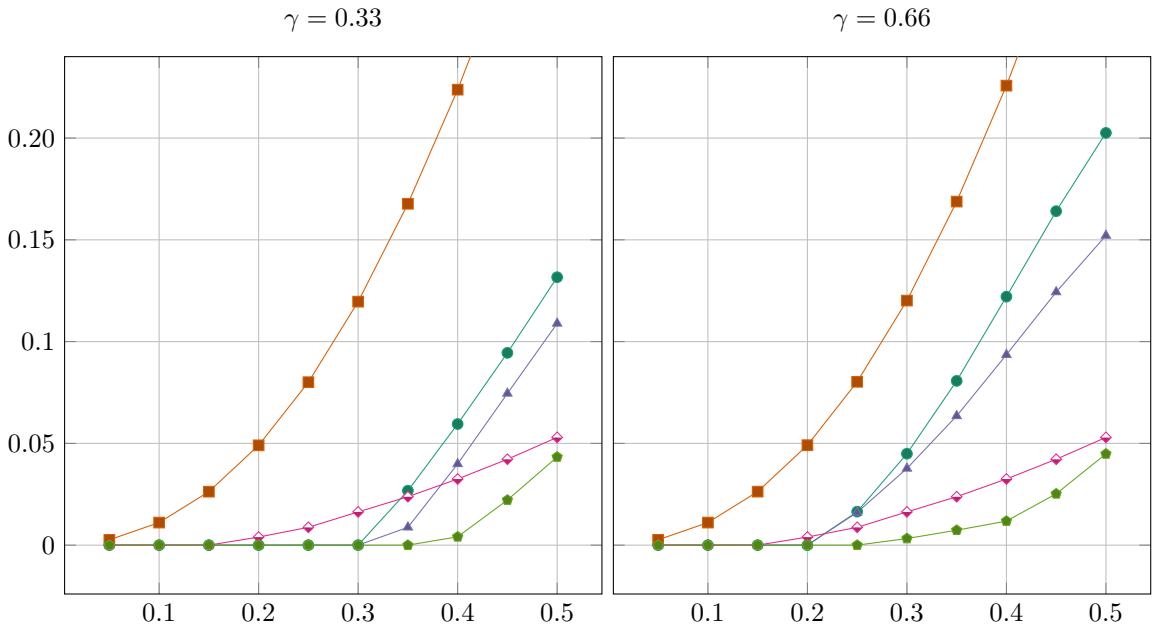


Figure 5: Surplus of the optimal policy (y-axis, reward per progress minus α) as a function of the attacker's relative hash rate α (x-axis). We compare the protocols Bitcoin (—●—), Ethereum (—■—), Byzantium (—▲—), GhostDAG (—◆—), and Parallel Proof-of-Work (—●—). All results are obtained from our generic attack model with optimizations turned off. We set communication advantage $\gamma \in \{0.33, 0.66\}$ (left/right) and limit the BlockDAG to at most 6 blocks. Line $y = 0$ represents honest behavior.

Some protocols, however, are sensitive to the *Force Consider* optimization: Ethereum, Byzantium, and GhostDAG. Figure 6 shows the differences for these protocols with *Force Consider* turned on and off. As before, we report the (expected) reward per progress minus α, which represents the *surplus* of selfish mining as compared to honest behavior.

We observe that *Force Consider* reduces the surplus. This suggest that the optimal selfish mining strategy against Ethereum, Byzantium, and GhostDAG involves ignoring own blocks, at least temporally. We also conclude that the *Force Consider* optimization has to be applied with care.

8 Discussion and Conclusion

Our implementation is optimized for correctness and ease of presentation. The results reproduce on a $1 000 laptop in a few hours. This comes at the cost of restricting the problem size (at most 100 000 states) and the size of the BlockDAG (6–11 blocks depending on model and protocol). This restricts the applicability of our tooling. E.g., the configuration of Parallel Proof-of-Work proposed by its authors [11] requests $k = 51$ parallel votes before growing the linear history by a single block. Restricting the size of the BlockDAG to anything below k is futile, and hence the proposed configuration escapes our treatment.

This raises the question, how far the problem size can be scaled after optimizing the implementation and reaching for more capable hardware. But considering the exponential growth in Figure 3, we think algorithmic advances are inevitable. Modern reinforcement learning algorithms, e.g. DQN

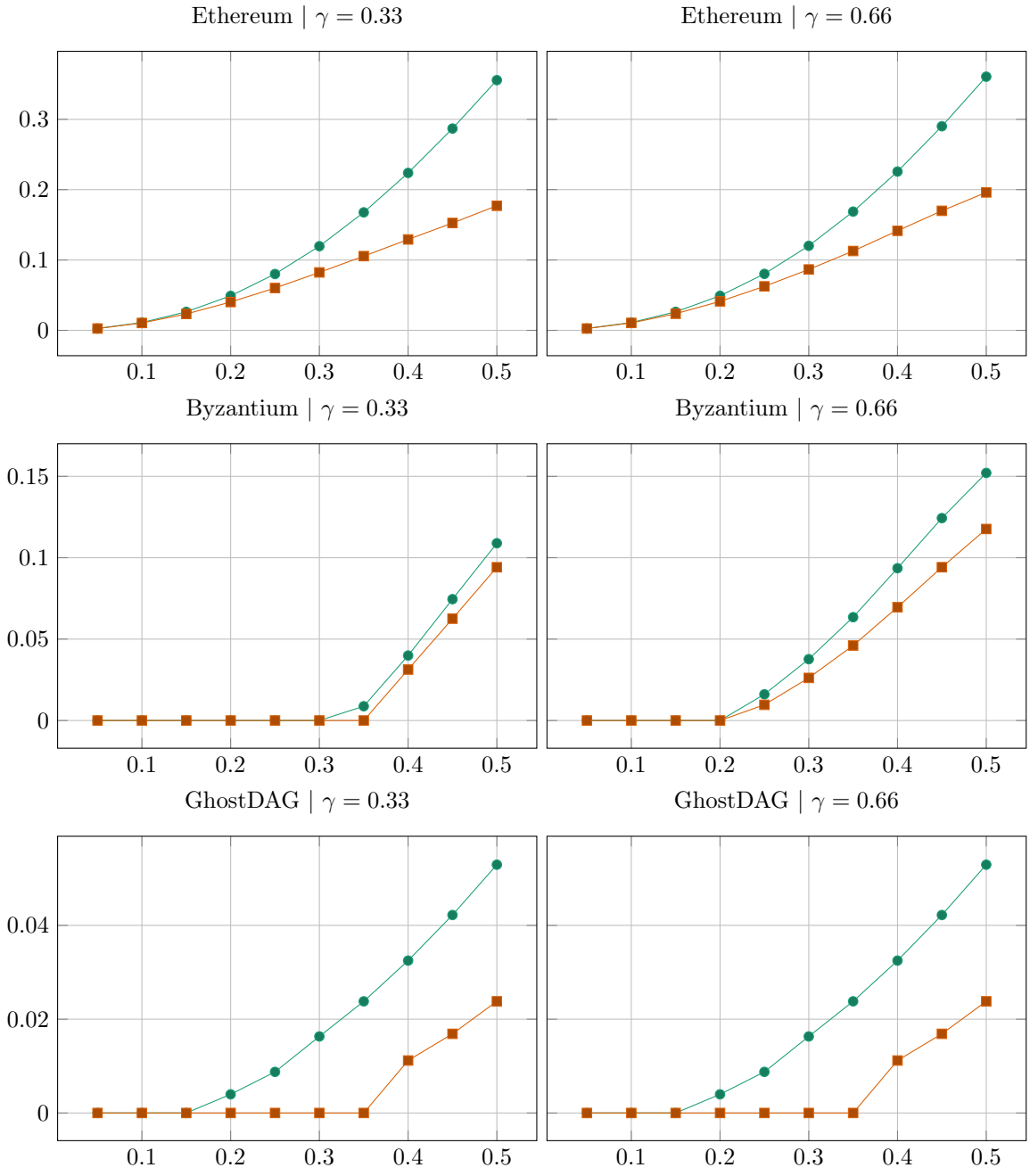


Figure 6: Surplus of the optimal policy (y-axis, reward per progress minus α) as a function of the attacker's relative hash rate α (x-axis). We compare our generic model **with** (—■—) and **without** (—●—) **forcing the attacker to consider their own blocks**. We report the results for Ethereum (top), its Byzantium upgrade (middle), and GhostDAG (bottom); the other protocols, Bitcoin and Parallel Proof-of-Work, are not sensitive to this optimization. Note the different scales on the y-axis and see Figure 5 for comparing results across protocols. We set communication advantage $\gamma \in \{0.33, 0.66\}$ (left/right) and limit the BlockDAG to at most 6 blocks. Line $y = 0$ represents honest behavior.

or PPO, should be an option, as they have been applied to similar problems before [2, 12]. In principle, such techniques allow for unbounded state spaces, potentially opening a path to removing the BlockDAG size limit altogether. Beware though, that this comes at the cost of optimality. Our approach provides optimal policies and can serve as a baseline for evaluating approximate results in the future.

This brings us to other avenues for future research. We here measure two effects of the attacker's policy, reward and progress on the common chain (Section 5.3), and analyse a single objective function, reward per progress (Section 5.5), modeling the long term incentives after the DAA has adjusted to the policy. We deem it worthwhile to observe more effects, e.g., number of blocks mined and number of blocks rewritten on the defender's chain, to enable other relevant objective functions, e.g., for modelling consistency, liveness, and short term incentives.

We also see low hanging fruits to extend the range of protocols supported. E.g., some protocols use a "two-for-one trick" where the proof-of-work puzzle can have multiple possible solutions [1, 15]. This can be modeled by coloring the blocks probabilistically, just when they are appended to the BlockDAG. The same mechanism would allow to (approximately) model hash-based precedence rules [12, 17] which, we think, can be gamed. Turning to precedence rules, the random-tie breaking proposed by the inventors of selfish mining themselves [4], to reduce the negative effects of high communication advantages γ on Bitcoin, should be an interesting case study as well. Specifying this would require probabilistic state `update` functions—simple as long as the number of outcomes is finite.

Conclusion

We have presented a framework for finding optimal selfish mining attacks against a wide range of proof-of-work consensus protocols. Our work reproduces established results for well studied protocols, Bitcoin and Ethereum, and also covers protocols that have not been analyzed before (GhostDAG) or only with approximate policy optimization techniques (Parallel Proof-of-Work). A key feature of our approach is its modularity: the protocol designer specifies the protocol as a concise Python program, the subsequent analysis, including generating the selfish mining MDP and optimizing the policy, follows automatically. This modularity, combined with the release of all our tooling [10], enables rapid iteration during protocol design.

Acknowledgements

This work is inspired by a previous collaboration with George Bissias [12]. I would like to thank Rainer Böhme, George Bissias, Michael Fröwis, and the anonymous reviewers for their valuable feedback on earlier versions of this article.

References

[1] Ittai Abraham, Danny Dolev, Ittay Eyal, and Joseph Y. Halpern. "Colordag: An Incentive-Compatible Blockchain". In: *37th International Symposium on Distributed Computing, DISC 2023, October 10-12, 2023, L'Aquila, Italy*. Vol. 281. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 1:1–1:22.

[2] Roi Bar-Zur, Ameer Abu-Hanna, Ittay Eyal, and Aviv Tamar. *WeRLman: To Tackle Whale (Transactions), Go Deep (RL)*. Cryptology ePrint 2022/175. IACR, 2022.

[3] Roi Bar-Zur, Ittay Eyal, and Aviv Tamar. "Efficient MDP Analysis for Selfish-Mining in Blockchains". In: *Conference on Advances in Financial Technologies*. ACM, 2020, pp. 113–131.

[4] Ittay Eyal and Emin Gün Sirer. "Majority Is Not Enough: Bitcoin Mining Is Vulnerable". In: *Financial Cryptography and Data Security*. Vol. 8437. LNCS. Springer, 2014, pp. 436–454.

[5] Bryan Ford and Rainer Böhme. "Rationality Is Self-Defeating in Permissionless Systems". In: *arXiv:1910.08820 [cs]* (2019). arXiv: `1910.08820 [cs]`.

[6] Juan Garay and Aggelos Kiayias. "SoK: A Consensus Taxonomy in the Blockchain Era". In: *Topics in Cryptology – CT-RSA 2020*. Vol. 12006. Lecture Notes in Computer Science. Springer, 2020, pp. 284–318.

[7] Arthur Gervais, Ghassan O. Karame, Karl Wüst, Vasileios Glykantzis, Hubert Ritzdorf, and Srdjan Capkun. "On the Security and Performance of Proof of Work Blockchains". In: *Conference on Computer and Communications Security*. ACM, 2016, pp. 3–16.

[8] Charlie Hou, Mingxun Zhou, Yan Ji, Phil Daian, Florian Tramèr, Giulia Fanti, and Ari Juels. "SquirRL: Automating Attack Analysis on Blockchain Incentive Mechanisms with Deep Reinforcement Learning". In: *Network and Distributed Systems Security Symposium*. Internet Society, 2021.

[9] Patrik Keller. "Parallel Proof-of-Work with DAG-style Voting and Targeted Reward Discounting". In: *arXiv:2312.03111 [cs]* (2023). arXiv: `2312.03111 [cs]`.

[10] Patrik Keller. *Protocol specifications, automated attack search, and reproducible experiments for this paper*. `https://github.com/pkel/cpr/tree/3ef74c0055eb745da437adce7736b677f816fafd`. 2025.

[11] Patrik Keller and Rainer Böhme. "Parallel Proof-of-Work with Concrete Bounds". In: *Conference on Advances in Financial Technologies*. ACM, 2022.

[12] Patrik Keller, Ben Glickenhaus, George Bissias, and Griffith Gregory. "Tailstorm: A Secure and Fair Blockchain for Cash Transactions". In: *Conference on Advances in Financial Technologies*. ACM, 2023.

[13] Brendan D. McKay and Adolfo Piperno. "Practical Graph Isomorphism, II". in: *Journal of Symbolic Computing* 60 (2014), pp. 94–112.

[14] Kevin Alarcón Negy, Peter R. Rizun, and Emin Gün Sirer. "Selfish Mining Re-Examined". In: *Financial Cryptography and Data Security*. Vol. 12059. LNCS. Springer, 2020, pp. 61–78.

[15] Rafael Pass and Elaine Shi. "FruitChains: A Fair Blockchain". In: *Symposium on Principles of Distributed Computing*. ACM, 2017, pp. 315–324.

[16] Ayelet Sapirshtein, Yonatan Sompolinsky, and Aviv Zohar. "Optimal Selfish Mining Strategies in Bitcoin". In: *Financial Cryptography and Data Security*. Vol. 9603. LNCS. Springer, 2016, pp. 515–532.

[17] Yonatan Sompolinsky, Shai Wyborski, and Aviv Zohar. "Phantom Ghostdag: A Scalable Generalization of Nakamoto Consensus". In: *Conference on Advances in Financial Technologies*. ACM, 2021, pp. 57–70.

[18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd edition. Bradford Books, 2018.

[19] Qin Wang, Jiangshan Yu, Shiping Chen, and Yang Xiang. "SoK: DAG-based Blockchain Systems". In: *ACM Comput. Surv.* 55.12 (2023), 261:1–261:38.

[20] Aviv Yaish, Gilad Stern, and Aviv Zohar. "Uncle Maker: (Time)Stamping out the Competition in Ethereum". In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*. ACM, 2023, pp. 135–149.

[21] Aviv Yaish, Saar Tochner, and Aviv Zohar. "Blockchain Stretching & Squeezing: Manipulating Time for Your Best Interest". In: *Proceedings of the 23rd ACM Conference on Economics and Computation*. EC '22. Association for Computing Machinery, 2022, pp. 65–88.

[22] Ren Zhang and Bart Preneel. "Lay down the Common Metrics: Evaluating Proof-of-Work Consensus Protocols' Security". In: *Symposium on Security and Privacy*. IEEE, 2019, pp. 1190–1207.

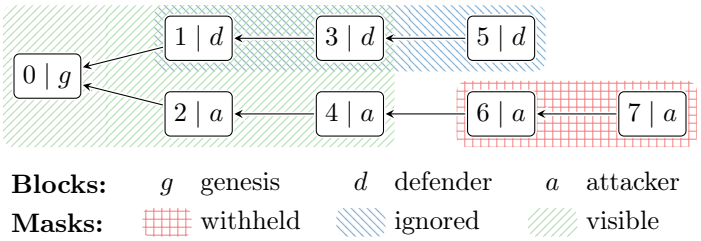


Figure 7: Example state for the Bitcoin protocol. Vertices are blocks and arrows indicate parents. The available actions are `Consider(1)`, `Release(6)`, and `Continue`. Block 5 was just mined; it will become visible during the next `Continue` action. To emulate the traditional *Match* action, the attacker would `Release(6)` and then `Continue`. To emulate the *Override* action, the attacker would `Release(6)`, `Release(7)`, and then `Continue`.

A Discussion of the Remaining Assumptions

Our work is based on the nine assumptions, **A1**–**A9**, introduced in Section 3. We have discussed the three most important ones, **A7**–**A9**, in Section 3 already, and now cover the rest.

To A1: We assume reliable (or P2P or Bracha) broadcast, as it is largely orthogonal to consensus. Modelling consensus on top of reliable broadcast is common practice [6].

To A1 and A2: All messages are blocks and these must be organized as a DAG. This deliberately restricts our scope. In particular, we avoid modelling individual transactions and out-of-band communication.

To A2: We assume a connected DAG with single root block. Otherwise we would have to consider permanent splits and could not truncate the common chain.

To A3: We assume that all blocks require the same proof-of-work. This is a deliberate restriction to keep the state space tractable. Some interesting protocols use more than one puzzle, e.g., Fruitchains [15] and Colordag [1]. Modelling this is possible by randomly coloring the blocks. This however, would add another exponential factor to the state size (all possible colorings).

To A4: We assume all blocks are delivered in topological order. Typically protocols set an inductive block validity rule: a block with an invalid parent is invalid itself. Hence miners verify the blocks in topological order. Mining to confirm unverified blocks is risky. Other protocol designs are certainly viable, but we deliberately ignore them.

To A5: We assume deterministic behavior for a given BlockDAG and fixed order of delivery. This bounds the number of possible protocol-dependent local states: at most one per random transition explored. We need this, to explore the state space by traversing all possible state-action-transition paths.

To A6: We assume the protocol is a consensus protocol in the first place. This enables to bound the state space, by enforcing honest behavior and truncating the common chain.

B Traditional Selfish Mining Model for Bitcoin

We now present the traditional selfish mining model against Bitcoin [16]. Like our attack model (Section 3), it features two miners: the attacker and the defender. The defender extends the longest chain, breaking ties in order first seen. The attacker can fork the chain, mine blocks in private, and

selectively release withheld blocks. The model is limited to one fork and there are at most two chains: the attacker mines the private chain and the defender mines the public chain. Figure 7 shows an example state in *our* generic model, reusing the notation of Figure 1.

The traditional model has four actions: *Adopt*, *Match*, *Override*, and *Wait*. The *Adopt* action discards the private chain. The attacker will start a new fork from the most recent public block. With *Match*, the attacker releases just enough blocks to induce a block race between the public and the (released subset of the) private chain. With probability γ the block race is resolved in favor of the attacker, that is, the defender discards the public chain and continues mining on the (released subset of the) private chain. Otherwise, the defender continues mining the public chain. The *Override* action is similar to *Match*, but the attacker releases one additional block. This *forces* the defender to discard the public chain with probability one. Lastly, the *Wait* action lets the attacker continue mining on the private chain without doing anything. *All actions* induce one block being mined: with probability α the private chain grows by one, otherwise the public chain grows by one.

Note how γ models the communication advantage of the attacker. Honest Bitcoin miners resolve ties in favor of the block first received. High γ implies that the attacker can, in reaction to an honest node mining and sending block H, deliver his own block A to the remaining defenders before they learn about block H. These defenders will then discard the public chain in favor of the private chain. The second parameter, α, models the attacker's relative hash rate.

A key feature of this selfish mining model is its simple state space when implemented as MDP: two non-negative integers describe the lengths of the private and public chain and a ternary variable tracks the feasibility of *Match*. In the example in Figure 7, there are 3 blocks on the public chain, 4 blocks on the private chain, and the *Match* action is feasible. The remaining information, e.g., block ids and the masks, is redundant.

The downside of such highly optimized MDPs is that they have to be tailored for each protocol individually. Our approach is to expand the state space such that it covers a wide range of protocols in one go. We do this, keeping the differences as small as possible: we do not introduce conflicting assumptions, use the same parameters (α and γ), and generalize the actions in a protocol-independent way.

C Protocol Specifications

This appendix provides the full protocol specifications that have been omitted in Section 4.2. Listing 2 specifies Ethereum Proof-of-Work. Listing 3 specifies Ethereum's Byzantium upgrade. Listing 4 specifies GhostDAG, with some helper functions deferred to Listing 5. Listing 6 specifies Parallel Proof-of-Work.

Listing 2 Specification of the Ethereum Protocol

```
h: int = 3

def init():
    state.head = genesis

def mining():
    return {state.head} | available_uncles()

def available_uncles():
    hist = history_of(state.head)
    allowed_parents = set(hist[-h - 1 : -2])
    uncles = set()
    leaves = {b for b in G if len(children(b)) == 0}
    for b in leaves:
        p, _ = parent_and_uncles(b)
        if p in allowed_parents:
            uncles.add(b)
    return uncles

def update(block):
    if height(block) > height(state.head):
        state.head = block

def history_of(block):
    if block == genesis:
        return [genesis]
    else:
        parent, _ = parent_and_uncles(block)
        return history_of(parent) + [block]

def parent_and_uncles(block):
    ranked = sorted(parents(block), key=lambda p: -height(p))
    if len(ranked) > 0:
        return ranked[0], set(ranked[1:])
    else:
        return None, set()

def history():
    return history_of(state.head)

def progress(block):
    return 1

def coinbase(block):
    _, uncles = parent_and_uncles(block)
    return [(miner_of(b), 1) for b in {block} | uncles]
```

Listing 3 Specification of the Byzantium Protocol

```
def mining():
    uncles = available_uncles()
    # choose at most 2 uncles, own blocks first
    ranked = sorted(uncles, key=lambda u: miner_of(u) != me)
    return {state.head} | set(ranked[0:2])

def update(block):
    prg_new = sum(progress(b) for b in history_of(block))
    prg_old = sum(progress(b) for b in history_of(state.head))
    if prg_new > prg_old:
        state.head = block

def progress(block):
    _, uncles = parent_and_uncles(block)
    return 1 + len(uncles)

def coinbase(block):
    _, uncles = parent_and_uncles(block)
    lst = [(miner_of(block), 1 + 0.03125 * len(uncles))]
    h = height(block)
    max_d = h + 1
    for u in uncles:
        d = h - height(u)
        lst.append((miner_of(u), (max_d - d) / max_d))
    return lst
```

The remaining functionality is inherited from Ethereum as specified in Listing 2.

Listing 4 Specification of the GhostDAG Protocol

```
k: int = 3

def mining():
    return tips(G)

def history_of(G):  # eprint.iacr.org/2018/104.pdf ; Alg. 1
    if len(G) == 1:
        return ({genesis}, [genesis])

    blue, hist = dict(), dict()
    for b in tips(G):
        blue[b], hist[b] = history_of(past(G, b))
    b_max = sorted(tips(G), key=lambda b: (-len(blue[b]), hash(b)))[0]
    blue = blue[b_max] | {b_max}
    hist = hist[b_max] + [b_max]

    ac = anticone(G, b_max)
    for b in sorted(ac, key=lambda b: (height(b), hash(b))):
        if is_k_cluster(G, blue | {b}):
            blue = blue | {b}
            hist = hist + [b]  # only blue blocks get a reward

    return (blue, hist)

def is_k_cluster(G, S):
    return all(len(anticone(G, b) & S) <= k for b in S)

def history():
    _blue, history = history_of(G)
    return history

# init() and fn update() do nothing
# progress() and coinbase() are like in Bitcoin
```

Listing 5 Specification of the GhostDAG Protocol (Utility Functions)

```
# protocol helper functions

def tips(subgraph):
    return {b for b in subgraph if len(children(b) & subgraph) == 0}

def past_or_future(relation, subgraph, block):
    acc = set()
    stack = set(relation(block)) & subgraph
    while len(stack) > 0:
        b = stack.pop()
        if b not in acc:
            acc.add(b)
            for p in set(relation(b)) & subgraph:
                stack.add(p)
    return acc

def past(subgraph, block):
    return past_or_future(parents, subgraph, block)

def future(subgraph, block):
    return past_or_future(children, subgraph, block)

def anticone(subgraph, block):
    return (
        subgraph
        - {block}
        - past(subgraph, block)
        - future(subgraph, block)
    )

# remaining protocol specification functions

def init():
    pass

def update(block):
    pass

def progress(block):
    return 1

def coinbase(block):
    return [(miner_of(block), 1)]
```

Listing 6 Specification of the Parallel Proof-of-Work Protocol

```
k: int = 3

def init():
    state.head = genesis

def mining():
    votes = children(state.head)
    if len(votes) >= k:
        # choose k votes, own votes first
        ranked = sorted(votes, key=lambda v: miner_of(v) != me)
        return set(ranked[0:k])
    return {state.head}

def update(block):
    if is_vote(block):
        block = parents(block).pop()
    if height(block) > height(state.head):
        state.head = block
    elif height(block) == height(state.head):
        if len(children(block)) > len(children(state.head)):
            state.head = block

def is_vote(block):
    return len(parents(block)) == 1

def history_of(block):
    if block == genesis:
        return [genesis]
    elif is_vote(block):
        return history_of(parents(block).pop())
    else:
        return history_of(parents(block).pop()) + [block]

def history():
    return history_of(state.head)

def progress(block):
    return k + 1

def coinbase(block):
    return [(miner_of(b), 1) for b in {block} | parents(block)]
```